

|      |           |
|------|-----------|
| 队伍编号 | MC2601946 |
| 题号   | D         |

## “箱”尽其用——多约束下货物运输装箱的分层组合优化与多车型决策

### 摘 要

本文针对多场景、多目标货物运输装箱策略优化问题，以降低全社会物流成本为背景，综合考虑货物姿态限制、堆叠支撑关系、易碎件特殊承托要求、局部承压上限及顶部安全间隙等实际运输约束，建立了系统化的三维装箱优化模型，并分别求解了单车型最优装载方案与多车型组合运输方案，开展了参数灵敏度分析与可行性验证。

**针对问题一**，基于三维坐标系构建**货物姿态描述模型**，定义标准件、易碎件、定向件的合法姿态集合与支撑关系，建立以空间利用率与载重利用率**加权和**最大为目标的**单车型三维装箱优化模型**。采用“**层模板构造—层序组合优化**”的分层求解策略，通过网格搜索枚举模板组合，并对易碎件单独进行封顶可行性检验。求解得到：单独选用车型 1 需 **18 辆车**，空间利用率 **96.5%**、载重利用率 **52.6%**；单独选用车型 2 需 **8 辆车**，空间利用率 **96.3%**、载重利用率 **74.2%**。对顶部安全间隙的灵敏度分析表明，模型在基准值（3 cm）附近具有良好鲁棒性，安全间隙超过 10 cm 后装载性能显著下降。

**针对问题二**，在问题一的基础上进一步引入车型选择与货物跨车分配决策，分别以总运输车辆数最少和总运输成本最低为目标，建立多车型组合装箱优化模型。首先对货物进行**分组配对**，并通过垂直单分割、水平单分割、垂直双分割、水平双分割四种规则分割方式枚举混合层模板，并采用 **Pareto 非支配筛选**压缩模板库，保留在层高不增大前提下两类货物件数均不劣的候选模板。随后分别以总车辆数最小和总成本最小为目标建立模型，决策变量包括车辆启用、车型选择及货物分配。求解采用两阶段方法：第一阶段基于二维动态规划对每个配对组实现货物数量的精确覆盖，同步记录前驱状态以恢复最优模板组合；第二阶段将层实例分配至各车辆，引入高度剪枝、支撑闭合剪枝及分布剪枝三类策略进行深度优先搜索，确保每辆车满足约束。求解结果表明两个目标下的最优方案均为全部采用 **8 辆车型 2**，总运输成本 **5600 元**，未使用任何车型 1。压力热力图显示最大局部压强为  $319\text{ kg/m}^2$ ，远低于承压上限，验证了方案的可行性与稳定性。

**针对问题三**，基于前述模型与求解结果撰写技术报告，系统阐述了“**层模板生成—组合优化—坐标恢复—验证校核**”的分层建模思路，说明了程序模块划分与执行流程，分析了车型尺寸、货物结构比例及成本参数对装载率与运输成本的影响规律，给出了**优先选用车型 2**作为主力运输车辆的企业实施建议。

**关键词：**三维装箱；层模板组合；多车型优化；Pareto 筛选；剪枝策略

# 目录

|                                       |           |
|---------------------------------------|-----------|
| <b>一、 问题重述</b>                        | <b>1</b>  |
| 1.1 问题背景 . . . . .                    | 1         |
| 1.2 问题信息 . . . . .                    | 1         |
| 1.3 待求解问题 . . . . .                   | 1         |
| <b>二、 问题分析</b>                        | <b>2</b>  |
| 2.1 问题一的分析 . . . . .                  | 2         |
| 2.2 问题二的分析 . . . . .                  | 2         |
| 2.3 问题三的分析 . . . . .                  | 3         |
| <b>三、 问题假设</b>                        | <b>3</b>  |
| <b>四、 符号说明</b>                        | <b>4</b>  |
| <b>五、 问题一的模型建立与求解</b>                 | <b>4</b>  |
| 5.1 基于三维坐标系构建描述各货物姿态的三维模型 . . . . .   | 5         |
| 5.1.1 建立车厢坐标系 . . . . .               | 5         |
| 5.1.2 描述各货物三维姿态与位置 . . . . .          | 5         |
| 5.1.3 分析各不同种类货物的姿态限制 . . . . .        | 7         |
| 5.1.4 分析货物间支撑关系与稳定性要求 . . . . .       | 8         |
| 5.2 构建单车型三维装箱策略优化模型 . . . . .         | 11        |
| 5.2.1 定义决策变量 . . . . .                | 11        |
| 5.2.2 利用加权求和法确定优化模型目标函数 . . . . .     | 11        |
| 5.2.3 确定优化模型的装配约束条件 . . . . .         | 12        |
| 5.3 基于层模板构造与层序组合优化的求解寻优 . . . . .     | 15        |
| 5.3.1 构造单层模板库 . . . . .               | 15        |
| 5.3.2 扩展易碎品的封顶可行性 . . . . .           | 15        |
| 5.3.3 基于约束判据的方案可行性验证 . . . . .        | 16        |
| 5.3.4 利用网格搜索进行层模板组合与优化 . . . . .      | 16        |
| 5.4 最优装配方案展示与分析 . . . . .             | 17        |
| 5.4.1 单独选用车型 1 的最优装配方案 . . . . .      | 17        |
| 5.4.2 单独选用车型 2 的最优装配方案 . . . . .      | 18        |
| 5.5 模型灵敏度分析 . . . . .                 | 18        |
| <b>六、 问题二的模型建立与求解</b>                 | <b>19</b> |
| 6.1 进行货物分组配对以枚举单层模板 . . . . .         | 20        |
| 6.2 进行 Pareto 筛选对模板库进行压缩 . . . . .    | 21        |
| 6.3 构建以总运输车辆数最少为目标的三维装箱优化模型 . . . . . | 21        |
| 6.3.1 定义模型决策变量 . . . . .              | 22        |

|           |                                   |           |
|-----------|-----------------------------------|-----------|
| 6.3.2     | 确定优化模型目标函数 . . . . .              | 23        |
| 6.3.3     | 确定优化模型的装配约束条件 . . . . .           | 23        |
| 6.4       | 构建以总运输成本最低为目标的三维装箱优化模型 . . . . .  | 24        |
| 6.4.1     | 确定优化模型目标函数 . . . . .              | 24        |
| 6.5       | 采用剪枝策略进行层序组合优化求解 . . . . .        | 25        |
| 6.5.1     | 基于动态规划的精确覆盖 . . . . .             | 25        |
| 6.5.2     | 基于剪枝策略的分车搜索 . . . . .             | 26        |
| 6.6       | 不同目标下的最优方案展示与对比分析 . . . . .       | 27        |
| <b>七、</b> | <b>问题三的技术报告</b>                   | <b>27</b> |
| 7.1       | 项目目标与管理视角结论 . . . . .             | 27        |
| 7.2       | 总体建模思路 . . . . .                  | 28        |
| 7.3       | 程序实现与工程结构 . . . . .               | 28        |
| 7.4       | 模型与算法性能分析 . . . . .               | 29        |
| 7.5       | 参数变化对装载率与运输成本的影响 . . . . .        | 29        |
| 7.6       | 结论与实施建议 . . . . .                 | 30        |
| <b>八、</b> | <b>模型的评价与推广</b>                   | <b>30</b> |
| 8.1       | 模型的优点 . . . . .                   | 30        |
| 8.2       | 模型的不足与改进 . . . . .                | 30        |
| 8.3       | 模型的推广 . . . . .                   | 31        |
|           | <b>参考文献</b>                       | <b>31</b> |
|           | <b>附录 1 支撑材料清单</b>                | <b>32</b> |
|           | <b>附录 2 问题一的 Python 代码</b>        | <b>32</b> |
| 2.1       | 问题一第一小问的 Python 代码 1 . . . . .    | 32        |
| 2.2       | 问题一第一小问的 Python 代码 2 . . . . .    | 33        |
| 2.3       | 问题一第二小问的 Python 代码 . . . . .      | 48        |
|           | <b>附录 3 问题二的 Python 代码</b>        | <b>68</b> |
| 3.1       | 总运输车辆数最小目标问题的 Python 代码 . . . . . | 68        |
| 3.2       | 总运输成本最低目标问题的 Python 代码 . . . . .  | 80        |

# 一、问题重述

## 1.1 问题背景

随着我国现代物流体系和供应链体系的持续完善，“十五五”规划建议明确提出要“降低全社会物流成本”，推动物流行业向智慧化、绿色化、标准化方向转型。在此背景下，货物运输过程中的装箱与配载问题已成为影响运输效率、周转能力、企业成本控制和碳排放水平的关键决策问题。

然而，实际装箱过程受到多种因素限制，如车辆空间有限、货物参量各异，同时存在堆叠、摆放方向等约束。如何配载不同货物，实现“满容”与“满载”，是提高装载效率、减少运输成本的关键。若仅依赖人工经验配载，容易造成空间浪费、车辆趟次增加和综合运输成本上升。因此，需要针对不同运输场景建立多约束、多目标的三维装箱优化模型，并结合车辆类型选择、运输次数控制和成本测算等因素，优化装箱运输方案。

## 1.2 问题信息

- 定义“满容”为最大化利用车辆空间，满容率即货物占用空间与整个空间之比；“满载”为最大化利用车辆载重，满容率即装载货物重量与车厢最大载重之比。
- 运输车辆共有两种标准车型可选：车型 1 内尺寸为 420 cm×210 cm×220 cm，额定载重为 6000 kg，单次运输成本为 450 元/趟；车型 2 内尺寸为 680 cm×245 cm×250 cm，额定载重为 10000 kg，单次运输成本为 700 元/趟。
- 待运输货物均视为刚性长方体，装载过程中不可挤压、不可拆分、货物之间不得重叠、货物与车厢之间不得重叠、必须正交摆放。
- 堆叠时需满足承重约束，即下层货物的承重上限为 500 kg/m<sup>2</sup>。
- 货物总高度不得超过车厢内部高度，且顶层货物与车厢顶部之间需预留不少于 3 cm 的安全间隙。

## 1.3 待求解问题

**问题一：**分别以车型 1 和车型 2 为唯一装载对象，在已知车型箱体参数、运输成本以及各类货物参数信息的条件下，考虑可堆叠性和放置方向的约束，确保全部货物均可运输完成，分别以满载率与满容率最大化、所用车辆数最少为目标，建立单车型三维装箱策略优化模型，求解每种车型对应的最优装箱方案，并计算其满容率与满载率。

**问题二：**在车型 1 和车型 2 均可同时选用的条件下，确保全部货物运输完成，分别以总运输车辆数最少、总运输成本最低为目标，建立多车型组合装箱策略优化模型，确定不同车型的选用数量及各车次对应的装载方案，并进行不同目标下的方案对比分析。

**问题三：**基于前述问题的所得的模型与求解结果，撰写技术报告，以系统说明模型构建思路、算法设计方法、程序实现过程以及模型与算法的性能表现；并进一步分析关键参数变化对装载率与运输成本的影响规律。

## 二、问题分析

本套题目包含两个问题，围绕多场景、多目标条件下的货物运输装箱策略展开研究，从单车型三维装箱优化到多车型组合配送优化，以及最后对模型与算法性能分析。各问题由局部装载优化逐步扩展到整体运输决策，从单车型分析深化到多车型组合比较，问题层层递进，由浅入深地完成了一系列对货物运输装箱策略的研究。

### 2.1 问题一的分析

问题一属于确定约束条件下的三维装箱优化问题。其核心任务是在分别选择车型 1 或车型 2 的条件下，根据题目给定的车厢内部尺寸、额定载重以及各类货物的尺寸、重量、数量和摆放规则，建立能够准确描述货物与车厢空间关系的数学模型，并在满足所有装载要求与约束的前提下，求得单车装载率最大的最优装箱方案。

由于货物种类有限、尺寸参数明确，且装载规则具有较强的结构性，需要同时处理货物选择、摆放状态以及堆叠关系等多重决策变量，具有离散组合优化特征。针对这类具有明确物理边界和离散决策变量的装载问题，主要存在两种求解思路<sup>[1]</sup>：其一是单件货物坐标直接优化法，建立整体装载模型；其二是基于层模板的组合搜索法，先构造满足底面铺排规则的单层模板，再通过模板之间的组合完成整体装载。前者需要同时处理货物间多种约束，变量维数高、耦合强，若直接在单件坐标层面进行求解，搜索空间将迅速膨胀，模型求解和可实施性检验都较为困难。因此可先将问题分解为单层构型设计与层间组合优化两个阶段。前一阶段通过对货物姿态进行预处理，并依据底面尺寸匹配关系构造候选层模板；转化为模板数量空间中的网格搜索问题，以各类层模板的使用次数为整数决策变量，在有限离散空间内进行枚举。

因此，问题一采用“单层模板构造—离散网格搜索—封顶可行性检验—整体约束验证”的求解思路<sup>[2]</sup>。具体而言，先构造标准层模板库与  $G_3$  顶层模板集合；进一步建立模板数量组合优化模型。以各类模板的使用次数为整数决策变量，以五类货物数量完全满足为数量约束，以总层高不超过车辆有效高度、总质量不超过车辆载重上限为容量约束，并将  $G_3$  必须封顶作为特殊结构约束。再将各模板使用次数离散化，建立模板数量搜索网格，对所有候选组合逐点枚举，计算其对应的货物覆盖数量、总层高及结构可行性。

### 2.2 问题二的分析

问题二属于多车型条件下的三维装箱与组合优化问题，是基于问题一的多车型组合优化问题。其核心任务是在两种车型参数、五类货物尺寸与重量信息以及易碎件、定向件、支撑稳定性、顶部安全间隙等约束条件均已确定的情况下，建立一个能够同时描述单车内部三维可行装载与多车整体运输分配的数学模型，并分别以总运输车辆数最少和总运输成本最低为目标，求解全部货物的最优运输方案。与问题一仅关注单辆车辆内部装载不同，问题二进一步引入了车型选择、车辆数量确定以及货物在多辆车之间的全局分配<sup>[2]</sup>，因此是一个具有明显层次性与耦合性的确定性优化问题。

针对该类问题，主要存在两种求解思路：一类是直接建立大规模整数规划模型进行精确求解<sup>[3]</sup>，另一类是基于单车可行装载结构构造启发式组合优化方法。其中，精确求解思路需要同时描述车辆是否启用、车型如何选择、各类货物如何在不同车辆间分配，以及每辆车内部各货物的三维放置，不仅包含大量 0-1 决策变量与整数变量，还嵌套了复杂的空间几何约束和层间支撑约束，车辆之间相互耦合，若直接求解，往往会面临状态空间急剧膨胀、计算复杂度极高、模型实施困难等问题。尤其在同时考虑易碎件顶层放置、定向件姿态受限和联合支撑有效性时，直接获得整体最优解的代价较大，难以满足建模求解的效率要求<sup>[4]</sup>。

因此，在问题一研究结果的基础上，选用分层模板构造与全局启发式搜索相结合的方法。利用问题一中所建立的单车型三维可行装载模型，将每一种车型下满足装载结构表示为若干可复用的层模板和层序方案；在此基础上，进一步建立多车型联合运输优化模型，把问题二分解为“单车可行装载生成”和“多车全局方案选择”两个层次。构造混合车型参数模型，分别描述两类车辆的底面尺寸、有效高度、载重上限及对应运输成本；然后构造车辆启用—车型选择模型，以候选车辆是否被启用及其所属车型作为决策变量，刻画多车辆系统中的运输资源配置关系；进一步构造多车装载分配模型，将全部货物或等价层模板分配至不同车辆，并要求每辆车内部满足边界、不重叠、支撑、封顶及承压等约束。

## 2.3 问题三的分析

### 三、问题假设

为简化与完善问题信息，在此做出以下假设：

- 假设一：所有货物不会打滑，在承重范围内不被压缩。
- 假设二：货物密度分布均匀，重心位于其几何中心，稳定性由几何重心投影判定。
- 假设三：不考虑货物借助车壁形成侧向支撑，仅依据底部接触关系进行支撑判定。
- 假设四：装载过程中货物之间以及货物与车厢内壁之间视为无间隙贴合放置。
- 假设五：若一上层货物同时压在多个下层货物上，其总载荷由接触面积比例分担。
- 假设六：局部承压上限按实际接触面积计算<sup>[6]</sup>。
- 假设七：定向件均明确“向上”放置标识，顶面与底面严格确定。

## 四、符号说明

| 符号                          | 符号说明           | 符号                | 符号说明                      |
|-----------------------------|----------------|-------------------|---------------------------|
| $\mathcal{I}$               | 单件货物全集         | $c_i$             | 货物 $i$ 的类别                |
| $\mathcal{I}_{\text{std}}$  | 标准件集合          | $L_i, W_i, H_i$   | 货物 $i$ 的长、宽、高             |
| $\mathcal{I}_{\text{frag}}$ | 易碎件集合          | $m_i$             | 货物 $i$ 单重                 |
| $\mathcal{I}_{\text{dir}}$  | 定向件集合          | $v_i$             | 货物 $i$ 原始体积               |
| $\mathcal{T}$               | 车型集合           | $v_i$             | 货物 $i$ 的实际体积              |
| $O_i$                       | 货物 $i$ 的合法姿态集合 | $x_i, y_i, z_i$   | 货物 $i$ 右后下角坐标             |
| $P(i)$                      | 支撑货物 $i$ 父节点集合 | $R_i$             | 货物 $i$ 水平面投影矩形            |
| $C(i)$                      | 货物 $i$ 支撑子节点集合 | $A_{ji}$          | 货物 $j$ 对 $i$ 的接触面积        |
| $L_{(t)}$                   | 车型 $t$ 车厢内长    | $g_i$             | 货物 $i$ 重心的投影点             |
| $W_{(t)}$                   | 车型 $t$ 车厢内宽    | $T_i$             | 货物 $i$ 向下传递的载荷            |
| $H_{(t)}$                   | 车型 $t$ 车厢内高    | $p_{ji}$          | $j \rightarrow i$ 接触的局部压强 |
| $Q_{(t)}$                   | 车型 $t$ 额定载重    | $\rho_v$          | 空间利用率                     |
| $\text{Cost}_{(t)}$         | 车型 $t$ 单次运输成本  | $\rho_w$          | 载重利用率                     |
| $V_{(t)}$                   | 车型 $t$ 车厢容积    | $V_{\text{load}}$ | 已装货物总体积                   |
| $g$                         | 顶部安全间隙         | $W_{\text{load}}$ | 已装货物总重量                   |
| $p_{\max}$                  | 局部承压上限         | $V_{\text{all}}$  | 全部货物总体积                   |

## 五、问题一的模型建立与求解

问题一给出 2 种车型箱体参数以及 5 种货物参数信息，要求约束下确保全部货物均可运输完成；以满载率与满容率最大化为目标，优化装箱方案，是确定性的三维装箱优化与空间排布问题。建立三维货物姿态模型描述货物装载方式。以最大化满载率与满容率为目标，建立单车型三维装箱策略优化模型，求解每种车型对应的最优装箱方案。通过构造单层模板，在网格搜索中组合多层装载结构，对层序方案进行比较，最终确定最优策略内每件货物的坐标、姿态及满容率与满载率。

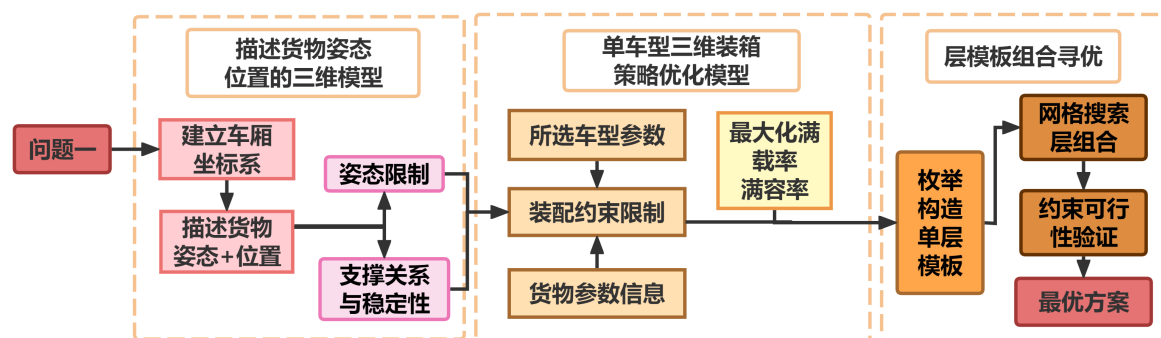


图 1 问题一框架图

## 5.1 基于三维坐标系构建描述各货物姿态的三维模型

### 5.1.1 建立车厢坐标系

根据题目描述，按照题意基于车厢建立一个三维坐标系。以车厢右下角三维直角坐标系原点，沿车头方向作为  $x$  轴正半轴；沿车厢宽度由右向左作为  $y$  轴正半轴；竖直向上作为  $z$  轴正半轴，如图 2 所示建立坐标系。

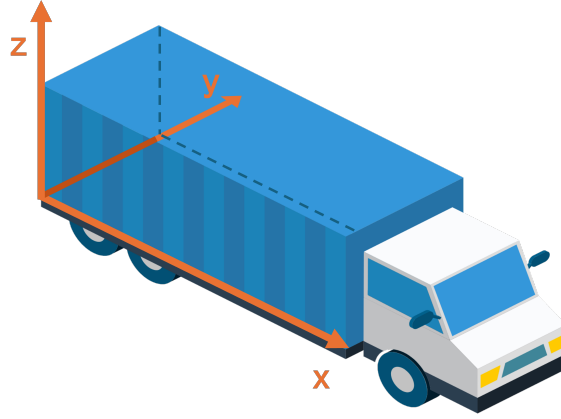


图 2 车厢坐标系示意图

进一步设定车辆参数，设所选车辆型号为  $t$ ，其内部尺寸与承载参数定义如下：

车厢内部长度、宽度、高度分别为  $L_{(t)}$ 、 $W_{(t)}$ 、 $H_{(t)}$ ；车辆最大载重为  $Q_{(t)}$ ；

顶部安全间隙为  $\delta$ ；有效装载高度为  $H'_{(t)} = H_{(t)} - \delta$ 。

综上，车厢内部空间可表示为一个长方体区域：

$$\mathcal{B}_{(t)} = [0, L_{(t)}] \times [0, W_{(t)}] \times [0, H_{(t)}] \quad (1)$$

货物允许占据的实际空间为：

$$\mathcal{B}'_{(t)} = [0, L_{(t)}] \times [0, W_{(t)}] \times [0, H_{(t)} - \delta] \quad (2)$$

### 5.1.2 描述各货物三维姿态与位置

对于整个装箱过程，其中存在多个货物的位置关系，需要考虑不可挤压、不可拆分、货物之间不得重叠、货物与车厢之间不得重叠、必须正交摆放等的条件。为了更清晰的进行描述各货物的装载进车厢后的位置，将箱问题中的每件货物都抽象为一个由“位置坐标 + 姿态尺寸”共同确定的轴对齐长方体。

设全部货物集合为  $\mathcal{I} = \{1, 2, \dots, n\}$ ；对于任意货物  $i \in \mathcal{I}$  的位置采用其右后下角顶点坐标表示，记为： $(x_i, y_i, z_i)$ 。并且定义：

货物原始长、宽、高分别为  $L_i$ 、 $W_i$ 、 $H_i$ ；货物重量为  $m_i$ ；货物体积为  $v_i = L_i W_i H_i$ 。

理论上，一个长方体在三维空间中最多有 6 种轴对齐姿态，对应于原始三维尺寸  $(L_i, W_i, H_i)$  在  $(x, y, z)$  三轴上的所有排列。因此，姿态编码采用如下六种形式，选择统一采用 6 种正交姿态编码指示货件的旋转状态<sup>[7]</sup>：



(长, 宽, 高) → (X, Y, Z)、(X, Z, Y)、(Y, X, Z)、(Y, Z, X)、(Z, X, Y)、(Z, Y, X)  
含义为原始“长、宽、高”分别映射到坐标轴的方式, 如图 3 (a) 所示, 即:

- 第 1 个字母: 货件的“长”映射到哪根车厢轴;
- 第 2 个字母: 货件的“宽”映射到哪根车厢轴;
- 第 3 个字母: 货件的“高”映射到哪根车厢轴。

由此, 设货物  $i$  的允许摆放姿态集合为  $O_i$ 。而对于任一姿态  $k \in O_i$ , 其沿  $x$  轴方向尺寸为  $l_{ik}$ ; 沿  $y$  轴方向尺寸为  $w_{ik}$ ; 沿  $z$  轴方向尺寸为  $h_{ik}$ , 如图 3 (b) 所示。

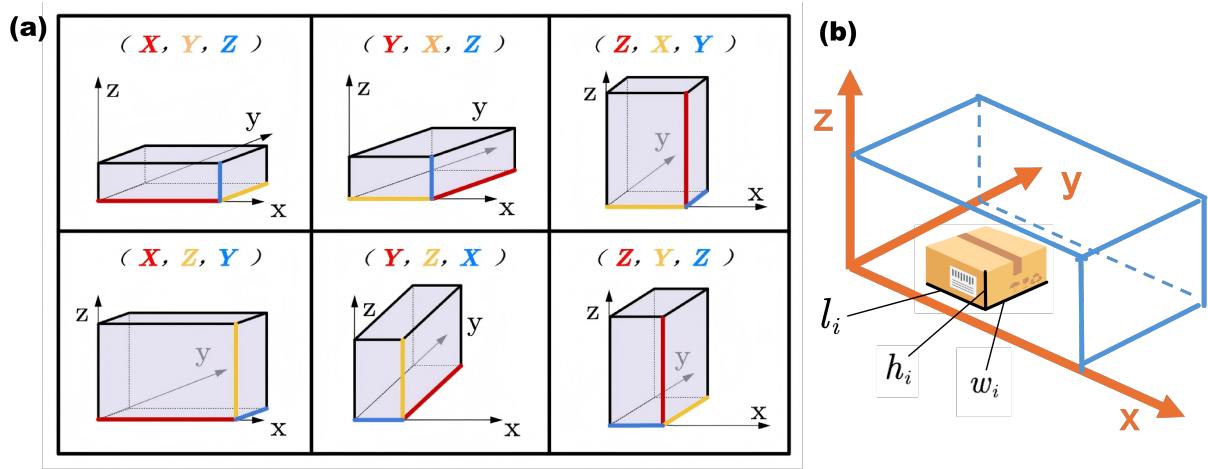


图 3 姿态编码规则示意图 (a) 与摆放姿态下沿方向轴的尺寸示意 (b)

对于货物  $i$ , 定义参数  $u_i$ 、 $o_{ik}$  为:

$$u_i = \begin{cases} 1, & \text{装入该车} \\ 0, & \text{不装入该车} \end{cases} \quad o_{ik} = \begin{cases} 1, & \text{采用姿态 } k \\ 0, & \text{不采用姿态 } k \end{cases} \quad (3)$$

若货物被装载, 则其实际装载尺寸为:

$$\begin{cases} l_i = \sum_{k \in O_i} l_{ik} o_{ik} \\ w_i = \sum_{k \in O_i} w_{ik} o_{ik} \\ h_i = \sum_{k \in O_i} h_{ik} o_{ik} \end{cases} \quad (4)$$

其占据的空间区域可表示为:

$$[x_i, x_i + l_i] \times [y_i, y_i + w_i] \times [z_i, z_i + h_i] \quad (5)$$

并且确定姿态唯一, 即

$$\sum_{k \in O_i} o_{ik} = u_i, \quad \forall i \in I \quad (6)$$

### 5.1.3 分析各不同种类货物的姿态限制

不同类别货物对摆放姿态的要求不同，需要对各类货物的合法姿态集合作进一步分类讨论。设货物类别为  $c_i$ ，其中  $c_i \in \{\text{std}, \text{frag}, \text{dir}\}$ ，分别表示标准件、易碎件、定向件。

进一步定义分类集合： $\mathcal{I}_{\text{std}}$ ：标准件集合； $\mathcal{I}_{\text{frag}}$ ：易碎件集合； $\mathcal{I}_{\text{dir}}$ ：定向件集合。

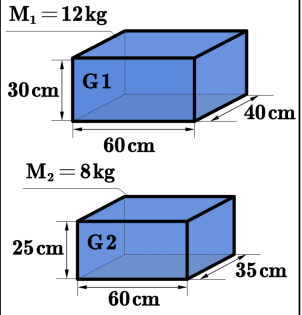
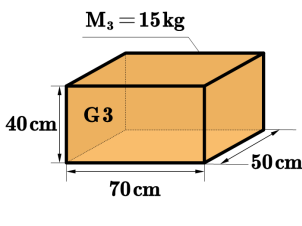
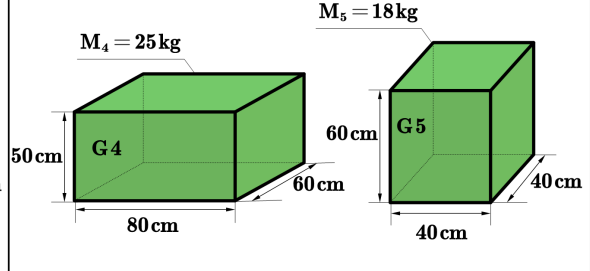
| 标准件                                                                                                                                                                                                                                 | 易碎件                                                                                                                                                          | 定向件                                                                                                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <p><math>M_1 = 12\text{kg}</math><br/>30 cm<br/>60 cm<br/>40 cm<br/>G1<br/><math>M_2 = 8\text{kg}</math><br/>25 cm<br/>60 cm<br/>35 cm<br/>G2</p> |  <p><math>M_3 = 15\text{kg}</math><br/>40 cm<br/>70 cm<br/>50 cm<br/>G3</p> |  <p><math>M_4 = 25\text{kg}</math><br/>50 cm<br/>80 cm<br/>60 cm<br/>G4<br/><math>M_5 = 18\text{kg}</math><br/>60 cm<br/>40 cm<br/>40 cm<br/>G5</p> |

图 4 三种不同类别的货物汇总

#### • 标准件姿态

标准件允许任意方向摆放，因此其姿态集合由全部正交姿态构成。标准件的姿态编码包含六种形式： $(X, Y, Z)$ 、 $(X, Z, Y)$ 、 $(Y, X, Z)$ 、 $(Y, Z, X)$ 、 $(Z, X, Y)$ 、 $(Z, Y, X)$ 。

需要说明的是，若某些原始尺寸相等，则不同编码可能对应相同的实际几何尺寸。此时应对姿态集合进行去重处理。因此，对标准件  $i$ ，其合法姿态集合记为：

$$\mathcal{O}_i = \{\text{全部几何上互异的正交姿态}\}, \quad i \in \mathcal{I}_{\text{std}}$$

#### • 易碎件姿态

题目并未额外规定易碎件哪一面必须朝上，因此在姿态自由度上，易碎件与标准件相同，也允许在全部正交姿态中选取几何上互异的姿态，即：

$$\mathcal{O}_i = \{\text{全部几何上互异的正交姿态}\}, \quad i \in \mathcal{I}_{\text{frag}}$$

#### • 定向件姿态

与标准件和易碎件不同，定向件具有明确方向要求，因此不能进行任意翻转。定向件必须保持题意所规定的竖直朝向，在该竖直方向固定后，若其底面长宽不相等，则仍可在水平面内进行  $90^\circ$  旋转，从而形成两种不同的水平放置方式。

设货物  $i \in \mathcal{I}_{\text{dir}}$  为定向件，在某一固定解释情景下，其竖直方向已确定。此时记其合法姿态集合为  $\mathcal{O}_i$ ，则有：

1. 若固定竖直后底面两边长度不同，则  $\mathcal{O}_i = \{(X, Y, Z), (Y, X, Z)\}$ 、 $|\mathcal{O}_i| = 2$ ;
2. 若固定竖直后底面两边长度相同，则两种水平旋转后的几何尺寸完全一致，去重处理得到  $\mathcal{O}_i = \{(X, Y, Z)\}$ 、 $|\mathcal{O}_i| = 1$ .

因此、，对任意定向件  $i$ ，其姿态集合可统一表示为：

$$O_i = \{\text{固定竖直方向下全部几何上互异的水平放置姿态}\}, \quad i \in I_{\text{dir}}.$$

$$\text{从而有：} 1 \leq |O_i| \leq 2, \quad i \in I_{\text{dir}}$$

#### 5.1.4 分析货物间支撑关系与稳定性要求

当一件货物位于车厢底面时，其稳定性可天然满足；但当货物位于某层货物上层时，其可行性不仅取决于与其他货物是否重叠，更取决于下层是否能够形成有效支撑。因此，必须建立货物间的支撑关系模型。

##### • 支撑父节点与子节点集合的定义

对任意已装载货物  $i$ ，定义其底面在水平面的投影矩形为：

$$R_i = [x_i, x_i + l_i] \times [y_i, y_i + w_i] \quad (7)$$

若另一货物  $j$  的顶面与货物  $i$  的底面恰好位于同一高度，即：

$$z_j + h_j = z_i \quad (8)$$

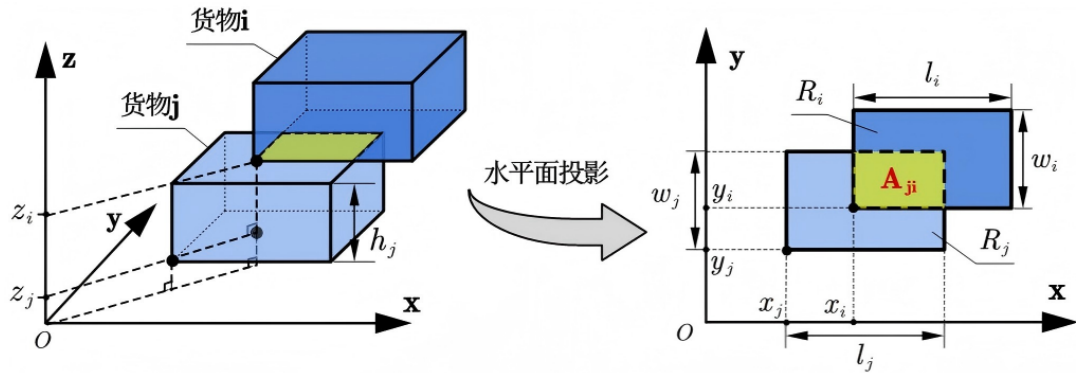


图 5 支撑关系示意图

且二者在水平面上的投影存在正面积重叠（如图 4 所示），则称货物  $j$  对货物  $i$  提供支撑。此时称  $j$  为  $i$  的父节点， $i$  为  $j$  的子节点。

据此，定义货物  $i$  的支撑父节点集合为：

$$P(i) = \{j \in I : z_j + h_j = z_i, |R_j \cap R_i| > 0\}$$

同时定义货物  $i$  的子节点集合为：

$$C(i) = \{k \in I : i \in P(k)\}$$

即集合  $P(i)$  表示“谁在支撑  $i$ ”；集合  $C(i)$  表示“ $i$  正在支撑谁”。

### • 接触面积与有效支撑域的定义

对于任意父节点  $j \in P(i)$ ，其与货物  $i$  的接触区域定义为： $R_j \cap R_i$ ，接触面积记为： $A_{ji} = |R_j \cap R_i|$ 。即货物  $i$  的全部有效支撑区域可表示为所有支撑接触区域的并集，物理意义为货物  $i$  的底面中真正被下层承托到的部分。

$$\Omega_i = \bigcup_{j \in P(i)} (R_j \cap R_i)$$

对不同类别货物，稳定性要求有所不同：

- 对易碎件，要求其底面必须被标准件有效支撑域完全覆盖；
- 对标准件与定向件，只要求其底面重心投影落在有效支撑域之内。

因此，有效支撑域  $\Omega_i$  是连接“几何布局”和“稳定性判定”的关键桥梁。

为了进一步判定重心稳定性，定义货物  $i$  的底面几何中心投影为：

$$g_i = \left( x_i + \frac{l_i}{2}, y_i + \frac{w_i}{2} \right) \quad (9)$$

在假设二（均匀密度）的条件下， $g_i$  可视为货物重心在水平面上的投影位置。则标准件和定向件的稳定性可通过判断： $g_i \in \Omega_i$  来实现。

### • 易碎件的特殊支撑要求

易碎件是所有货物中结构约束最严格的一类货物。虽然其几何形状仍为规则长方体，且可在合法姿态集合中进行正交摆放，但其支撑关系明显区别于标准件和定向件。为避免运输过程中因上部压载、局部悬空或支撑不足而导致损坏，将易碎件作为支撑图中的特殊节点处理，并施加额外支撑条件限制。

设  $i \in \mathcal{I}_{\text{frag}}$  为任一易碎件，其底面投影区域为  $R_i$ ，支撑父节点集合为  $P(i)$ ，子节点集合为  $C(i)$ ，有效支撑域为  $\Omega_i$ 。

#### (1) 易碎件的下层必须是标准件。

若易碎件直接放置于车厢底板上，即  $z_i = 0$ ，则其由车厢底板直接支撑；若易碎件位于上层，即  $z_i > 0$ ，则其支撑父节点只能来自标准件，即

$$P(i) \subseteq \mathcal{I}_{\text{std}}, \quad i \in \mathcal{I}_{\text{frag}}, z_i > 0$$

这表明易碎件不能建立在其他易碎件或定向件之上。当易碎件不位于地面时，必须至少存在一个支撑父节点，排除悬空放置的情形。即：

$$P(i) \neq \emptyset, \quad i \in \mathcal{I}_{\text{frag}}, z_i > 0$$

#### (2) 易碎件不能出现局部悬空。

与标准件和定向件仅要求重心投影落在有效支撑域内不同，易碎件要求其底面必须被完整承托，不允许出现任何局部悬空区域。因此应满足

$$R_i \subseteq \Omega_i, \quad i \in \mathcal{I}_{\text{frag}}, z_i > 0$$

等价地可写为

$$|\Omega_i| = |R_i| = l_i w_i$$

### (3) 易碎品不可堆叠。

最为关键的是易碎件上方不允许再放置其他货物，因此其子节点集合必须为空，即

$$C(i) = \emptyset, \quad i \in \mathcal{I}_{\text{frag}}.$$

这说明易碎件在支撑结构中只能作为终端节点，而不能作为中间承重层。

#### • 局部承压载荷计算

仅有支撑关系和重心稳定仍不足以保证堆叠安全，因为上层货物可能通过较小的接触面积向下层施加过大压力，导致局部压强超过允许范围。为此，进一步建立基于支撑图的局部承压计算模型。

依据假设五，按实际接触面积计算局部压强，若货物  $i$  由多个父节点共同支撑，则其总载荷需按照接触面积比例向下分摊。定义父节点  $j$  对货物  $i$  载荷的分担比例为：

$$\alpha_{ji} = \frac{A_{ji}}{\sum_{k \in P(i)} A_{ki}}, \quad j \in P(i) \quad (10)$$

显然有：

$$\sum_{j \in P(i)} \alpha_{ji} = 1 \quad (11)$$

进一步定义货物  $i$  向下传递的总载荷为  $T_i$ 。该总载荷包含货物  $i$  自身重量  $m_i$  和所有位于  $i$  上方、并通过其向下传递的载荷。

因此有递推关系：

$$T_i = m_i + \sum_{k \in C(i)} \alpha_{ik} T_k \quad (12)$$

其中， $\alpha_{ik} T_k$  表示货物  $k$  的总载荷中分配给下层货物  $i$  的部分。

该递推过程可以自上而下逐层计算。对任意接触对  $j \rightarrow i$ ，货物  $i$  施加给父节点  $j$  的局部载荷为： $\alpha_{ji} T_i$ ，接触面积为  $A_{ji}$ 。

因此，接触对  $j \rightarrow i$  上的局部压强定义为：

$$p_{ji} = \frac{\alpha_{ji} T_i}{A_{ji}/10^4} \quad (m^2) \quad (13)$$

题目规定局部单位面积承压上限为  $500 \text{ kg/m}^2$ ，故必须满足：

$$p_{ji} \leq p_{\max} = 500 \quad (14)$$

## 5.2 构建单车型三维装箱策略优化模型

### 5.2.1 定义决策变量

根据题目分析，需要同时决定货物是否装入车辆、装入后的放置姿态以及其在车厢中的具体位置。因此定义如下决策变量：

#### (1) 货物装载变量

对任意货物  $i$ ，定义二元变量  $u_i$  用于确定装载选择：

$$u_i = \begin{cases} 1, & \text{若货物 } i \text{ 被装入该车} \\ 0, & \text{若货物 } i \text{ 未被装入该车} \end{cases} \quad (15)$$

#### (2) 姿态选择变量

货物  $i$  的合法姿态集合为  $O_i$ 。对任意  $k \in O_i$ ，定义二元变量

$$\delta_{ik} = \begin{cases} 1, & \text{若货物 } i \text{ 以第 } k \text{ 种合法姿态放置} \\ 0, & \text{若货物 } i \text{ 未以第 } k \text{ 种合法姿态放置} \end{cases} \quad (16)$$

并满足

$$\sum_{k \in O_i} \delta_{ik} = u_i, \quad \forall i \quad (17)$$

#### (3) 空间位置变量

对任意货物  $i$ ，定义连续变量  $x_i, y_i, z_i$ ，分别表示货物  $i$  左后下角在车厢坐标系中的长度方向、宽度方向和高度方向坐标。进一步地，设货物  $i$  在姿态  $k$  下的尺寸为  $(l_{ik}, w_{ik}, h_{ik})$ ，则其实际装载尺寸可表示为

$$\begin{cases} l_i = \sum_{k \in O_i} l_{ik} \delta_{ik} \\ w_i = \sum_{k \in O_i} w_{ik} \delta_{ik} \\ h_i = \sum_{k \in O_i} h_{ik} \delta_{ik} \end{cases} \quad (18)$$

综上，问题一的装载方案可由变量  $\{u_i, \delta_{ik}, x_i, y_i, z_i\}$  共同定义。

### 5.2.2 利用加权求和法确定优化模型目标函数

在题目固定单一车型  $t$  的前提下，车厢内部可用空间与允许承载重量均已知，因此单车装箱方案的优劣可以通过“装了多少体积”和“装了多少重量”两个维度进行衡量，即问题一的优化目标即为实现最大化满容率与满载率，构造为双目标函数。

设车型  $t$  的车厢内部尺寸为  $L_{(t)} \times W_{(t)} \times H_{(t)}$ ，则车厢几何容积为

$$V_{(t)} = L_{(t)} W_{(t)} H_{(t)} \quad (19)$$

根据上文分析，设货物  $i$  的体积为  $v_i$ ，是否被装入当前车辆由变量  $u_i$  表示，其中  $u_i = 1$  表示装入， $u_i = 0$  表示未装入。则当前车辆实际装入货物的总体积为：

$$V_{\text{load}} = \sum_{i \in I} u_i v_i \quad (20)$$

据此定义单车的空间利用率为：

$$\rho_V = \frac{V_{\text{load}}}{V_{(t)}} = \frac{\sum_{i \in I} u_i v_i}{V_{(t)}} \quad (21)$$

即反映货物对车厢内部空间的占用程度，数值越大，说明空间被利用得越充分。

另一方面，设货物  $i$  的重量为  $m_i$ ，车型  $t$  的额定载重为  $Q_{(t)}$ ，则当前车辆所承载货物的总重量为

$$W_{\text{load}} = \sum_{i \in I} u_i m_i \quad (22)$$

据此定义单车的载重利用率为：

$$\rho_W = \frac{W_{\text{load}}}{Q_{(t)}} = \frac{\sum_{i \in I} u_i m_i}{Q_{(t)}} \quad (23)$$

即反映车辆载重能力的使用程度，数值越大，说明车辆吨位资源被利用得越充分。

综上，问题一单车型装箱的双目标“力求实现满容率与满载率最大化”可写为：

$$\max \rho_V, \quad \max \rho_W$$

考虑到后续求解过程中需要对候选方案进行统一评价，采用加权求和的方法将双目标转化为单目标。在此不额外强调某一目标的重要性，则可取等权形式：

$$\max F = \rho_V + \rho_W \quad (24)$$

即目标函数最终表示为：

$$\max F = \frac{\sum_{i \in I} u_i v_i}{V_{(t)}} + \frac{\sum_{i \in I} u_i m_i}{Q_{(t)}} \quad (25)$$

该目标函数意味着在保证装箱方案满足全部可行性约束的基础上，优先选择那些既能充分占用车厢空间、又能充分利用车辆载重能力的装载策略。

### 5.2.3 确定优化模型的装配约束条件

在确定单车装箱问题的优化目标后，需要进一步给出保证装载方案可行的各类约束条件。题目中要求装载过程满足实际运输中的摆放规则、支撑关系与受力安全要求，因此装配策略必须满足可行性条件约束。

#### • 货物完整运输约束

题目要求全部货物均必须被运输，因此每件货物都要恰好出现在一辆车中，即

$$\sum_{r \in \mathcal{R}} u_i = 1, \quad \forall i \in I \quad (26)$$

- **姿态唯一性约束**

对任意货物  $i$ ，其装入后只能从合法姿态集合中选择一种摆放方式，即

$$\sum_{o \in O_i} \delta_{i,o} = u_i, \quad \forall i \in I \quad (27)$$

当  $u_i = 0$  时，说明货物  $i$  未装入当前车辆，所有姿态变量均应为 0；

当  $u_i = 1$  时，说明货物  $i$  被装入当前车辆，只能选取一种合法姿态。

- **车厢边界约束**

装入车辆的货物都必须完全位于车厢内部的有效装载空间中，且需留出预留安全间隙。则货物  $i$  若被装入车辆，应满足：

$$0 \leq x_i \leq L_{(t)} - l_i \quad (28)$$

$$0 \leq y_i \leq W_{(t)} - w_i \quad (29)$$

$$0 \leq z_i \leq H_{(t)} - g - h_i \quad (30)$$

- **货物非重叠约束**

对于任意两件不同货物  $i \neq j$ ，若二者均被装入当前车辆，只要三维三个方向都存在区间交叠，就意味着两件货物发生了实体碰撞。则在三维空间中必须至少在一个方向上完全分离，即下列 6 种相对位置关系中至少应有一种成立：

$$x_i + l_i \leq x_j \quad x_j + l_j \leq x_i$$

$$y_i + w_i \leq y_j \quad y_j + w_j \leq y_i$$

$$z_i + h_i \leq z_j \quad z_j + h_j \leq z_i$$

- **整车载重约束**

在货物放入车厢后必须满足车辆的载重安全要求。设车型  $t$  的额定载重为  $Q_{(t)}$ ，则当前车辆所装入全部货物的总重量应满足：

$$\sum_{i \in I} u_i m_i \leq Q_{(t)} \quad (31)$$

- **货物间支撑关系约束**

当货物直接放置在车厢底板上时，其支撑关系由底板提供，不需要额外判断；而对任意离地货物  $i (z_i > 0)$ ，其支撑父节点集合不能为空，即

$$P(i) \neq \emptyset \quad (32)$$



### • 易碎件约束

任意易碎件  $i \in \mathcal{I}_{\text{frag}}$ ，根据上文确定的特殊支撑要求：仅能由标准件支撑、底面必须获得完整支撑、且上方不得继续堆放货物，应满足

$$P(i) \subseteq \mathcal{I}_{\text{std}} \quad R_i \subseteq \Omega_i, \quad i \in \mathcal{I}_{\text{frag}}, z_i > 0 \quad (33)$$

### • 标准件与定向件稳定性约束

标准件与定向件采用底面重心投影是否落在“下层联合形成的有效支撑域内”作为标准件稳定性的判定准则，若货物重心在底面的投影落入其实际支撑块所形成的联合支撑区域内，则认为该货物满足稳定放置条件。即  $z_i > 0$  时，需满足

$$g_i \in \Omega_i, \quad i \in (\mathcal{I}_{\text{std}} \cup \mathcal{I}_{\text{dir}}), \quad z_i > 0 \quad (34)$$

### • 局部承压约束

根据上文接触面积、载荷分担比例及递推载荷计算公式的基础上，可进一步得到任意支撑接触面对上的局部承压表达。对任意支撑关系  $j \rightarrow i$ ，其局部压强为

$$p_{ji} = \frac{\alpha_{ji} T_i}{A_{ji}/10^4} \quad (35)$$

根据题目要求，单位面积局部承压不得超过给定上限：

$$p_{ji} \leq p_{\max} = 500 \quad (36)$$

综上，经过总结得到问题一的模型目标函数为

$$\max F = \frac{\sum_{i \in \mathcal{I}} u_i v_i}{V(t)} + \frac{\sum_{i \in \mathcal{I}} u_i m_i}{Q(t)} \quad (37)$$

决策变量:  $\mathbf{x} = (u_i, \delta_{i,o}, x_i, y_i, z_i)$

$$s.t. \left\{ \begin{array}{l} \text{货物完整运输约束: } \sum_{r \in \mathcal{R}} u_i = 1, \quad \forall i \in \mathcal{I} \\ \text{姿态唯一性约束: } \sum_{o \in \mathcal{O}_i} \delta_{i,o} = u_i, \quad \forall i \in \mathcal{I} \\ \text{车厢边界约束: } \begin{cases} 0 \leq x_i \leq L(t) - l_i \\ 0 \leq y_i \leq W(t) - w_i \\ 0 \leq z_i \leq H(t) - g - h_i \end{cases} \\ \text{货物非重叠约束: } \begin{cases} x_i + l_i \leq x_j \text{ 或 } x_j + l_j \leq x_i \text{ 或 } y_i + w_i \leq y_j \text{ 或} \\ y_j + w_j \leq y_i \text{ 或 } z_i + h_i \leq z_j \text{ 或 } z_j + h_j \leq z_i \end{cases} \\ \text{整车载重约束: } \sum_{i \in \mathcal{I}} u_i m_i \leq Q(t) \\ \text{货物间支撑关系约束: } P(i) \neq \emptyset \quad z_i > 0 \\ \text{易碎件约束: } P(i) \subseteq \mathcal{I}_{\text{std}} \quad R_i \subseteq \Omega_i, \quad i \in \mathcal{I}_{\text{frag}}, z_i > 0 \\ \text{标准件与定向件稳定性约束: } g_i \in \Omega_i, \quad i \in (\mathcal{I}_{\text{std}} \cup \mathcal{I}_{\text{dir}}), \quad z_i > 0 \\ \text{局部承压约束: } p_{ji} = \frac{\alpha_{ji} T_i}{A_{ji}/10^4} \leq p_{\max} \end{array} \right.$$

### 5.3 基于层模板构造与层序组合优化的求解寻优

考虑到单车装载问题同时包含货物姿态选择、三维空间排布、支撑关系判定及约束，若直接在“单件—位置—姿态”的自由组合空间中进行全局搜索，求解规模将随货物数量迅速膨胀，难以在有限时间内获得结构清晰且稳定的高质量解。结合附件 1 中货物类别较少、尺寸规则性较强、批量较大的特点，将问题重构为“层模板构造—层序组合”的分层求解问题。

#### 5.3.1 构造单层模板库

设车厢底面尺寸为  $L_{(t)} \times W_{(t)}$ ，单层模板记为  $T$ 。对于任一模板  $T$ ，定义特征参数：覆盖面积为  $A(T)$ ，层高为  $h(T)$

层内总重量为  $M(T)$ ，层内总体积为  $V(T)$

则模板  $T$  的底面覆盖率定义为

$$\eta(T) = \frac{A(T)}{L_{(t)}W_{(t)}} \quad (38)$$

底面覆盖率越高，说明该模板对车厢底面的利用越充分；层高越低，则在总高度受限条件下可容纳的层数越多；层内总重量越大，则对载重利用率贡献越明显。因此，单层模板可以从“底面覆盖率—层高层数—载重利用率贡献”三个维度综合评价。

在模板构造过程中，采用“主模板 + 边缘补洞模板”的思路。

**Step1:** 选取某一类货物作为主要铺设对象，使其尽可能覆盖底面大部分区域。

**Step2:** 对于边缘剩余的狭长区域或角隙，再选取尺寸匹配度更高的货物进行补边，以避免整层仅由单一货物构成时产生的大面积边缘浪费，提高单层覆盖率。

经单层模板实验比较发现：

- 对于车型 1， $G_1$  的尺寸与车厢底面匹配程度较高，更适合作为主模板； $G_2$  可用于局部补边和层结构增强；
- 对于车型 2， $G_5$  与  $G_2$  的单层模板在覆盖率和重量贡献方面表现更优，更适合作为主体层的构造单元。

#### 5.3.2 扩展易碎品的封顶可行性

由于  $G_3$  为易碎件，其装载需要获得充分支撑、其上方不得继续堆载。因此，不将易碎件  $G_3$  直接纳入主体层模板搜索，而是作为一种顶部封顶层进行单独分析。设主体层组合方案为  $S$ ，若在其顶部加入  $G_3$  封顶层后得到新方案  $S'$ ，重新检验：

- 在有效装载高度内，顶部剩余高度是否足以容纳  $G_3$
- 下层是否能为  $G_3$  提供有效支撑
- 加入  $G_3$  后整车空间利用率与载重利用率是否提升

### 5.3.3 基于约束判据的方案可行性验证

根据上文思路中对单车装载问题的边界约束、非重叠约束、支撑稳定性约束、易碎件约束及局部承压约束进行的完整建模，基于上述约束判据对装载方案进行可行性验证。进行依次检查货物是否超出车厢有效空间、货物之间是否存在几何重叠、非地面货物是否具有有效支撑、易碎件是否满足“充分支撑且上方无载荷”的特殊要求，以及各接触区域的局部压强是否超过允许上限。

### 5.3.4 利用网格搜索进行层模板组合与优化

获得若干单层模板后，进一步将其视为整车装载的基本模块。设候选模板集合为  $\mathcal{T}$ ，对每一模板  $T \in \mathcal{T}$ ，记其使用层数为  $n_T$ ，则整车总高度、重量和体积分别可表示为：

$$H_{\text{tot}} = \sum_{T \in \mathcal{T}} n_T h(T) \quad (39)$$

$$M_{\text{tot}} = \sum_{T \in \mathcal{T}} n_T M(T) \quad (40)$$

$$V_{\text{tot}} = \sum_{T \in \mathcal{T}} n_T V(T) \quad (41)$$

考虑车辆顶部安全间隙  $g$ ，整车层组合需满足：

$$\sum_{T \in \mathcal{T}} n_T h(T) \leq H_{(t)} - g \quad (42)$$

$$\sum_{T \in \mathcal{T}} n_T M(T) \leq Q_{(t)} \quad (43)$$

其中  $H_{(t)}$  和  $Q_{(t)}$  分别为车型  $t$  的车厢高度与最大载重。

在完成单层模板构造后将每一种候选层模板视为一个离散装载单元，并以模板使用次数作为决策变量，采用网格搜索方法对模板数量组合进行逐点枚举。设候选模板集合为  $\mathcal{T} = \{T_1, T_2, \dots, T_m\}$ ，其中  $x_j$  表示模板  $T_j$  的使用次数，则有

$$x_j \in [0, U_j] \cap \mathbb{Z}, \quad j = 1, 2, \dots, m,$$

其中  $U_j$  为模板  $T_j$  在货物总量与车厢容量约束下的理论最大使用次数。由此，所有可能的模板数量组合构成一个有限的离散搜索网格：

$$\mathcal{G} = \{(x_1, x_2, \dots, x_m) \mid x_j \in [0, U_j] \cap \mathbb{Z}\}.$$

对于任一网格点  $(x_1, x_2, \dots, x_m)$ ，计算对应方案中各类货物的装载数量及总层高：

$$N_g = \sum_{j=1}^m x_j n_g(T_j), \quad g = 1, 2, 3, 4, 5 \quad (44)$$

$$H = \sum_{j=1}^m x_j h(T_j) \quad (45)$$

在搜索过程中，对网格中的每一个候选点依次进行检验：判断五类货物数量是否满足精确闭合要求、总层高是否超过车型所提供的有效总高度、易碎品  $G_3$  的有效支撑和封顶约束。若某一网格点不满足上述任一条件，则直接剔除；若满足全部约束，则将其记为可行解，并与当前最优方案比较，保留总层高更小、结构更紧凑的方案。

5.4 最优装配方案展示与分析

5.4.1 单独选用车型 1 的最优装配方案

根据所建立的单层模板构造与组合优化模型，计算得到单独选用车型 2 的情况下所需车辆数为 **18 辆**，且得到的三维装载结果如图 6 所示。可以看出，货物在车厢内部沿长度与宽度方向进行了较为规则的排布，整体呈现出明显的分层堆叠特征，货物之间基本贴合<sup>[8]</sup>，顶部保留了预设安全空间，从而满足装载边界与安全间隙要求。该方案下，车型 1 的空间利用率达到 96.5%，对应有效装载体积为  $18.732\text{ m}^3$ ；载重利用率为 52.6%，实际装载质量为 3156kg。从可视化结果来看，车厢内部空隙较少，说明模型在空间组织方面具有较好的紧凑性与可行性。

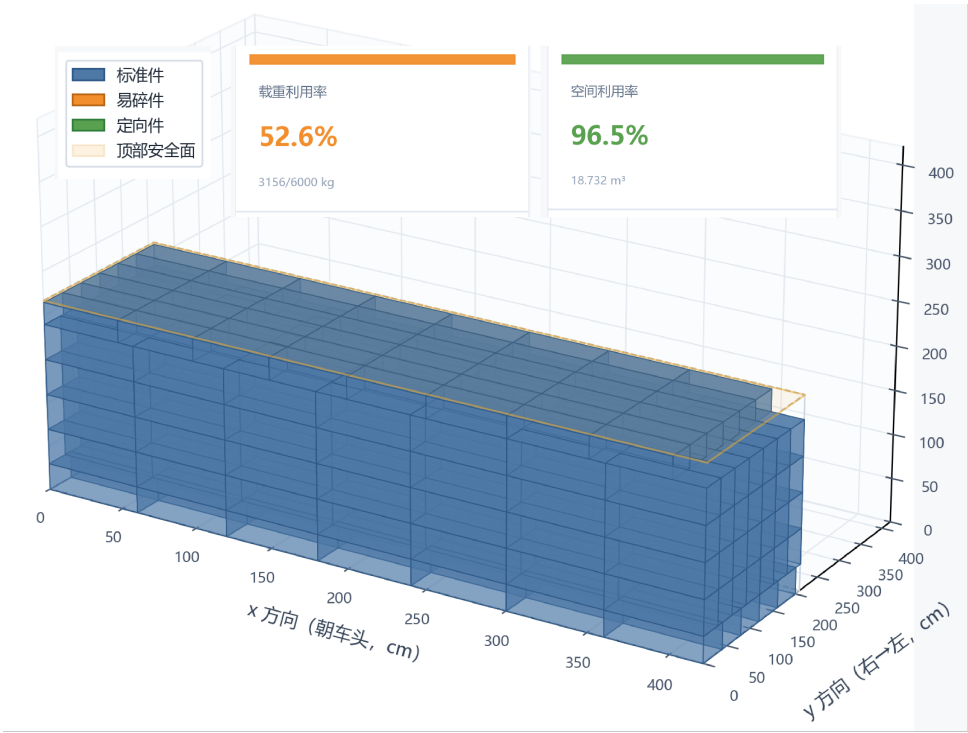


图 6 单独选用车型 1 的最优装配方案样例

进一步分析可知，该方案的空间利用率显著高于载重利用率，表明在车型 1 条件下，本题装载的主要约束并非车辆载重上限，而是车厢内部的几何空间与层间堆叠结构。在满足不重叠、顶部安全面及摆放规则等条件后，车厢有效容积已接近充分利用，但车辆的剩余载质量仍较大，因此该类货物组合呈现出较明显的“先受空间限制、后受载重限制”特征。

5.4.2 单独选用车型 2 的最优装配方案

根据所建立的单层模板构造与组合优化模型，计算得到单独选用车型 2 的情况下所需车辆数为 **8 辆**，且得到的三维装载结果如图 7 所示。由图 7 可见，货物在车厢内部沿长度、宽度及高度方向实现了较为规整的分层布置，不同类别货物按照约束要求完成组合堆叠，整体排布紧凑，车厢顶部同样保留了预设安全空间，满足装载边界、层间支撑及安全间隙等约束条件。该方案下，车型 2 的空间利用率达到 96.3%，对应有效装载体积为  $40.103\text{ m}^3$ ；载重利用率达到 74.2%，实际装载质量为 7424kg。从可视化结果可以看出，车厢内部剩余空隙较少，说明模型在车型 2 条件下同样能够形成结构稳定、排列紧凑的可行装载方案。

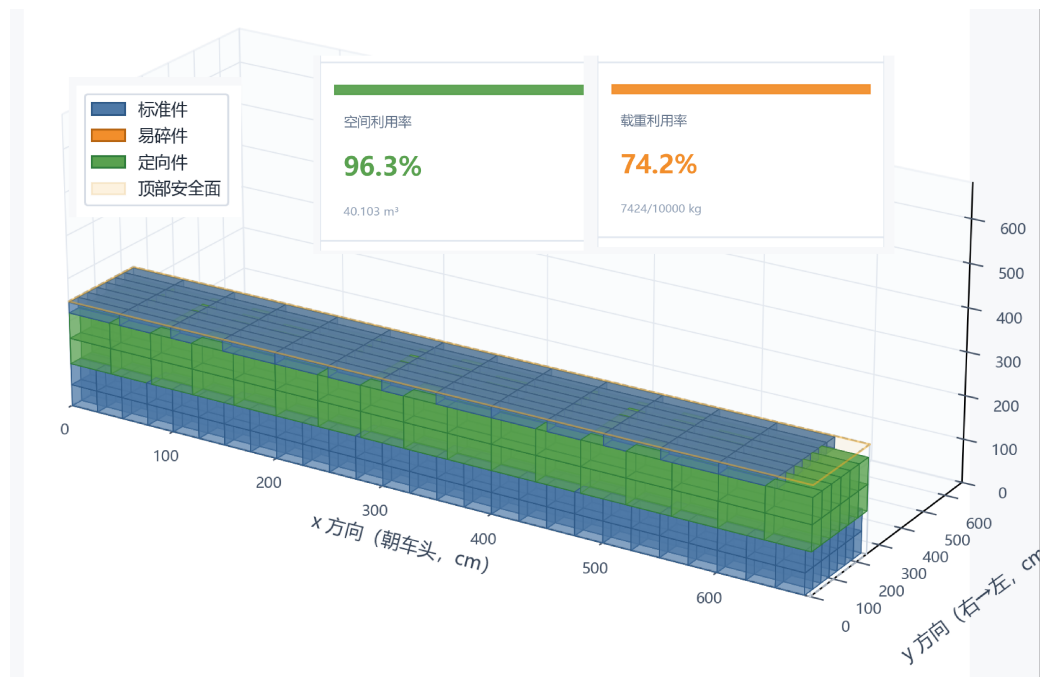


图 7 单独选用车型 1 的最优装配方案样例

进一步分析可知，车型 2 的空间利用率与车型 1 基本相当，均保持在较高水平，说明所建立的模板化装载模型在两种车型下都能充分挖掘车厢内部几何空间；但车型 2 的载重利用率明显高于车型 1，表明在更大装载空间和更高载重上限条件下，车型 2 能够在保证空间高效利用的同时装载更多质量的货物。

5.5 模型灵敏度分析

为检验问题一模型对关键参数变化的稳定性与适应性，本文选取顶部安全空间参数进行单因素灵敏度分析。在保持货物种类与数量、车型参数、层模板库及其余约束条件不变的前提下，分别取为 0, 3, 6, 10, 15, 20, 25, 30 cm，并对车型 1 与车型 2 重新求解装载方案，记录空间利用率、载重利用率、装载总高度及层数等指标的变化情况。其中， $TOP\_GAP = g = 3\text{ cm}$  为题设采用的基准值。通过比较不同安全空间下模型输出结果的变化幅度，可以判断该参数扰动对装载方案的影响程度，并进一步验证模型在实际装载安全要求变化下的鲁棒性，得到图 8 结果所示。

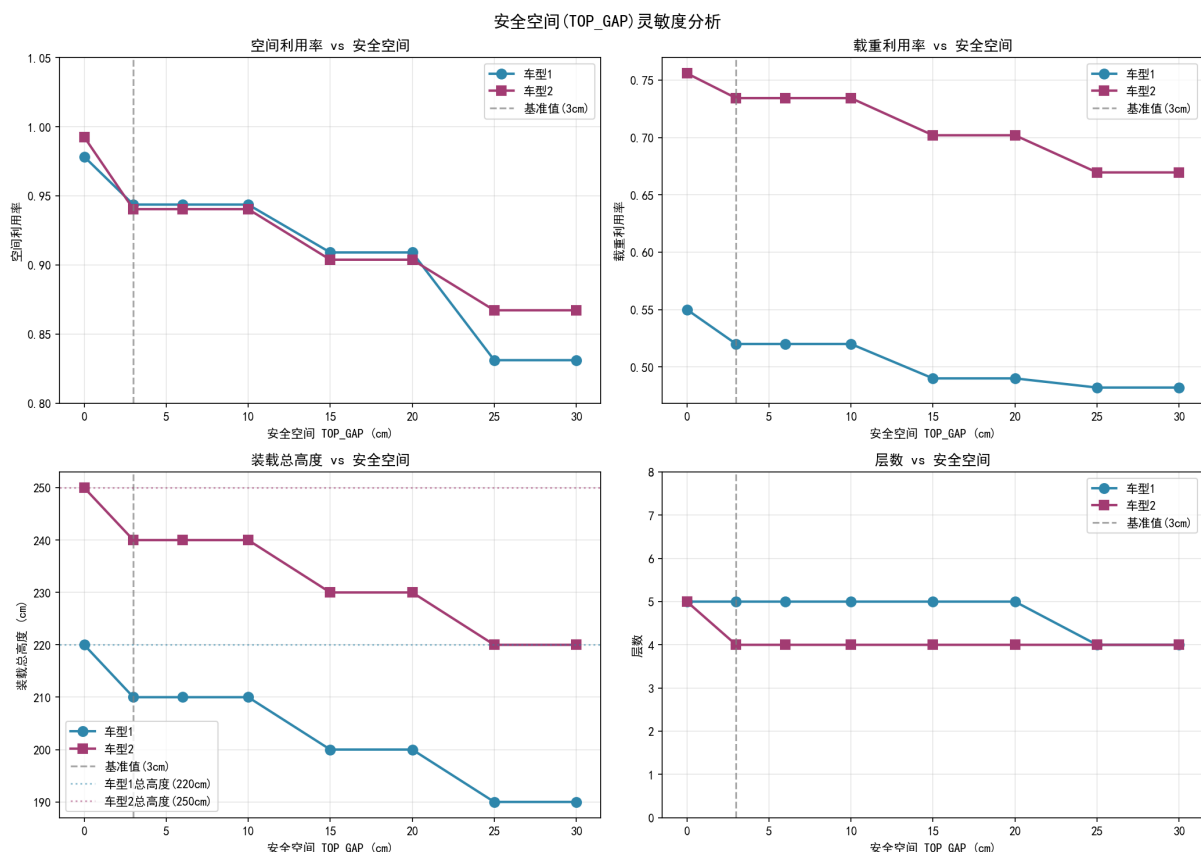


图 8 问题一灵敏度分析结果图

由图 8 可知,  $TOP\_GAP$  增大后, 空间利用率均逐步下降, 且在较小扰动范围内变化较为平缓, 在较大安全空间下下降明显。这表明模型在基准值附近具有较好的稳定性, 但当顶部预留空间过大时, 会显著削弱车厢有效容积, 降低整体装载效率。

综合来看, 问题一模型对  $TOP\_GAP$  在基准值附近的小幅波动不敏感, 具有较好的局部鲁棒性; 但当该参数超过一定阈值后, 装载方案会发生明显改变, 装载性能下降加快。因此, 取  $TOP\_GAP = 3$  cm 作为基准参数是合理的: 既保留了必要的安全裕度, 又避免了过大预留空间对装载能力造成的显著损失。

## 六、问题二的模型建立与求解

问题一中已围绕单种车型的空间利用率与载重利用率优化, 建立了单车三维装箱模型, 并提出了基于层模板构造与层序组合优化的求解思路。问题二则是在此基础上的进一步推广, 研究多车型条件下联合运输方案优化。需要在车型 1 与车型 2 之间进行选择, 将五类货物分配到若干辆车中, 并保证装载仍然满足可行性要求。

基于问题一所构建的货物姿态三维模型与单车型装箱模型, 分别以总运输车辆数最少、总运输成本最低为目标, 构建多车多车型联合优化模型。由于决策参考维度增加, 选择在问题一“层模板构造—层序组合”的分层求解思路拓展, 对货物进行分组再组合, 结合模板枚举与 Pareto 筛选, 利用剪枝策略得到不同目标下的最优运输方案。

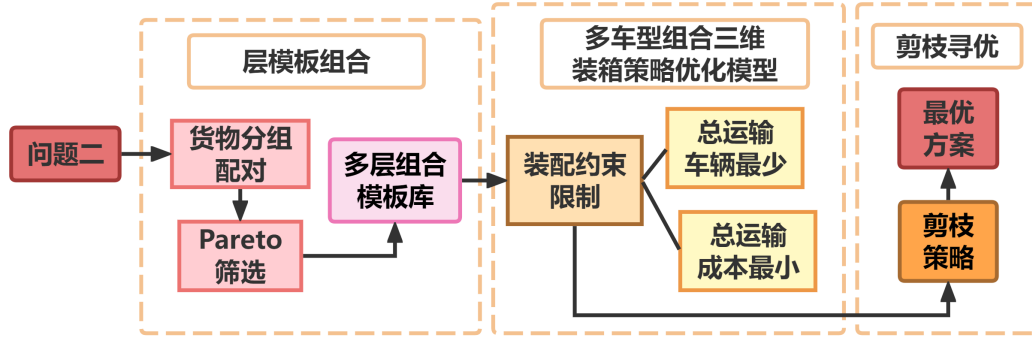


图 9 问题二框架图

### 6.1 进行货物分组配对以枚举单层模板

为提高层模板构造的针对性，在固定  $G_4$ 、 $G_5$  的向上姿态后，对货物进行分组配对处理。基于层高匹配关系，具体划分如下：

- 配对组 1： $G_2 + G_4$ ； $G_4$  固定高 50 cm， $G_2$  可形成 50 cm 高层，因此两者可围绕 50 cm 高度体系构造混合层；
- 配对组 2： $G_1 + G_5$ ； $G_5$  固定高 60 cm， $G_1$  可形成 60 cm 高层，因此两者可围绕 60 cm 高度体系构造混合层；
- 特殊组： $G_3$  单独处理； $G_3$  由于只能位于顶层且必须由标准件支撑，故单独作为顶层模块处理。

该分组策略将原本五类货物同时耦合的复杂搜索问题，拆解为两个主模板体系和一个封顶体系，从而显著降低组合复杂度。其中，单层模板是装箱方案的基本结构单元，后续全局方案均由这些模板叠加形成。

设单层模板集合为  $\mathcal{T}$ 。对于任意模板  $T \in \mathcal{T}$ ，定义：

- $n_g(T)$ ：模板  $T$  中第  $g$  类货物的件数；
- $h(T)$ ：模板  $T$  的层高；
- $M(T)$ ：模板  $T$  的总重量；
- $A(T)$ ：模板  $T$  的底面覆盖面积。

进一步定义其覆盖率为

$$\eta(T) = \frac{A(T)}{L_{(T)} \times W_{(T)}} \quad (46)$$

#### • 纯层模板

纯层模板是指一层中仅放置同一类货物。对于每类货物，在其合法姿态下计算其在车厢底面上的最大平铺数量，并据此形成纯层模板。



## • 混合层模板

为提高底面利用率，可以在纯层基础上进一步构造混合层模板。混合层仅在同一配对组内部生成，即在  $G_2 + G_4$  之间构造 50 cm 高混合层；在  $G_1 + G_5$  之间构造 60 cm 高混合层。

在同一层内，两种货物通过对车厢底面进行规则分割完成混合摆放。选择如图 10 所示四类分割方式。

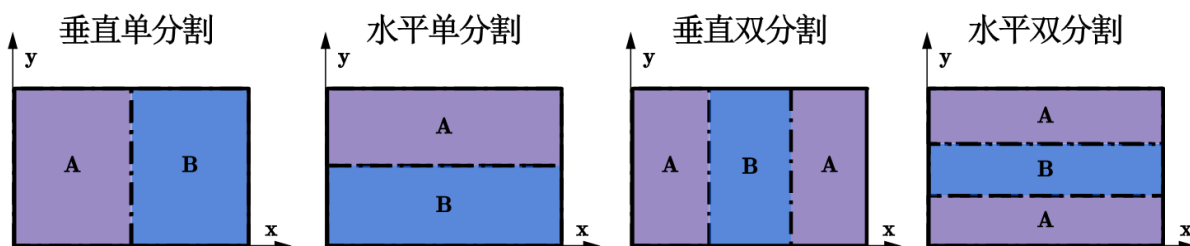


图 10 四类分割方式

垂直单分割；沿 X 轴切一刀，A、B 边放不同货物水平单分割；沿 Y 轴切一刀，A、B 边放不同货物垂直双分割；沿 X 轴切两刀，形成三段，A-B-A 或 B-A-B 模式水平双分割。沿 Y 轴切两刀，形成三段，A-B-A 或 B-A-B 模式

这些分割方式本质上都是将一个大型矩形底面拆分为若干规则子区域，再分别填充两类货物，从而兼顾层高一致性、件数效率与底面完整支撑性。

## 6.2 进行 Pareto 筛选对模板库进行压缩

在纯层与混合层全部枚举完成后，会生成大量候选模板。若直接将全部模板用于后续组合优化，会导致状态数量过多，从而增加计算复杂度。为此采用 Pareto 非支配筛选方法，对模板库进行压缩。

若某模板在层高不更大的前提下，两类货物的装载件数均不低于另一模板，则后者在后续精确覆盖中不具备保留价值。即对于同一配对组中的两个模板  $T_1, T_2$ ，若满足：

$$n_A(T_2) \geq n_A(T_1), \quad n_B(T_2) \geq n_B(T_1), \quad h(T_2) \leq h(T_1) \quad (47)$$

且至少有一个不等式严格成立，则认为模板  $T_1$  被模板  $T_2$  支配，可从候选集合中删除。经过 Pareto 筛选后，模板数量大幅下降，但主要高质量候选均被保留。

## 6.3 构建以总运输车辆数最少为目标的三维装箱优化模型

根据问题二要求，以总运输车辆数最少为目标，即为在满足全部货物必须被运输、每辆车内部装载可行以及两种车型可选的前提下，尽可能减少所启用车辆的总数。与问题一相比，问题二的核心变化在于“货物属于哪一辆车”以及“这辆车是什么车型”，是基于问题一单车装载模型，增加一个车辆选择与货物分配环节。某件货物被分配到某辆具体车辆中，则该车辆内部的一切装载安排仍完全服从问题一中的各种约束。



### 6.3.1 定义模型决策变量

由于需要同时描述多辆车，设候选车辆集合为

$$\mathcal{R} = \{1, 2, \dots, R_{\max}\}$$

其中  $R_{\max}$  为足够大的候选车辆上界。

同时，设货物总集合为

$$\mathcal{I} = \mathcal{I}_{G1} \cup \mathcal{I}_{G2} \cup \mathcal{I}_{G3} \cup \mathcal{I}_{G4} \cup \mathcal{I}_{G5}$$

其中  $\mathcal{I}_{G1}, \mathcal{I}_{G2}, \mathcal{I}_{G3}, \mathcal{I}_{G4}, \mathcal{I}_{G5}$  分别表示五类货物的集合。

根据题目分析，定义以下决策变量以准确刻画多车运输过程：

#### (1) 车辆车型选择变量

对每一辆候选车辆  $r \in \mathcal{R}$ ，定义二元变量

$$q_r^{(1)} = \begin{cases} 1, & \text{第 } r \text{ 辆车选用车型 1} \\ 0, & \text{第 } r \text{ 辆车未选用车型 1} \end{cases} \quad q_r^{(2)} = \begin{cases} 1, & \text{若第 } r \text{ 辆车选用车型 2} \\ 0, & \text{第 } r \text{ 辆车未选用车型 2} \end{cases} \quad (48)$$

于是，第  $r$  辆车是否被启用可表示为：

$$q_r = q_r^{(1)} + q_r^{(2)} \quad (49)$$

- 当  $q_r = 0$  时，表示该候选车辆未被使用；
- 当  $q_r = 1$  时，表示该车辆被启用，且其车型由  $q_r^{(1)}$  和  $q_r^{(2)}$  唯一确定。

每辆车最多只能选用一种车型，因此有约束：

$$q_r^{(1)} + q_r^{(2)} \leq 1, \quad \forall r \in \mathcal{R} \quad (50)$$

车辆  $r$  的几何尺寸和载重能力随车型选择而变化，因此其长度、宽度、有效高度和载重上限分别可写为

$$L_r = 420q_r^{(1)} + 680q_r^{(2)} \quad (51)$$

$$W_r = 210q_r^{(1)} + 245q_r^{(2)} \quad (52)$$

$$H'_r = (220 - g)q_r^{(1)} + (250 - g)q_r^{(2)} \quad (53)$$

$$Q_r = 6000q_r^{(1)} + 10000q_r^{(2)} \quad (54)$$

其中  $g = 3 \text{ cm}$  为与问题一相同的顶部安全间隙， $H'_r$  表示车辆  $r$  的有效装载高度。

#### (2) 货物分配变量

为表示货物  $i$  是否被安排到车辆  $r$  中，定义二元变量

$$u_{ir} = \begin{cases} 1, & \text{若货物 } i \text{ 被分配到第 } r \text{ 辆车} \\ 0, & \text{若货物 } i \text{ 未被分配到第 } r \text{ 辆车} \end{cases}$$

$u_{ir}$  表示该货物究竟属于哪一辆车。由于题目要求全部货物必须完成运输，因此每件货物必须且只能被分配到一辆实际启用的车辆中，即

$$\sum_{r \in \mathcal{R}} u_{ir} = 1, \quad \forall i \in \mathcal{I} \quad (55)$$

同时，货物只能被分配给已启用车辆，因此有

$$u_{ir} \leq q_r, \quad \forall i \in \mathcal{I}, \forall r \in \mathcal{R} \quad (56)$$

综上，结合问题一中的货物装载变量、姿态选择变量、空间位置变量，最终确定问题二的装载方案可由变量  $\{q_r u_{ir}, \delta_{ik}, x_i, y_i, z_i\}$  共同定义。

### 6.3.2 确定优化模型目标函数

对于问题二的“总运输车辆数量最少”目标，目标函数写为

$$\min Z_1 = \sum_{r \in \mathcal{R}} q_r \quad (57)$$

该式表示最小化所有被实际启用车辆的总数。由于  $q_r = q_r^{(1)} + q_r^{(2)}$ ，因此该目标等价于最小化启用的车型 1 车辆数与车型 2 车辆数之和。

### 6.3.3 确定优化模型的装配约束条件

分析问题二，由于出现了不同车型的选择问题，需要进一步加入车型唯一性约束与货物分配与车辆启用一致性约束。

#### • 货物分配与车辆启用一致性约束

若某辆车未被启用，则任何货物都不能分配给它，因此有

$$u_{ir} \leq q_r, \quad \forall i \in \mathcal{I}, \forall r \in \mathcal{R} \quad (58)$$

#### • 车型唯一性约束

每辆候选车至多对应一种车型，因此有

$$q_r^{(1)} + q_r^{(2)} \leq 1, \quad \forall r \in \mathcal{R} \quad (59)$$

保证车辆类型的唯一性，也使得后续车厢参数可以通过  $q_r^{(1)}$  和  $q_r^{(2)}$  唯一确定。

综上，经过总结得到问题二最小化总运输车辆数的目标函数为

$$\min Z_1 = \sum_{r \in \mathcal{R}} q_r \quad (60)$$

决策变量:  $\mathbf{x} = (q_r, u_{ir}, \delta_{ik}, x_i, y_i, z_i)$

$$\begin{aligned}
& \left\{ \begin{array}{l}
\text{货物分配与车辆启用一致性约束: } u_{ir} \leq q_r, \quad \forall i \in \mathcal{I}, \forall r \in \mathcal{R} \\
\text{车型唯一性约束: } q_r^{(1)} + q_r^{(2)} \leq 1, \quad \forall r \in \mathcal{R} \\
\text{货物完整运输约束: } \sum_{r \in \mathcal{R}} u_i = 1, \quad \forall i \in \mathcal{I} \\
\text{姿态唯一性约束: } \sum_{o \in \mathcal{O}_i} \delta_{i,o} = u_i, \quad \forall i \in \mathcal{I} \\
\text{车厢边界约束: } \begin{cases} 0 \leq x_i \leq L_{(t)} - l_i \\ 0 \leq y_i \leq W_{(t)} - w_i \\ 0 \leq z_i \leq H_{(t)} - g - h_i \end{cases} \\
\text{货物非重叠约束: } \begin{cases} x_i + l_i \leq x_j \text{ 或 } x_j + l_j \leq x_i \text{ 或 } y_i + w_i \leq y_j \text{ 或} \\ y_j + w_j \leq y_i \text{ 或 } z_i + h_i \leq z_j \text{ 或 } z_j + h_j \leq z_i \end{cases} \\
\text{整车载重约束: } \sum_{i \in \mathcal{I}} u_i m_i \leq Q_{(t)} \\
\text{货物间支撑关系约束: } P(i) \neq \emptyset \quad z_i > 0 \\
\text{易碎件约束: } P(i) \subseteq \mathcal{I}_{\text{std}} \quad R_i \subseteq \Omega_i, \quad i \in \mathcal{I}_{\text{frag}}, z_i > 0 \\
\text{标准件与定向件稳定性约束: } g_i \in \Omega_i, \quad i \in (\mathcal{I}_{\text{std}} \cup \mathcal{I}_{\text{dir}}), \quad z_i > 0 \\
\text{局部承压约束: } p_{ji} = \frac{\alpha_{ji} T_i}{A_{ji}/10^4} \leq p_{\max}
\end{array} \right.
\end{aligned}$$

## 6.4 构建以总运输成本最低为目标的三维装箱优化模型

### 6.4.1 确定优化模型目标函数

问题二的第二个目标是在所有货物都被运输完成的前提下，使总运输成本最低。由于两种车型的单车运输成本不同，且车厢空间与载重能力也不同，总运输成本最低的目标约束体系与总运输车辆数最少的完全相同，决策变量也相同，变化的只是目标函数本身。

由车型参数可知，车型 1 的单车成本为 450 元，车型 2 的单车成本为 700 元。因此，总运输成本可写为

$$\min Z_2 = 450 \sum_{r \in \mathcal{R}} q_r^{(1)} + 700 \sum_{r \in \mathcal{R}} q_r^{(2)} \quad (61)$$

由于  $q_r^{(1)}$  与  $q_r^{(2)}$  已经决定了每一辆启用车辆的车型，因此该式准确表示了全部发车方案对应的总运输成本。为了便于比较，也可将其写成

$$\min Z_2 = \sum_{r \in \mathcal{R}} \text{Cost}_r \quad (62)$$

其中  $\text{Cost}_r$  表示第  $r$  辆启用车辆的运输成本，取值由车型决定：若为车型 1 则  $\text{Cost}_r = 450$ ，若为车型 2 则  $\text{Cost}_r = 700$ ，若该车未启用则  $\text{Cost}_r = 0$ 。

决策变量和约束条件与上一模型相同，总结得到最小化总运输成本的目标函数为

$$\min Z_2 = 450 \sum_{r \in \mathcal{R}} q_r^{(1)} + 700 \sum_{r \in \mathcal{R}} q_r^{(2)} \quad (63)$$

## 6.5 采用剪枝策略进行层序组合优化求解

### 6.5.1 基于动态规划的精确覆盖

设  $\mathcal{T}_{24}$  为配对组  $G_2 + G_4$  的 Pareto 最优模板集合,  $\mathcal{T}_{15}$  为配对组  $G_1 + G_5$  的 Pareto 最优模板集合。对任意模板  $T$ , 记模板中货物  $A$ 、 $B$  的件数分别为  $n_A(T)$ 、 $n_B(T)$ ; 模板层高为  $h(T)$ 。

则对于一般配对组  $A + B$ , 其精确覆盖问题可表示为:

$$\sum_{T \in \mathcal{T}} z_T n_A(T) = N_A \quad (64)$$

$$\sum_{T \in \mathcal{T}} z_T n_B(T) = N_B \quad (65)$$

其中  $z_T$  为模板  $T$  的使用次数,  $N_A, N_B$  分别为该配对组两类货物的目标件数。相应总层高为

$$H = \sum_{T \in \mathcal{T}} z_T h(T) \quad (66)$$

求解目标为

$$\min H$$

其中: 对于  $G_2 + G_4$  组, 有  $N_{G_2} = 1000$ ,  $N_{G_4} = 400$ ;

对于  $G_1 + G_5$  组, 有  $N_{G_1} = 800$ ,  $N_{G_5} = 500$ 。

由于两个配对组在模板使用上相互独立, 故可分别求解其精确覆盖, 再与  $G_3$  顶层模板合并形成总层集合。

对于  $G_2 + G_4$  和  $G_1 + G_5$  两个配对组, 分别建立二维动态规划模型。以配对组  $A + B$  为例, 定义状态  $dp[a][b]$ , 表示“恰好装入  $a$  件  $A$  和  $b$  件  $B$  所需的最小总高度”。

对于模板  $T$ , 状态转移方程为

$$dp[a + n_A(T)][b + n_B(T)] = \min(dp[a + n_A(T)][b + n_B(T)], dp[a][b] + h(T)) \quad (67)$$

通过该方法, 可以在模板集合上实现“货物数量精确闭合 + 总层高尽可能小”的覆盖结果。最后, 再叠加  $G_3$  顶层模板, 形成完整层实例集合。

为恢复最优解对应的具体模板组合, 本文在动态规划转移过程中同步记录前驱状态与所使用模板。记前驱记录为  $pre[a][b]$ , 则在获得终点状态后, 可由  $(N_A, N_B)$  反向回溯至  $(0, 0)$ , 从而恢复该配对组的模板使用序列及各模板实例数量, 可分别得到:

- $\mathcal{S}_{24}$ :  $G_2 + G_4$  组的最优层实例集合
- $\mathcal{S}_{15}$ :  $G_1 + G_5$  组的最优层实例集合

再与已确定的  $G_3$  顶层集合  $\mathcal{S}_3$  合并, 可得问题一的总层实例集合

$$\mathcal{S} = \mathcal{S}_{24} \cup \mathcal{S}_{15} \cup \mathcal{S}_3$$

### 6.5.2 基于剪枝策略的分车搜索

设层实例集合为  $\mathcal{S}$ ，对任意层  $s \in \mathcal{S}$ ，记其层高和总重量分别为  $h(s)$  和  $M(s)$ 。定义二元变量

$$a_{sr} = \begin{cases} 1, & \text{若层 } s \text{ 分配到第 } r \text{ 辆车} \\ 0, & \text{否则} \end{cases} \quad r = 1, 2, \dots, 8 \quad (68)$$

则需满足

$$\sum_{r=1}^8 a_{sr} = 1, \quad \forall s \in \mathcal{S} \quad (69)$$

即每一层必须且只能分配到一辆车。对每辆车，还需满足高度约束

$$\sum_{s \in \mathcal{S}} a_{sr} h(s) \leq 247, \quad r = 1, 2, \dots, 8 \quad (70)$$

以及载重约束

$$\sum_{s \in \mathcal{S}} a_{sr} M(s) \leq 10000, \quad r = 1, 2, \dots, 8 \quad (71)$$

此外，含  $G_3$  的车辆必须满足两项结构约束： $G_3$  必须位于车内最上层； $G_3$  下方必须存在标准层支撑。

将层实例按既定顺序依次处理，对当前待分配层  $s_k$ ，依次尝试放入不同辆车中；若放入后仍满足当前约束，则递归处理下一层，否则回溯。搜索过程中，实时维护每辆车的已用高度、已用重量、标准支撑层状态及  $G_3$  放置状态。当全部层实例均被成功分配时，即得到完整可行方案。

为避免指数级无效搜索，本文在分车过程中引入如下三类剪枝策略。

#### (1) 高度剪枝

设第  $r$  辆车当前已用高度为  $H_r^{\text{used}}$ ，若尝试放置层  $s$  后有

$$H_r^{\text{used}} + h(s) > H_{(t)} \quad (72)$$

则该分支不可能满足单车高度约束，立即剪枝。该策略直接利用了车型 2 的物理容量限制，是最基础的可行性筛选条件。

#### (2) 支撑闭合剪枝

$G_3$  作为易碎件，必须位于顶层且由标准层提供有效支撑。因此，在搜索过程中需实时判断剩余标准层是否足以支撑尚未安置的  $G_3$  层。

设当前状态下尚需支撑的  $G_3$  车辆数为  $R_{G3}$ ，尚未分配的可用标准层数为  $R_{std}$ 。若  $R_{std} < R_{G3}$  则后续不可能完成  $G_3$  的支撑闭合，应直接终止该搜索分支。

#### (3) $G_3$ 分布剪枝

设目标方案中  $G_3$  顶层总数为  $N_{G3}^{\text{tar}}$ ，当前已分配  $G_3$  层数为  $N_{G3}^{\text{used}}$ ，剩余待分配  $G_3$  层数为  $N_{G3}^{\text{rem}}$ 。若满足

$$N_{G3}^{\text{used}} > N_{G3}^{\text{tar}} \quad \text{或} \quad N_{G3}^{\text{used}} + N_{G3}^{\text{rem}} < N_{G3}^{\text{tar}} \quad (73)$$

则当前分支必然无法达到目标  $G_3$  分布，直接剪枝。

上述三类剪枝分别对应单车容量约束、 $G_3$  支撑约束和  $G_3$  分布约束。通过将这些结构性信息嵌入搜索过程，算法能够在较浅层级提前截断无效分支，从而显著降低搜索规模。与无约束暴力枚举相比，剪枝后的搜索仅在可行区域附近展开，使货物分配问题具备可计算性。

## 6.6 不同目标下的最优方案展示与对比分析

根据问题二的求解结果，在“总运输车辆数最少”与“总运输成本最低”两个目标下，最优方案均为全部采用 8 辆型号 2，未使用任何型号 1。总运输成本为 5600 元，总运输车辆数为 8 辆。并且可视化得到图 11。

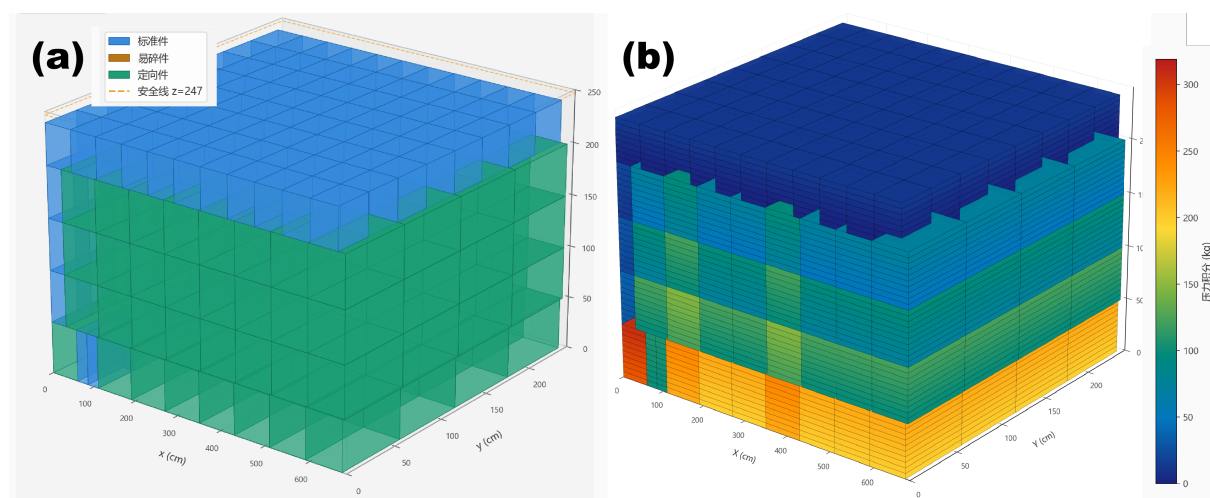


图 11 选用车型 2 的装配方案 (a) 与 3D 压力热力图 (b)

以其中一辆典型车辆为例，其压力热力图如图 11 (b) 所示。该车总载重为 4559 kg，最大局部压强为 319 kg/m<sup>2</sup>，远低于题目规定的 500 kg/m<sup>2</sup> 承压上限，表明堆叠方案满足所有支撑接触面的安全要求。图中颜色分布显示压力主要集中在中下层标准件接触区域，易碎件 ( $G_3$ ) 所在顶层区域压力极小，符合“易碎件上方无堆载、下方由标准件完全支撑”的约束条件。

此外，通过三维坐标输出图 11 (a)，可以逐件还原各货物在车厢内的精确位置与姿态。经统一验证器检验，所有车辆的边界、非重叠、支撑投影及承压指标均通过，证明了 8 辆车型 2 方案的可行性与稳定性。

综上所述，在当前货物结构与约束下，车型 2 在装载效率与成本效益上均优于车型 1 及任何混合车型方案，因此两个优化目标收敛于同一最优解。

## 七、问题三的技术报告

### 7.1 项目目标与管理视角结论

本项目围绕“多场景、多目标货物运输装箱策略优化”问题，面向物流企业的实际决策需求，分别解决了以下三类任务：

1. 在固定车型条件下，求解单车装载效果最优的三维装箱方案
2. 在只允许使用同一车型时，求解完成全部运输任务所需车辆数最少的方案
3. 在两种车型均可选用时，比较“总车辆数最少”与“总运输成本最低”两类决策目标下的运输方案

在当前统一验证口径下，本项目得到的核心结论为：

单车双目标下，车型 2 的综合表现优于车型 1；

若全程只使用车型 1，则最新结果为 18 辆车型 1；

若全程只使用车型 2，则可稳定构造并验证 8 辆车型 2；

在当前固定姿态、支撑、承压和逐件坐标验证口径下，最终采用的最优可行方案仍为 8 辆车型 2；

8 辆车型 2 的总运输成本为 5600 元，低于 18 辆车型 1 的 8100 元

因此在当前数据与约束下，车型 2 方案在车辆数和成本两个维度上均占优。从企业管理角度看，本项目的实际意义在于：不仅给出“装多少”“用几辆车”的结果，还能输出每件货物的坐标、姿态、层序和可行性验证报告，具有较强的工程落地价值。

## 7.2 总体建模思路

要求不仅给出优化值，还必须输出逐件坐标、姿态方向和装载率。因此，本项目没有直接采用单一的大规模三维整数规划，而是构建了“层模板生成 + 组合优化 + 坐标恢复 + 验证校核”的分层求解框架。

这一框架的**基本思想**如下：

1. 先求层：把三维装箱问题拆为若干高质量二维铺排层模板
  2. 再组车：通过动态规划、优先搜索和组合分配，把不同层模板组合成完整车辆
  3. 后恢复坐标：将层级方案恢复为逐件坐标
  4. 最终验证：分别检查高度、重量、姿态、支撑、投影和承压是否满足题意要求
- 该路线具有四个**实际优点**：

1. 能直接输出题目要求的逐件装载方案
2. 便于把 G3、G4、G5 等特殊约束拆开处理
3. 模板复用性强，便于从问题一扩展到问题二
4. 计算效率高于纯逐件全局搜索

## 7.3 程序实现与工程结构

### 1. 程序模块划分

当前全部代码可概括为四类工程<sup>[8]</sup>：

- (1) 负责单车最优层模板搜索与结果导出
- (2) 负责纯车型 1 路线的多车求解
- (3) 负责纯车型 2 路线的精确闭合、8 车分配与验证
- (4) 分别处理“最少车辆”和“最低成本”的混合车型搜索

### 2. 程序执行流程

程序的典型执行流程为：

- (1) 读取基础参数
- (2) 生成层模板
- (3) 求解配对层的精确组合
- (4) 分配车辆并恢复逐件坐标
- (5) 输出 ‘placements.csv’、‘summary.json’ 和总览文件
- (6) 运行统一验证器，检查姿态、支撑和承压是否满足要求

### 3. 工程可交付性

与只输出目标值的理论模型不同，本项目程序能够稳定输出，能直接作为企业装载执行方案的数字底稿：

(1) 每件货物坐标；每件货物姿态；(2) 每辆车层序；每辆车空间利用率与载重利用率；(3) 统一验证报告。

## 7.4 模型与算法性能分析

### 1. 为什么采用“层模板 + 组合优化”

该题的难点不在于单纯把体积装进去，而在于多种约束共同作用。如果直接做逐件全局搜索，状态空间过大、验证代价高、结果也不易稳定复现。层模板方法等价于先把局部结构“标准化”，再在较小搜索空间中进行精确组合。

### 2. 稳定性表现

从当前工程表现看：

- 问题一第一小问能稳定输出单车最优结果；
- 纯车型 2 路线能稳定输出 8 车精确闭合方案；
- 验证器能对姿态、支撑和承压做统一校核；
- 当尝试更激进的 7 车 2+1 车 1 混合路线时，能够识别不可行或未闭合情形。

这说明模型不仅能求出“好结果”，也能排除“看起来更省但实际不可行”的方案。

## 7.5 参数变化对装载率与运输成本的影响

### 1. 车型尺寸参数的影响

车厢长度和宽度增加，会显著提高单层模板的可铺排数量，从而提升空间利用率；有效高度增加，会增加可叠层数，尤其有利于 G1、G2、G5 等标准层和定向层组合；

对本题而言，车型 2 在底面积和高度上的优势，是其能以更少车辆完成任务的根本原因。

### 2. 货物结构比例的影响

标准件占比提高，更容易形成高覆盖率支撑层，空间利用率通常提高；



易碎件 G3 占比提高，会抬高顶层约束压力，并增加支撑层需求，导致车辆数上升；定向件 G4、G5 占比提高，则会增强姿态约束和支撑投影约束。

### 3. 成本参数的影响

若车型 1 与车型 2 的单次运输成本差距缩小，混合方案的吸引力会增强；

若车型 1 成本大幅下降，低成本混合车队可能成为更优解；

但在当前参数下，车型 2 在单位容积和单位载重的成本效率上都更优，因此最终仍是车型 2 主导。

## 7.6 结论与实施建议

综合全部模型、算法和程序结果，本项目的最终建议如下：

1. 若关注单车装载质量，车型 2 的双目标表现优于车型 1；
2. 若只允许单车型执行全量运输，纯车型 2 方案明显优于纯车型 1 方案；
3. 在可混用车型条件下，当前最优可行方案仍为 8 辆车型 2；
4. 从企业执行角度，应优先采用车型 2 作为主力运输车型，并保留基于层模板和逐件坐标的装载输出流程，以提升标准化装载效率。

## 八、模型的评价与推广

### 8.1 模型的优点

- 分层求解策略有效降低复杂度。采用“层模板构造 + 层序组合优化”的分层框架，将三维装箱问题分解为二维铺排与一维堆叠两个子问题，显著减少了决策变量与搜索空间，避免了直接逐件全局搜索的维数爆炸问题。
- 充分考虑多类实际约束。模型系统集成了货物姿态限制、支撑稳定性、易碎件特殊承托要求、局部承压以及顶部安全间隙等真实运输规则，具有较强的工程适用性。
- 支持多车型与多目标优化。问题二中构建了可同时选用车型 1 和车型 2 的联合优化模型，并分别以“总车辆数最少”和“总成本最低”为目标，通过 Pareto 筛选与剪枝策略获得可行解，实现了灵活决策支持。
- 结果可落地、可验证。模型输出每件货物的坐标、姿态、层序等详细信息，并配套统一验证器，能够逐项检查边界、重叠、支撑和承压等约束，保证了方案的可行性与可靠性。

### 8.2 模型的不足与改进

- 层模板构造依赖规则化分割。当前单层模板主要通过固定分割方式（如垂直/水平切割）生成，对于形状差异较大的货物组合可能遗漏更优的排列。

改进方向：引入启发式搜索或整数规划对单层底面布局进行更精细的优化，进一步提高空间利用率。

- 多车型搜索中未采用动态规划精确分配。问题二中车型组合的分配主要依靠剪枝搜索，当货物种类增多或车辆数增大时，搜索效率可能下降。

改进方向：可构建多车型的整数规划模型或使用列生成方法，实现更大规模问题的精确或近似最优求解。

### 8.3 模型的推广

- 适用于多种物流场景，模型框架可推广至冷链物流、快递分拣中心、集装箱码头等场景，只需调整货物类别、车辆尺寸和约束参数即可复用。
- 支持动态货量与多周期调度。在层模板与车辆分配的基础上，可进一步整合时间维度，构建“多周期-多车型”联合调度模型，用于每日运输计划的自动化生成。
- 结合智能算法提升性能。层模板组合与车辆分配模块可嵌入遗传算法、模拟退火或强化学习等智能优化方法，适应更复杂的货物分布和约束变化。

## 参考文献

- [1] 刘旺盛, 魏琦, 曾燕. 多温共配模式下三维装箱与车辆路径协同优化 [J]. 集美大学学报 (自然科学版), 2026, 31(1): 77-93. DOI: 10.19715/j.jmuzr.2026.01.08.
- [2] 徐翔斌, 马国飞. 基于“构造-学习-强化”的超启发式算法求解三维装箱问题 [J/OL]. 计算机集成制造系统, 2026. <https://doi.org/10.13196/j.cims.2025.0368>.
- [3] 张长勇, 张宇浩, 李铮. 基于树结构与深度强化学习的在线三维装箱算法 [J]. 包装工程, 2026, 47(5): 130-143. DOI: 10.19554/j.cnki.1001-3563.2026.05.015.
- [4] 马莹, 周煜, 方启玮, 夏盛炜. 解决物流装箱问题的三维装箱算法 [J]. 科学技术与工程, 2025, 25(27): 11692-11704. DOI: 10.12404/j.issn.1671-1815.2407329.
- [5] 封丛, 丁楠, 邓续晖, 冶瑞. 双目标下集装箱多式联运路径-方式优化 [J]. 交通科技与经济, 2026, 28(3): 40-47. DOI: 10.19348/j.cnki.issn1008-5696.2026.03.006.
- [6] 初阳, 燕雪峰, 张玄烨, 徐云雯, 李德伟. 一类三维装箱问题的多智能体分层强化学习求解算法研究 [J]. 控制理论与应用, 2025, 42(12): 2569-2576. DOI: 10.7641/CTA.2025.50058.
- [7] 王菀莹. 自动化立体仓库货物混合装箱与货位分配优化研究 [D]. 兰州交通大学, 2022.
- [8] Gemini, Ai 工具 (Ai 使用说明在附件中)

## 附 录

### 附录 1 支撑材料清单

| 支撑材料 |      |                              |
|------|------|------------------------------|
| 问题一  | 程序文件 | 第一题第一问.py                    |
| 问题一  | 程序文件 | 第一题第二问车辆 2.py                |
| 问题一  | 程序文件 | solve_truck1_full.py         |
| 问题二  | 程序文件 | solve_min_trucks.py          |
| 问题二  | 程序文件 | monte_carlosolve_min_cost.py |
| -    | 说明文件 | AI 工具使用详情                    |

### 附录 2 问题一的 Python 代码

#### 2.1 问题一第一小问的 Python 代码 1

```
from __future__ import annotations
```

```
"""
```

```
run_validate_q1_direct.py
```

点击 RUN 即可验证 `placements.csv` 是否满足当前更正版口径。

使用方法：

1. 把本文件放到工程根目录（与 `project/` 同级）

2. 只改下方“用户配置区”

3. 点击 RUN 或命令行执行：`python run_validate_q1_direct.py`

```
"""
```

```
import json
```

```
from pathlib import Path
```

```
from project.validate_placements_current_rules import validate,
    save_report
```

```
# =====
```

```
# 用户配置区（只改这里）
```

```
# =====
```

```
CSV_PATH = r"outputs_q1_best/truck_2/placements.csv"
```

```
TRUCK_TYPE = 2
```

```
G4_VERTICAL = "H"
```

```
G5_VERTICAL = "H"
```

```

REPORT_FILENAME = "validation_report.json"
PRINT_ALL_ERRORS = True
# =====

def main() -> None:
    csv_path = Path(CSV_PATH)
    report = validate(csv_path, TRUCK_TYPE, G4_VERTICAL, G5_VERTICAL)
    report_path = csv_path.parent / REPORT_FILENAME
    save_report(report, report_path)

    print("Validation report")
    print(f"    csv                        : {report['csv_path']}")
    print(f"    truck_type                   : {report['truck_type']}")
    print(f"    scenario                     : {report['scenario']}")
    print(f"    placed_count                 : {report['placed_count']}")
    print(f"    total_weight_kg              : {report['total_weight_kg']:.3f}")
    print(f"    total_volume_cm3             : {report['total_volume_cm3']:.3f}")
    print(f"    space_util_total             : {report['space_utilization_total']:.6f}")
    print(f"    space_util_usable            : {report['space_utilization_usable']:.6f}")
    print(f"    weight_utilization           : {report['weight_utilization']:.6f}")
    )
    print(f"    error_count                  : {report['error_count']}")
    print(f"    warning_count                : {report['warning_count']}")
    print(f"    report_json                  : {report_path}")
    if PRINT_ALL_ERRORS and report["errors"]:
        print("\nErrors:")
        for e in report["errors"]:
            print(f"    - {e}")
    elif not report["errors"]:
        print("\n All implemented checks passed under current project rules.")

if __name__ == "__main__":
    main()

```

## 2.2 问题一第一小问的 Python 代码 2

```

from __future__ import annotations
import csv, json, math
from collections import Counter
from dataclasses import dataclass, field
from pathlib import Path

```

```

from typing import Dict, List, Optional, Tuple

TOP_GAP = 3.0
EPS = 1e-6

TRUCKS = {
    1: {"L": 420.0, "W": 210.0, "H": 220.0, "Q": 6000.0, "cost":
        450.0},
    2: {"L": 680.0, "W": 245.0, "H": 250.0, "Q": 10000.0, "cost":
        700.0},
}

ITEMS = {
    "G1": {"cat": "std", "L0": 60.0, "W0": 40.0, "H0": 30.0, "w": 12.0,
        "count": 800},
    "G2": {"cat": "std", "L0": 50.0, "W0": 35.0, "H0": 25.0, "w": 8.0,
        "count": 1000},
    "G3": {"cat": "frag", "L0": 70.0, "W0": 50.0, "H0": 40.0, "w":
        15.0, "count": 300},
    "G4": {"cat": "dir", "L0": 80.0, "W0": 60.0, "H0": 50.0, "w": 25.0,
        "count": 400},
    "G5": {"cat": "dir", "L0": 40.0, "W0": 40.0, "H0": 60.0, "w": 18.0,
        "count": 500},
}

SCENARIO_DEFAULT = {"G4": "H", "G5": "H"}

@dataclass
class Ori:
    code: str; x_dim: str; y_dim: str; z_dim: str
    l: float; w: float; h: float; v_dim: str; h_rot: int

@dataclass
class Item:
    id: str; group: str; cat: str
    L0: float; W0: float; H0: float; w: float; vol: float
    oris: List[Ori] = field(default_factory=list)

    def dims_dict(self) -> Dict[str, float]:
        return {"L": self.L0, "W": self.W0, "H": self.H0}

```

```

@dataclass
class Placement:
    item_id: str; group: str; cat: str
    x: float; y: float; z: float; l: float; w: float; h: float
    weight: float; ori_code: str; v_dim: str; h_rot: int; can_support:
        bool

    @property
    def volume(self) -> float:
        return self.l * self.w * self.h

@dataclass
class Truck:
    t_type: int; L: float; W: float; H: float; Q: float; cost: float;
        gap: float

    @property
    def usable_h(self) -> float:
        return self.H - self.gap

    @property
    def volume(self) -> float:
        return self.L * self.W * self.H

@dataclass
class TemplateCandidate:
    id: str; t_type: int; main_group: str; layer_h: float
    placements: List[Placement]; objective: float
    main_count: int = 0; layer_weight: float = 0.0
    coverage_ratio: float = 0.0; weight_utilization: float = 0.0
    supportable_area_ratio: float = 0.0

@dataclass
class PackResult:
    t_type: int; placements: List[Placement]
    frag_placed: int; std_placed: int; dir_placed: int
    total_weight: float; total_volume: float
    space_util: float; weight_util: float
    remaining: Dict[str, int]; layer_seq: List[str]

```

```

@dataclass
class TemplatePool:
    candidates: List[TemplateCandidate]
    by_group: Dict[str, List[TemplateCandidate]]

    def get_best(self, group: str) -> Optional[TemplateCandidate]:
        if group not in self.by_group or not self.by_group[group]:
            return None
        return self.by_group[group][0]

    def get_top_k(self, group: str, k: int = 3) -> List[
        TemplateCandidate]:
        if group not in self.by_group:
            return []
        return self.by_group[group][:k]

    def _axis_code(x_dim: str, y_dim: str, z_dim: str) -> str:
        m = {"L": "X", "W": "Y", "H": "Z"}
        return m["L"] + m["W"] + m["H"]

    def _build_ori(item: Item, x_dim: str, y_dim: str, z_dim: str, h_rot:
        int) -> Ori:
        d = item.dims_dict()
        return Ori(code=_axis_code(x_dim, y_dim, z_dim),
                    x_dim=x_dim, y_dim=y_dim, z_dim=z_dim,
                    l=d[x_dim], w=d[y_dim], h=d[z_dim],
                    v_dim=z_dim, h_rot=h_rot)

    def gen_oris(item: Item, scenario: Dict[str, str]) -> List[Ori]:
        raw = []
        if item.cat in {"std", "frag"}:
            verticals = ["L", "W", "H"]
        else:
            verticals = [scenario.get(item.group, "H")]
        for v in verticals:
            h_dims = [d for d in ["L", "W", "H"] if d != v]
            x0, y0 = h_dims[0], h_dims[1]
            raw.append(_build_ori(item, x0, y0, v, 0))
            dd = item.dims_dict()
            if dd[x0] != dd[y0]:
                raw.append(_build_ori(item, y0, x0, v, 1))

```

```

seen = set(); out = []
for o in raw:
    key = (o.l, o.w, o.h)
    if key not in seen: seen.add(key); out.append(o)
return out

def attach_oris(items: List[Item], scenario: Dict[str, str]) -> List[
Item]:
    for item in items:
        item.oris = gen_oris(item, scenario)
    return items

def build_truck(t_type: int) -> Truck:
    spec = TRUCKS[t_type]
    return Truck(t_type=t_type, L=spec["L"], W=spec["W"], H=spec["H"],
        Q=spec["Q"], cost=spec["cost"], gap=TOP_GAP)

def build_items() -> List[Item]:
    items = []
    for group, spec in ITEMS.items():
        for i in range(spec["count"]):
            vol = spec["L0"] * spec["W0"] * spec["H0"]
            items.append(Item(id=f"{group}_{i:05d}", group=group, cat=
                spec["cat"],
                                L0=spec["L0"], W0=spec["W0"], H0=spec["H0"]
                                ),
                            w=spec["w"], vol=vol, oris=[]))
    return items

def build_inventory(items: List[Item]) -> Dict[str, int]:
    return Counter(item.group for item in items)

def _tile_region(item: Item, ori: Ori, x0: float, y0: float,
    reg_L: float, reg_W: float, z: float, prefix: str) ->
    List[Placement]:
    if ori.l <= 0 or ori.w <= 0: return []
    nx = int(reg_L // ori.l); ny = int(reg_W // ori.w)
    places = []; idx = 1
    for ix in range(nx):

```



```

    for iy in range(ny):
        places.append(Placement(
            item_id=f"{prefix}_{idx:03d}", group=item.group, cat=
                item.cat,
            x=x0 + ix * ori.l, y=y0 + iy * ori.w, z=z,
            l=ori.l, w=ori.w, h=ori.h, weight=item.w,
            ori_code=ori.code, v_dim=ori.v_dim, h_rot=ori.h_rot,
            can_support=(item.cat != "frag"))
        idx += 1
    return places

def enum_templates(t_type: int, scenario: Dict[str, str], groups: List[
    str],
                    alpha_vol: float = 1.0, beta_weight: float = 1.0) ->
                    List[TemplateCandidate]:
    truck = build_truck(t_type)
    prototypes = []
    for g in groups:
        spec = ITEMS[g]
        prototypes.append(Item(id=f"{g}_PROTO", group=g, cat=spec["cat"
            ],
                                L0=spec["L0"], W0=spec["W0"], H0=spec["
                                    H0"],
                                w=spec["w"], vol=spec["L0"]*spec["W0"]*
                                    spec["H0"])))
    attach_oris(prototypes, scenario)
    proto_map = {it.group: it for it in prototypes}

    candidates = []
    for group in groups:
        item = proto_map[group]
        for ori in item.oris:
            main_places = _tile_region(item, ori, 0.0, 0.0, truck.L,
                truck.W, 0.0, f"{group}_MAIN")
            if not main_places: continue
            covered = sum(p.l * p.w for p in main_places)
            floor = truck.L * truck.W
            cov_ratio = covered / floor if floor > 0 else 0.0
            layer_w = sum(p.weight for p in main_places)
            w_util = layer_w / truck.Q if truck.Q > 0 else 0.0
            layer_vol = sum(p.volume for p in main_places)
            supportable = sum(p.l * p.w for p in main_places if p.
                can_support)

```

```

sup_ratio = supportable / covered if covered > 0 else 0.0
obj = alpha_vol * cov_ratio + beta_weight * w_util
main_cnt = len(main_places)
candidates.append(TemplateCandidate(
    id=f"T{t_type}_{group}_{ori.code}", t_type=t_type,
    main_group=group,
    layer_h=ori.h, placements=main_places, objective=obj,
    main_count=main_cnt, layer_weight=layer_w,
    coverage_ratio=cov_ratio, weight_utilization=w_util,
    supportable_area_ratio=sup_ratio))
candidates.sort(key=lambda c: (c.objective, c.coverage_ratio, c.
    weight_utilization), reverse=True)
return candidates

def build_template_pool(t_type: int, scenario: Dict[str, str], groups:
    List[str],
                        alpha_vol: float = 1.0, beta_weight: float =
                        1.0) -> TemplatePool:
    candidates = enum_templates(t_type, scenario, groups, alpha_vol,
        beta_weight)
    by_group: Dict[str, List[TemplateCandidate]] = {}
    for cand in candidates:
        if cand.main_group not in by_group: by_group[cand.main_group] =
            []
        by_group[cand.main_group].append(cand)
    for group in by_group:
        by_group[group].sort(key=lambda c: (c.objective, c.
            coverage_ratio, c.weight_utilization), reverse=True)
    return TemplatePool(candidates=candidates, by_group=by_group)

def shift_template(template: TemplateCandidate, z_off: float, layer_idx
    : int,
                    actual: Optional[int] = None) -> List[Placement]:
    res = []; places = template.placements
    if actual is not None and actual < len(places): places = places[:
        actual]
    for i, p in enumerate(places, start=1):
        res.append(Placement(
            item_id=f"L{layer_idx:02d}_{p.item_id}_{i:03d}", group=p.
            group, cat=p.cat,
            x=p.x, y=p.y, z=p.z + z_off, l=p.l, w=p.w, h=p.h, weight=p.
            weight,

```

```

        ori_code=p.ori_code, v_dim=p.v_dim, h_rot=p.h_rot,
        can_support=p.can_support))
    return res

def can_place(template: TemplateCandidate, inv: Dict[str, int],
              curr_w: float, max_w: float, curr_h: float, max_h: float,
              allow_partial: bool = False) -> bool:
    if template.main_group not in inv: return False
    if not allow_partial and inv[template.main_group] < template.
        main_count: return False
    if allow_partial and inv[template.main_group] <= 0: return False
    if curr_w + template.layer_weight > max_w: return False
    if curr_h + template.layer_h > max_h: return False
    return True

def apply_template(template: TemplateCandidate, inv: Dict[str, int],
                  allow_partial: bool = False) -> Tuple[Dict[str, int],
                  int]:
    new_inv = dict(inv); cnt = 0
    if template.main_group in new_inv:
        cnt = min(new_inv[template.main_group], template.main_count) if
            allow_partial else template.main_count
        new_inv[template.main_group] -= cnt
    return new_inv, cnt

def select_next_layer(pool: TemplatePool, inv: Dict[str, int],
                    curr_w: float, max_w: float, curr_h: float, max_h:
                    float,
                    prefer: List[str], must_support: bool = True,
                    allow_partial: bool = False) -> Optional[
                    TemplateCandidate]:
    candidates = []
    for group in prefer:
        if group not in inv or inv[group] <= 0: continue
        for template in pool.get_top_k(group, 3):
            if can_place(template, inv, curr_w, max_w, curr_h, max_h,
                allow_partial):
                if must_support and template.supportable_area_ratio <
                    0.999: continue
                candidates.append(template)
    if not candidates: return None

```

```

candidates.sort(key=lambda c: (c.objective, c.coverage_ratio, c.
    weight_utilization), reverse=True)
return candidates[0]

def can_place_g3(layer_seq: List[str], layer_idx: int) -> bool:
    if "G3" in layer_seq: return False
    if layer_idx == 1: return True
    if layer_seq and layer_seq[-1] in ["G1", "G2"]: return True
    return False

def find_best_g3(pool: TemplatePool, inv: Dict[str, int],
    curr_w: float, max_w: float, curr_h: float, max_h:
        float,
    layer_seq: List[str], layer_idx: int) -> Optional[
        Tuple[TemplateCandidate, int]]:
    if inv.get("G3", 0) <= 0: return None
    if not can_place_g3(layer_seq, layer_idx): return None
    best = None; best_cnt = 0; best_score = -float('inf')
    for template in pool.get_top_k("G3", 3):
        if can_place(template, inv, curr_w, max_w, curr_h, max_h,
            allow_partial=True):
            cnt = min(inv["G3"], template.main_count)
            actual_w = cnt * template.placements[0].weight if template.
                placements else 0
            actual_vol = cnt * (template.placements[0].l * template.
                placements[0].w * template.placements[0].h) if template.
                placements else 0
            score = cnt * 1000 + actual_vol * 0.001 + actual_w * 0.01 -
                template.layer_h * 0.1
            if score > best_score: best_score = score; best = template;
                best_cnt = cnt
    if best: return (best, best_cnt)
    return None

def search_g3_comb(pool: TemplatePool, inv: Dict[str, int], truck:
    Truck):
    combs = [
        ["G1", "G1", "G1", "G1", "G3"], ["G1", "G1", "G1", "G2", "G3"],
        ["G1", "G1", "G2", "G2", "G3"], ["G1", "G2", "G2", "G2", "G3"],
        ["G2", "G2", "G2", "G2", "G3"], ["G1", "G1", "G1", "G3"],
        ["G1", "G1", "G2", "G3"], ["G1", "G2", "G2", "G3"],

```

```

["G2", "G2", "G2", "G3"], ["G1", "G1", "G3"],
["G1", "G2", "G3"], ["G2", "G2", "G3"],
["G1", "G3"], ["G2", "G3"],
["G4", "G1", "G1", "G3"], ["G4", "G1", "G2", "G3"],
["G4", "G2", "G2", "G3"], ["G4", "G1", "G3"],
["G4", "G2", "G3"], ["G4", "G3"],
["G5", "G1", "G1", "G3"], ["G5", "G1", "G2", "G3"],
["G5", "G2", "G2", "G3"], ["G5", "G1", "G3"],
["G5", "G2", "G3"], ["G5", "G3"], ["G3"],
]
valid = []
for seq in combs:
    places = []; rem = dict(inv); curr_w = 0.0; curr_h = 0.0; ok =
        True
    for idx, group in enumerate(seq, start=1):
        if group not in pool.by_group or not pool.by_group[group]:
            ok = False; break
        template = pool.by_group[group][0]
        cnt = min(rem.get(group, 0), len(template.placements))
        if cnt <= 0: ok = False; break
        if curr_h + template.layer_h > truck.usable_h: ok = False;
            break
        if curr_w + template.layer_weight > truck.Q: ok = False;
            break
        layer_places = shift_template(template, curr_h, idx, cnt)
        places.extend(layer_places)
        rem[group] -= cnt; curr_h += template.layer_h
        curr_w += sum(p.weight for p in layer_places)
    if not ok: continue
    total_vol = sum(p.l * p.w * p.h for p in places)
    space_util = total_vol / truck.volume if truck.volume > 0 else
        0.0
    weight_util = curr_w / truck.Q if truck.Q > 0 else 0.0
    objective = space_util + weight_util
    g3_placed = sum(1 for p in places if p.group == "G3")
    valid.append({"p":places, "r":rem, "w":curr_w, "s":space_util,
        "wt":weight_util, "o":objective, "q":seq, "g3":
            g3_placed})
if not valid: return None
valid.sort(key=lambda x: (x["g3"], x["o"]), reverse=True)
best = valid[0]
return best["p"], best["r"], best["w"], best["s"], best["q"]

```

```

def single_truck_pack_v2(truck: Truck, inv: Dict[str, int], pool:
    TemplatePool,
                        scenario: Dict[str, str], max_layers: int =
                        10) -> PackResult:
    if inv.get("G3", 0) > 0:
        g3_res = search_g3_comb(pool, inv, truck)
        if g3_res:
            places, rem, curr_w, space_util, layer_seq = g3_res
            frag = sum(1 for p in places if p.cat == "frag")
            std = sum(1 for p in places if p.cat == "std")
            dir_c = sum(1 for p in places if p.cat == "dir")
            total_vol = sum(p.l * p.w * p.h for p in places)
            weight_util = curr_w / truck.Q if truck.Q > 0 else 0.0
            return PackResult(t_type=truck.t_type, placements=places,
                              frag_placed=frag,
                              std_placed=std, dir_placed=dir_c,
                              total_weight=curr_w,
                              total_volume=total_vol, space_util=
                              space_util,
                              weight_util=weight_util, remaining=rem,
                              layer_seq=layer_seq)

    places: List[Placement] = []; curr_w = 0.0; curr_h = 0.0
    layer_seq: List[str] = []; rem = dict(inv)
    max_w = truck.Q; max_h = truck.usable_h
    dir_groups = ["G4", "G5"]; std_groups = ["G1", "G2"]; frag_groups = [
        "G3"]
    dir_rem = sum(rem.get(g, 0) for g in dir_groups)
    std_rem = sum(rem.get(g, 0) for g in std_groups)
    frag_rem = sum(rem.get(g, 0) for g in frag_groups)
    layer_idx = 0; std_cnt = 0

    while layer_idx < max_layers:
        layer_idx += 1; template = None; allow_partial = False
        if template is None and std_rem > 0:
            template = select_next_layer(pool, rem, curr_w, max_w,
                                         curr_h, max_h, std_groups, must_support=True,
                                         allow_partial=False)
            if not template:
                template = select_next_layer(pool, rem, curr_w, max_w,
                                             curr_h, max_h, std_groups, must_support=True,
                                             allow_partial=True)
                allow_partial = template is not None
            if template and not allow_partial: std_cnt += 1

```

```

if template is None and dir_rem > 0:
    template = select_next_layer(pool, rem, curr_w, max_w,
                                curr_h, max_h, dir_groups, must_support=True,
                                allow_partial=False)
    if not template:
        template = select_next_layer(pool, rem, curr_w, max_w,
                                    curr_h, max_h, dir_groups, must_support=True,
                                    allow_partial=True)
        allow_partial = template is not None
if not template: break
rem, actual = apply_template(template, rem, allow_partial)
layer_places = shift_template(template, curr_h, layer_idx,
                              actual if allow_partial else None)
places.extend(layer_places)
actual_w = sum(p.weight for p in layer_places)
actual_h = template.layer_h if layer_places else 0.0
curr_w += actual_w; curr_h += actual_h
layer_seq.append(template.main_group)
dir_rem = sum(rem.get(g,0) for g in dir_groups)
std_rem = sum(rem.get(g,0) for g in std_groups)
frag_rem = sum(rem.get(g,0) for g in frag_groups)
if dir_rem == 0 and std_rem == 0 and frag_rem == 0: break

total_vol = sum(p.l * p.w * p.h for p in places)
frag_placed = sum(1 for p in places if p.cat == "frag")
std_placed = sum(1 for p in places if p.cat == "std")
dir_placed = sum(1 for p in places if p.cat == "dir")
return PackResult(t_type=truck.t_type, placements=places,
                  frag_placed=frag_placed,
                  std_placed=std_placed, dir_placed=dir_placed,
                  total_weight=curr_w,
                  total_volume=total_vol,
                  space_util=total_vol / truck.volume if truck.
                      volume > 0 else 0.0,
                  weight_util=curr_w / truck.Q if truck.Q > 0 else
                      0.0,
                  remaining=rem, layer_seq=layer_seq)

def estimate_truck_count(inv: Dict[str, int], t_type: int, exp_util:
float = 0.65) -> Dict:
    total_vol = 0.0; total_w = 0.0; total_items = 0
    for group, count in inv.items():
        if count <= 0: continue

```

```

        spec = ITEMS[group]; item_vol = spec["L0"] * spec["W0"] * spec[
            "H0"]
        total_vol += item_vol * count; total_w += spec["w"] * count;
        total_items += count
    truck = TRUCKS[t_type]; truck_vol = truck["L"] * truck["W"] * truck
        ["H"]; truck_cap = truck["Q"]
    min_vol = math.ceil(total_vol / truck_vol); min_w = math.ceil(
        total_w / truck_cap)
    lower = max(min_vol, min_w)
    estimated = max(math.ceil(total_vol / (truck_vol * exp_util)), math
        .ceil(total_w / (truck_cap * exp_util)))
    return {"total_vol":total_vol,"total_w":total_w,"total_items":
        total_items,
            "truck_vol":truck_vol,"truck_cap":truck_cap,"min_vol":
                min_vol,
            "min_w":min_w,"lower":lower,"estimated":estimated}

def multi_truck_alloc_v2(t_type: int, scenario: Dict[str, str], items:
    List[Item],
                            max_trucks: int = 50, verbose: bool = True) ->
    Dict:
    truck = build_truck(t_type); inv = build_inventory(items)
    est = estimate_truck_count(inv, t_type); all_groups = list(ITEMS.
        keys())
    pool = build_template_pool(t_type, scenario, all_groups)
    if verbose:
        print("\n" + "=" * 72)
        print("多车分配 (模板法)")
        print("=" * 72)
        print(f"总货物数           : {est['total_items']}")
        print(f"总货物体积           : {est['total_vol']:.2f} cm^3")
        print(f"总货物重量           : {est['total_w']:.2f} kg")
        print(f"理论最少车辆           : {est['lower']}")
        print(f"估计车辆数           : {est['estimated']}")
    all_res: List[PackResult] = []; rem = dict(inv); truck_idx = 0
    while any(rem.get(g,0) > 0 for g in all_groups) and truck_idx <
        max_trucks:
        truck_idx += 1
        if verbose: print(f"\n[车辆 {truck_idx}] 装箱中...")
        res = single_truck_pack_v2(truck, rem, pool, scenario)
        all_res.append(res); rem = res.remaining
        if verbose:

```



```

        print(f" 已装：易碎件 {res.frag_placed}, 标准件 {res.
              std_placed}, 定向件 {res.dir_placed}")
        print(f" 层序：{'-'.join(res.layer_seq)}")
        print(f" 空间利用率：{res.space_util:.2%}, 载重利用率：{
              res.weight_util:.2%}")
        print(f" 剩余货物：{sum(rem.get(g,0) for g in all_groups)}
              ")
    total_placed = sum(r.frag_placed + r.std_placed + r.dir_placed for
                       r in all_res)
    return {"success":total_placed==est['total_items'], "t_type":t_type,
           "scenario":scenario,
           "truck_count":len(all_res), "est":est, "results":all_res,
           "total_placed":total_placed, "total_items":est['total_items']
           ], "remaining":rem}

def write_csv(res: Dict, csv_path: Path):
    csv_path.parent.mkdir(parents=True, exist_ok=True)
    with csv_path.open('w', encoding='utf-8-sig', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['id', 'group', 'category', 'x', 'y', 'z', 'l', 'w', 'h',
                          'weight'])
        for p in res.get('placements', []):
            writer.writerow([p.item_id, p.group, p.cat, p.x, p.y, p.z,
                             p.l, p.w, p.h, p.weight])

def write_summary(res: Dict, json_path: Path):
    json_path.parent.mkdir(parents=True, exist_ok=True)
    with json_path.open('w', encoding='utf-8') as f:
        json.dump(res, f, ensure_ascii=False, indent=2)

def main():
    t_type = 1; max_trucks = 50
    output_dir = Path(__file__).parent / "outputs"
    output_dir.mkdir(parents=True, exist_ok=True)

    print("=" * 60)
    print("车型1单车运完所有货物（模板法）")
    print("=" * 60)

    items = build_items()

```

```

result = multi_truck_alloc_v2(t_type, SCENARIO_DEFAULT, items,
                               max_trucks, verbose=True)

print("\n" + "=" * 60)
print("装箱结果汇总")
print("=" * 60)
print(f"车型                : {result['t_type']}")
print(f"总车辆数            : {result['truck_count']}")
print(f"总货物数              : {result['total_items']}")
print(f"已装货物数            : {result['total_placed']}")
print(f"是否全部装完          : {'是' if result['success'] else '否'}")
")

print(f"\n{'车辆':^6}{ '易碎件':^8}{ '标准件':^8}{ '定向件':^8}{ '空间利用率':^12}{ '载重利用率':^12}{ '层序':^20}")
print("-" * 66)
for idx, r in enumerate(result["results"], 1):
    seq_str = "-".join(r.layer_seq) if r.layer_seq else "N/A"
    print(f"{idx:^6}{r.frag_placed:^8}{r.std_placed:^8}{r.dir_placed:^8}{r.space_util:^12.1%}{r.weight_util:^12.1%}{seq_str:^20}")

truck_out_dir = output_dir / f"truck_type_{t_type}"
truck_out_dir.mkdir(parents=True, exist_ok=True)
for idx, r in enumerate(result["results"], 1):
    truck_dir = truck_out_dir / f"truck_{idx}"
    truck_dir.mkdir(parents=True, exist_ok=True)
    write_csv({"placements": r.placements}, truck_dir / "placements.csv")
    write_summary({
        "truck_idx": idx, "truck_type": t_type,
        "frag_placed": r.frag_placed, "std_placed": r.std_placed, "dir_placed": r.dir_placed,
        "total_weight": r.total_weight, "total_volume": r.total_volume,
        "space_utilization": r.space_util, "weight_utilization": r.weight_util,
        "layer_sequence": r.layer_seq,
    }, truck_dir / "summary.json")

overall = {
    "success": result["success"], "truck_type": t_type, "scenario": result["scenario"],

```

```

        "total_trucks": result["truck_count"], "total_placed": result["total_placed"],
        "total_items": result["total_items"], "remaining_counts": result["remaining"],
        "estimates": result["est"],
    }
    write_summary(overall, truck_out_dir / "overall_summary.json")
    print(f"\n结果已保存至: {output_dir}")

if __name__ == "__main__":
    main()

```

## 2.3 问题一第二小问的 Python 代码

```

from __future__ import annotations
from pathlib import Path
from typing import List, Tuple
import json, math, csv, ast, re

TRUCK2 = {"L": 680.0, "W": 245.0, "H": 250.0, "Q": 10000.0, "top_gap": 3.0}
USABLE_HEIGHT = TRUCK2["H"] - TRUCK2["top_gap"]
ITEMS = {
    "G1": {"dims": (60.0, 40.0, 30.0), "weight": 12.0, "count": 800, "category": "std"},
    "G2": {"dims": (50.0, 35.0, 25.0), "weight": 8.0, "count": 1000, "category": "std"},
    "G3": {"dims": (70.0, 50.0, 40.0), "weight": 15.0, "count": 300, "category": "frag"},
    "G4": {"dims": (80.0, 60.0, 50.0), "weight": 25.0, "count": 400, "category": "dir"},
    "G5": {"dims": (40.0, 40.0, 60.0), "weight": 18.0, "count": 500, "category": "dir"},
}
GROUPS = ["G1", "G2", "G3", "G4", "G5"]
FIXED_VERTICAL_HEIGHT = {"G4": 50.0, "G5": 60.0}

def save_json(path: Path, obj: dict) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as f:
        json.dump(obj, f, ensure_ascii=False, indent=2)

```

```

def write_csv(path: Path, rows: List[dict], fieldnames: List[str]) ->
None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8-sig", newline="") as f:
        w = csv.DictWriter(f, fieldnames=fieldnames)
        w.writeheader()
        w.writerows(rows)

def unique_base_orientations(group: str, target_height: float) -> List[
Tuple[int, int]]:
    dims = ITEMS[group]["dims"]
    out = set()
    for i, h in enumerate(dims):
        if abs(h - target_height) > 1e-9:
            continue
        base = [dims[j] for j in range(3) if j != i]
        out.add((int(base[0]), int(base[1])))
        out.add((int(base[1]), int(base[0])))
    return sorted(out)

def best_fill_count(L: int, W: int, group: str, target_height: float)
-> Tuple[int, Tuple[int, int]]:
    best = (0, (0, 0))
    for bl, bw in unique_base_orientations(group, target_height):
        cnt = (L // bl) * (W // bw)
        if cnt > best[0]:
            best = (cnt, (bl, bw))
    return best

def pure_layer_options(group: str) -> List[dict]:
    hs = [FIXED_VERTICAL_HEIGHT[group]] if group in
        FIXED_VERTICAL_HEIGHT else sorted(set(float(x) for x in ITEMS[
            group]["dims"]))
    opts = []
    for h in hs:
        cnt, base = best_fill_count(int(TRUCK2["L"]), int(TRUCK2["W"]),
            group, h)
        if cnt <= 0:
            continue
        used_area = cnt * base[0] * base[1]

```

```

    opts.append({
        "pair_key": None,
        "layout_family": "pure",
        "height": float(h),
        group + "_count": int(cnt),
        "used_area_cm2": int(used_area),
        "space_utilization_2d": float(used_area / (TRUCK2["L"] *
            TRUCK2["W"])),
        "layer_weight_kg": float(cnt * ITEMS[group]["weight"]),
        "notes": f"pure_{group}_h{int(h)}",
        "objective": float(used_area / (TRUCK2["L"] * TRUCK2["W"]))
            + 0.15 * (cnt * ITEMS[group]["weight"] / TRUCK2["Q"]),
    })
return opts

def append_pair_candidate(rows: List[dict], gA: str, gB: str, height:
float, family: str, a: int, b: int, area: float, notes: str) -> None
:
    if a <= 0 and b <= 0:
        return
    rows.append({
        "pair_key": f"{gA}+{gB}",
        "layout_family": family,
        "height": float(height),
        f"{gA}_count": int(a),
        f"{gB}_count": int(b),
        "used_area_cm2": float(area),
        "space_utilization_2d": float(area / (TRUCK2["L"] * TRUCK2["W"]
            )),
        "layer_weight_kg": float(a * ITEMS[gA]["weight"] + b * ITEMS[gB]
            ["weight"]),
        "notes": notes,
        "objective": float(area / (TRUCK2["L"] * TRUCK2["W"])) + 0.15 *
            ((a * ITEMS[gA]["weight"] + b * ITEMS[gB]["weight"]) /
            TRUCK2["Q"]),
    })

def dedup_records(records: List[dict], subset_keys: List[str]) -> List[
dict]:
    seen = set()
    out = []
    for r in records:

```

```

        key = tuple(r.get(k) for k in subset_keys)
        if key in seen:
            continue
        seen.add(key)
        out.append(r)
    return out

def enumerate_pair_mixed_layers(gA: str, gB: str, height: float, step:
int = 5) -> List[dict]:
    L = int(TRUCK2["L"])
    W = int(TRUCK2["W"])
    rows: List[dict] = []
    for cut in range(step, L, step):
        for order in [(gA, gB), (gB, gA)]:
            left_g, right_g = order
            cnt1, base1 = best_fill_count(cut, W, left_g, height)
            cnt2, base2 = best_fill_count(L - cut, W, right_g, height)
            if cnt1 <= 0 or cnt2 <= 0:
                continue
            a = cnt1 if left_g == gA else cnt2
            b = cnt2 if right_g == gB else cnt1
            area = cnt1 * base1[0] * base1[1] + cnt2 * base2[0] * base2
                [1]
            append_pair_candidate(rows, gA, gB, height, "
                vertical_split_1cut", a, b, area, f"{left_g}@left|{
                right_g}@right|cut={cut}")
    for cut in range(step, W, step):
        for order in [(gA, gB), (gB, gA)]:
            bot_g, top_g = order
            cnt1, base1 = best_fill_count(L, cut, bot_g, height)
            cnt2, base2 = best_fill_count(L, W - cut, top_g, height)
            if cnt1 <= 0 or cnt2 <= 0:
                continue
            a = cnt1 if bot_g == gA else cnt2
            b = cnt2 if top_g == gB else cnt1
            area = cnt1 * base1[0] * base1[1] + cnt2 * base2[0] * base2
                [1]
            append_pair_candidate(rows, gA, gB, height, "
                horizontal_split_1cut", a, b, area, f"{bot_g}@bottom|{
                top_g}@top|cut={cut}")
    for c1 in range(step, L - 2 * step, step):
        for c2 in range(c1 + step, L - step, step):
            segs = [c1, c2 - c1, L - c2]

```

```

for pat in [(gA, gB, gA), (gB, gA, gB)]:
    a = b = 0
    area = 0.0
    ok = True
    for segL, g in zip(segs, pat):
        cnt, base = best_fill_count(segL, W, g, height)
        if cnt <= 0:
            ok = False
            break
        if g == gA:
            a += cnt
        else:
            b += cnt
        area += cnt * base[0] * base[1]
    if ok:
        append_pair_candidate(rows, gA, gB, height, "
            vertical_split_2cut", a, b, area, f"pattern={pat
            }|cuts={c1},{c2}")
for c1 in range(step, W - 2 * step, step):
    for c2 in range(c1 + step, W - step, step):
        segs = [c1, c2 - c1, W - c2]
        for pat in [(gA, gB, gA), (gB, gA, gB)]:
            a = b = 0
            area = 0.0
            ok = True
            for segW, g in zip(segs, pat):
                cnt, base = best_fill_count(L, segW, g, height)
                if cnt <= 0:
                    ok = False
                    break
                if g == gA:
                    a += cnt
                else:
                    b += cnt
                area += cnt * base[0] * base[1]
            if ok:
                append_pair_candidate(rows, gA, gB, height, "
                    horizontal_split_2cut", a, b, area, f"pattern={
                    pat}|cuts={c1},{c2}")
rows.sort(key=lambda r: (r["objective"], r["space_utilization_2d"],
    r["layer_weight_kg"]), reverse=True)
return dedup_records(rows, ["pair_key", "layout_family", f"{gA}
    _count", f"{gB}_count", "height"])

```

```

def pair_option_family(gA: str, gB: str) -> List[dict]:
    rows = []
    for opt in pure_layer_options(gA):
        rows.append({
            "pair_key": f"{gA}+{gB}",
            "layout_family": "pure",
            "height": opt["height"],
            f"{gA}_count": opt[gA + "_count"],
            f"{gB}_count": 0,
            "used_area_cm2": opt["used_area_cm2"],
            "space_utilization_2d": opt["space_utilization_2d"],
            "layer_weight_kg": opt["layer_weight_kg"],
            "notes": opt["notes"],
            "objective": opt["objective"],
        })
    for opt in pure_layer_options(gB):
        rows.append({
            "pair_key": f"{gA}+{gB}",
            "layout_family": "pure",
            "height": opt["height"],
            f"{gA}_count": 0,
            f"{gB}_count": opt[gB + "_count"],
            "used_area_cm2": opt["used_area_cm2"],
            "space_utilization_2d": opt["space_utilization_2d"],
            "layer_weight_kg": opt["layer_weight_kg"],
            "notes": opt["notes"],
            "objective": opt["objective"],
        })
    hsA = {FIXED_VERTICAL_HEIGHT[gA]} if gA in FIXED_VERTICAL_HEIGHT
        else set(float(x) for x in ITEMS[gA]["dims"])
    hsB = {FIXED_VERTICAL_HEIGHT[gB]} if gB in FIXED_VERTICAL_HEIGHT
        else set(float(x) for x in ITEMS[gB]["dims"])
    for h in sorted(hsA & hsB):
        rows.extend(enumerate_pair_mixed_layers(gA, gB, h, step=5))
    rows.sort(key=lambda r: (r["objective"], r["space_utilization_2d"],
        r["layer_weight_kg"]), reverse=True)
    return dedup_records(rows, [f"{gA}_count", f"{gB}_count", "height"
    ])

def pareto_reduce_pair_options(opts: List[dict], gA: str, gB: str) ->
List[dict]:
    compact = []

```



```

for r in opts:
    a = int(r[f"{gA}_count"])
    b = int(r[f"{gB}_count"])
    h = int(round(float(r["height"]) * 10))
    if a <= 0 and b <= 0:
        continue
    compact.append({"a": a, "b": b, "h10": h, "family": str(r["
        layout_family"]), "notes": str(r["notes"])})
out = []
for i, o in enumerate(compact):
    dominated = False
    for j, p in enumerate(compact):
        if i == j:
            continue
        if p["a"] >= o["a"] and p["b"] >= o["b"] and p["h10"] <= o[
            "h10"] and (p["a"] > o["a"] or p["b"] > o["b"] or p["h10
            "] < o["h10"]):
            dominated = True
            break
    if not dominated:
        out.append(o)
uniq = {}
for o in out:
    uniq[(o["a"], o["b"], o["h10"], o["family"], o["notes"])] = o
return list(uniq.values())

def exact_equal_height_solution_pair(gA: str, gB: str, opts: List[dict
]) -> dict:
    needA = ITEMS[gA]["count"]
    needB = ITEMS[gB]["count"]
    reduced = pareto_reduce_pair_options(opts, gA, gB)
    Wb = needB + 1
    size = (needA + 1) * (needB + 1)
    INF = 10 ** 9
    dp = [INF] * size
    parent = [None] * size
    dp[0] = 0
    for a0 in range(needA + 1):
        base = a0 * Wb
        for b0 in range(needB + 1):
            idx = base + b0
            cur = dp[idx]
            if cur >= INF:

```

```

        continue
    for o in reduced:
        na = a0 + o["a"]
        nb = b0 + o["b"]
        if na > needA or nb > needB:
            continue
        nidx = na * Wb + nb
        nc = cur + o["h10"]
        if nc < dp[nidx]:
            dp[nidx] = nc
            parent[nidx] = (idx, o)
    target = needA * Wb + needB
    if dp[target] >= INF:
        return {"pair_key": f"{gA}+{gB}", "solved": False, "
                candidate_count_after_pareto": len(reduced)}
    sol = []
    cur = target
    while cur != 0:
        prev, o = parent[cur]
        sol.append(o)
        cur = prev
    sol.reverse()
    return {
        "pair_key": f"{gA}+{gB}",
        "solved": True,
        "min_total_height": dp[target] / 10.0,
        "min_layer_count": len(sol),
        "solution": sol,
        "candidate_count_after_pareto": len(reduced),
    }
}

```

```

def best_g3_pure_option() -> dict:
    opts = []
    for h in sorted(set(float(x) for x in ITEMS["G3"]["dims"])):
        cnt, base = best_fill_count(int(TRUCK2["L"]), int(TRUCK2["W"]),
                                    "G3", h)
        if cnt <= 0:
            continue
        used_area = cnt * base[0] * base[1]
        opts.append({
            "group": "G3",
            "height": float(h),
            "count": int(cnt),

```

```

        "base": base,
        "used_area_cm2": int(used_area),
        "space_utilization_2d": float(used_area / (TRUCK2["L"] *
            TRUCK2["W"])),
        "layer_weight_kg": float(cnt * ITEMS["G3"]["weight"]),
        "notes": f"pure_G3_h{int(h)}",
    })
best = None
for o in opts:
    layers = math.ceil(ITEMS["G3"]["count"] / o["count"])
    total_h = layers * o["height"]
    cand = {"option": o, "layers": layers, "total_height": total_h}
    if best is None or total_h < best["total_height"]:
        best = cand
return best

def build_exact_layer_multiset(pair_solutions: List[dict], g3_best:
dict) -> List[dict]:
    layers = []
    for ps in pair_solutions:
        gA, gB = ps["pair_key"].split("+")
        for o in ps["solution"]:
            layer = {"pair_key": ps["pair_key"], "height": o["h10"] /
                10.0, "notes": o["notes"], "layout_family": o["family"],
                "G1": 0, "G2": 0, "G3": 0, "G4": 0, "G5": 0}
            layer[gA] = int(o["a"])
            layer[gB] = int(o["b"])
            layer["layer_weight_kg"] = sum(layer[g] * ITEMS[g]["weight"]
                for g in GROUPS)
            layer["is_g3_layer"] = False
            layers.append(layer)
    rem = ITEMS["G3"]["count"]
    cap = g3_best["option"]["count"]
    height = g3_best["option"]["height"]
    while rem > 0:
        c = min(cap, rem)
        rem -= c
        layers.append({
            "pair_key": "G3", "height": height, "notes": g3_best["
                option"]["notes"], "layout_family": "pure",
            "G1": 0, "G2": 0, "G3": int(c), "G4": 0, "G5": 0,
            "layer_weight_kg": float(c * ITEMS["G3"]["weight"]), "
                is_g3_layer": True

```

```

    })
    return layers

def exact_assign_layers_to_8_trucks(layers: List[dict]) -> dict:
    non_g3 = [x for x in layers if not x["is_g3_layer"]]
    g3_layers = [x for x in layers if x["is_g3_layer"]]
    if len(g3_layers) != 4:
        return {"success": False, "stage": "unexpected_g3_layer_count"}
    support_notes = {"pure_G1_h60", "pure_G1_h30", "pure_G2_h50"}
    def is_support_compat(layer: dict) -> bool:
        return layer["notes"] in support_notes
    caps10 = [1770] * 4 + [2470] * 4
    items = sorted(non_g3, key=lambda x: (x["height"],
        is_support_compat(x), x["layer_weight_kg"]), reverse=True)
    items = [(int(round(x["height"] * 10)), int(round(x["
        layer_weight_kg"]))), x) for x in items]
    memo = set()
    def rem_support(i: int) -> int:
        return sum(1 for j in range(i, len(items)) if is_support_compat
            (items[j][2]))
    def canon(bins: List[dict]):
        def sig(b):
            last_support = is_support_compat(b["layers"][-1]) if b["
                layers"] else False
            return (b["used_h10"], b["used_w"], len(b["layers"]),
                last_support)
        return tuple(sorted(sig(b) for b in bins[:4])) + tuple(sorted(
            sig(b) for b in bins[4:]))
    def dfs(i: int, bins: List[dict]):
        key = (i, canon(bins))
        if key in memo:
            return None
        if i == len(items):
            if all(b["layers"] and is_support_compat(b["layers"][-1])
                for b in bins[:4]):
                return bins
            memo.add(key)
            return None
        need = sum(1 for b in bins[:4] if not (b["layers"] and
            is_support_compat(b["layers"][-1])))
        if rem_support(i) < need:
            memo.add(key)
            return None

```

```

h10, w, layer = items[i]
choices = []
for bi, b in enumerate(bins):
    if b["used_h10"] + h10 <= caps10[bi] and b["used_w"] + w <=
        int(TRUCK2["Q"]):
        score = caps10[bi] - (b["used_h10"] + h10)
        if bi < 4 and is_support_compat(layer):
            score -= 1000
        choices.append((score, bi))
choices.sort()
tried = set()
for _, bi in choices:
    b = bins[bi]
    sig = (b["used_h10"], b["used_w"], len(b["layers"]),
        is_support_compat(layer), bi < 4)
    if sig in tried:
        continue
    tried.add(sig)
    nb = [{"truck_idx": x["truck_idx"], "used_h10": x["used_h10"],
        "used_w": x["used_w"], "layers": x["layers"][:]} for
        x in bins]
    nb[bi]["used_h10"] += h10
    nb[bi]["used_w"] += w
    nb[bi]["layers"].append(layer)
    ans = dfs(i + 1, nb)
    if ans is not None:
        return ans
memo.add(key)
return None
bins = [{"truck_idx": i + 1, "used_h10": 0, "used_w": 0, "layers":
    []} for i in range(8)]
ans = dfs(0, bins)
if ans is None:
    return {"success": False, "stage": "non_g3_assignment_failed"}
for i, g3 in enumerate(sorted(g3_layers, key=lambda x: x["
    layer_weight_kg"], reverse=True)):
    ans[i]["used_h10"] += int(round(g3["height"] * 10))
    ans[i]["used_w"] += int(round(g3["layer_weight_kg"]))
    ans[i]["layers"].append(g3)
total_counts = {g: 0 for g in ITEMS}
trucks = []
ok = True
for i, b in enumerate(ans, start=1):
    if b["used_h10"] > int(round(USABLE_HEIGHT * 10)):

```

```

        ok = False
    if b["used_w"] > int(TRUCK2["Q"]):
        ok = False
    g_counts = {g: 0 for g in ITEMS}
    for layer in b["layers"]:
        for g in ITEMS:
            g_counts[g] += int(layer[g])
            total_counts[g] += int(layer[g])
    trucks.append({
        "truck_idx": i,
        "truck_type": 2,
        "used_height": b["used_h10"] / 10.0,
        "spare_height": USABLE_HEIGHT - b["used_h10"] / 10.0,
        "total_weight": float(b["used_w"]),
        "spare_weight": float(TRUCK2["Q"] - b["used_w"]),
        "layer_count": len(b["layers"]),
        "g_counts": g_counts,
        "layer_sequence": [("G3" if layer["is_g3_layer"] else "+".
            join([g for g in ["G1", "G2", "G4", "G5"] if layer[g] >
                0])) for layer in b["layers"]],
        "layers": b["layers"],
    })
    exact_counts_ok = all(total_counts[g] == ITEMS[g]["count"] for g in
        ITEMS)
    ok = ok and exact_counts_ok
    return {"success": ok, "stage": "packed", "exact_counts_ok":
        exact_counts_ok, "total_counts": total_counts, "trucks": trucks,
        "truck_count": 8}

def matching_base_orientations(group: str, height: float) -> List[Tuple
    [float, float]]:
    dims = ITEMS[group]["dims"]
    out = []
    for i, h in enumerate(dims):
        if abs(h - height) > 1e-9:
            continue
        base = [dims[j] for j in range(3) if j != i]
        out.append((float(base[0]), float(base[1])))
        out.append((float(base[1]), float(base[0])))
    uniq, seen = [], set()
    for x in out:
        if x not in seen:
            seen.add(x)

```

```

        uniq.append(x)
    return uniq

def region_capacities(group: str, height: float, L: float, W: float) ->
    List[dict]:
    out = []
    for l, w in matching_base_orientations(group, height):
        cap = int(L // l) * int(W // w)
        if cap > 0:
            out.append({"l": l, "w": w, "cap": cap})
    out.sort(key=lambda x: (x["cap"], -(x["l"] * x["w"])), reverse=True)
    return out

def choose_orientation_for_count(group: str, height: float, L: float, W
: float, need: int):
    cand_s = region_capacities(group, height, L, W)
    if not cand_s:
        raise ValueError(f"No feasible orientation for {group} h={
            height} region=({L},{W})")
    feasible = [c for c in cand_s if c["cap"] >= need]
    if feasible:
        feasible.sort(key=lambda c: (c["cap"] - need, c["l"] * c["w"]))
        c = feasible[0]
    else:
        c = cand_s[0]
    return c["l"], c["w"], c["cap"]

def place_group_in_region(group: str, count: int, height: float, x0:
float, y0: float, L: float, W: float, z: float, item_counter_start:
int) -> List[dict]:
    if count <= 0:
        return []
    l, w, cap = choose_orientation_for_count(group, height, L, W, count
)
    if cap < count:
        raise ValueError(f"Region too small for {group}: need {count},
            cap {cap}")
    n_x = int(L // l)
    n_y = int(W // w)
    x_margin = max(0.0, L - n_x * l)

```

```

y_margin = max(0.0, W - n_y * w)
x_start = x0 + x_margin / 2.0
y_start = y0 + y_margin / 2.0
rows = []
placed = 0
item_idx = item_counter_start
for iy in range(n_y):
    for ix in range(n_x):
        if placed >= count:
            break
        rows.append({
            "item_id": f"{group}_{item_idx:05d}",
            "group": group,
            "category": ITEMS[group]["category"],
            "x": x_start + ix * l,
            "y": y_start + iy * w,
            "z": z,
            "l": l,
            "w": w,
            "h": height,
            "weight": ITEMS[group]["weight"],
        })
        item_idx += 1
        placed += 1
    if placed >= count:
        break
return rows

def parse_regions_from_layer(layer: dict) -> List[dict]:
    nonzero = [g for g in GROUPS if int(layer.get(g, 0)) > 0]
    if len(nonzero) <= 1:
        g = nonzero[0]
        return [{"group": g, "x0": 0.0, "y0": 0.0, "L": TRUCK2["L"], "W": TRUCK2["W"]}]]
    notes = str(layer.get("notes", "") or "")
    family = str(layer.get("layout_family", "") or "")
    m = re.search(r"([A-Z0-9]+)@left\\|([A-Z0-9]+)@right\\|cut=(\\d+)",
        notes)
    if m:
        g1, g2, cut = m.group(1), m.group(2), float(m.group(3))
        return [{"group": g1, "x0": 0.0, "y0": 0.0, "L": cut, "W": TRUCK2["W"]}]]

```



```

        {"group": g2, "x0": cut, "y0": 0.0, "L": TRUCK2["L"] -
         cut, "W": TRUCK2["W"]}]
m = re.search(r"([A-Z0-9]+)@bottom\\|([A-Z0-9]+)@top\\|cut=(\\d+)",
              notes)
if m:
    g1, g2, cut = m.group(1), m.group(2), float(m.group(3))
    return [{"group": g1, "x0": 0.0, "y0": 0.0, "L": TRUCK2["L"], "
             W": cut},
            {"group": g2, "x0": 0.0, "y0": cut, "L": TRUCK2["L"], "
             W": TRUCK2["W"] - cut}]
m = re.search(r"pattern=(.+)\\|cuts=(\\d+), (\\d+)", notes)
if m:
    pat = ast.literal_eval(m.group(1))
    c1, c2 = float(m.group(2)), float(m.group(3))
    regs = []
    if "vertical" in family:
        widths = [c1, c2 - c1, TRUCK2["L"] - c2]
        x = 0.0
        for g, width in zip(pat, widths):
            regs.append({"group": g, "x0": x, "y0": 0.0, "L": width
                        , "W": TRUCK2["W"]})
            x += width
    elif "horizontal" in family:
        heights = [c1, c2 - c1, TRUCK2["W"] - c2]
        y = 0.0
        for g, h in zip(pat, heights):
            regs.append({"group": g, "x0": 0.0, "y0": y, "L":
                        TRUCK2["L"], "W": h})
            y += h
    if regs:
        return regs
total = sum(int(layer[g]) for g in nonzero)
x = 0.0
regs = []
for i, g in enumerate(nonzero):
    width = TRUCK2["L"] - x if i == len(nonzero) - 1 else round(
        TRUCK2["L"] * int(layer[g]) / max(total, 1), 3)
    regs.append({"group": g, "x0": x, "y0": 0.0, "L": width, "W":
                TRUCK2["W"]})
    x += width
return regs

```

```

def allocate_counts_to_regions(group: str, total_count: int, height:
float, regs: List[dict]) -> List[int]:
    if total_count <= 0:
        return [0] * len(regs)
    caps = []
    for reg in regs:
        cands = region_capacities(group, height, float(reg["L"]), float
(reg["W"]))
        caps.append(cands[0]["cap"] if cands else 0)
    if sum(caps) < total_count:
        raise ValueError(f"Insufficient regional capacity for {group}:
        need {total_count}, caps={caps}")
    order = sorted(range(len(regs)), key=lambda i: caps[i], reverse=
True)
    assigned = [0] * len(regs)
    rem = total_count
    for i in order:
        take = min(rem, caps[i])
        assigned[i] = take
        rem -= take
        if rem <= 0:
            break
    return assigned

def convert_result_to_truck_csvs(pack: dict, output_dir: Path) -> dict:
    output_dir.mkdir(parents=True, exist_ok=True)
    item_counter = {g: 1 for g in GROUPS}
    truck_csvs = []
    truck_summaries = []
    for truck in pack["trucks"]:
        rows = []
        z = 0.0
        for layer in truck["layers"]:
            height = float(layer["height"])
            regs = parse_regions_from_layer(layer)
            by_group = {}
            for idx, reg in enumerate(regs):
                by_group.setdefault(reg["group"], []).append(idx)
            assigned = [0] * len(regs)
            for g, idxs in by_group.items():
                total_cnt = int(layer.get(g, 0))
                if total_cnt <= 0:
                    continue

```

```

        sub = [regs[i] for i in idxs]
        sub_assign = allocate_counts_to_regions(g, total_cnt,
                                                height, sub)
        for iloc, iglobal in enumerate(idxs):
            assigned[iglobal] = sub_assign[iloc]
    for idx, reg in enumerate(regs):
        g = reg["group"]
        cnt = assigned[idx]
        if cnt <= 0:
            continue
        placed = place_group_in_region(g, cnt, height, float(
            reg["x0"]), float(reg["y0"]), float(reg["L"]), float(
            reg["W"]), z, item_counter[g])
        rows.extend(placed)
        item_counter[g] += cnt
    z += height
truck_dir = output_dir / f"truck_{int(truck['truck_idx'])}"
write_csv(truck_dir / "placements.csv", rows, ["item_id", "
    group", "category", "x", "y", "z", "l", "w", "h", "weight"])
summary = {
    "truck_idx": truck["truck_idx"],
    "truck_type": 2,
    "space_utilization": sum(r["l"] * r["w"] * r["h"] for r in
        rows) / (TRUCK2["L"] * TRUCK2["W"] * TRUCK2["H"]),
    "weight_utilization": sum(r["weight"] for r in rows) /
        TRUCK2["Q"],
    "total_items": len(rows),
    "used_height": truck["used_height"],
    "total_weight": truck["total_weight"],
    "g_counts": truck["g_counts"],
    "layer_sequence": truck["layer_sequence"],
}
save_json(truck_dir / "summary.json", summary)
truck_summaries.append(summary)
truck_csvs.append(str(truck_dir / "placements.csv"))
save_json(output_dir / "placements_overview.json", {"truck_count":
    len(truck_csvs), "truck_csvs": truck_csvs})
save_json(output_dir / "fleet_summary.json", {
    "truck_count": len(pack["trucks"]),
    "total_weight": sum(t["total_weight"] for t in pack["trucks"]),
    "total_cost": 700 * len(pack["trucks"]),
    "total_counts": pack["total_counts"],
    "exact_counts_ok": pack["exact_counts_ok"],
    "truck_summaries": truck_summaries,

```

```

    })
    return {"truck_count": len(truck_csvs), "truck_csvs": truck_csvs}

def validate_pack(pack: dict) -> dict:
    errors = []
    warnings = []
    if not pack.get("success", False):
        errors.append("pack.success is false")
    trucks = pack.get("trucks", [])
    if len(trucks) != 8:
        errors.append(f"truck_count != 8, got {len(trucks)}")
    total_counts = {g: 0 for g in GROUPS}
    g3_truck_count = 0
    for t in trucks:
        idx = t["truck_idx"]
        if int(t.get("truck_type", -1)) != 2:
            errors.append(f"truck {idx}: truck_type != 2")
        layers = t.get("layers", [])
        used_h = 0.0
        total_w = 0.0
        g_counts = {g: 0 for g in GROUPS}
        g3_positions = []
        for li, layer in enumerate(layers, start=1):
            used_h += float(layer["height"])
            total_w += float(layer["layer_weight_kg"])
            for g in GROUPS:
                g_counts[g] += int(layer[g])
                total_counts[g] += int(layer[g])
            if int(layer["G3"]) > 0:
                g3_positions.append(li)
                nonzero = [g for g in GROUPS if int(layer[g]) > 0]
                if nonzero != ["G3"]:
                    errors.append(f"truck {idx}: G3 layer {li} is not
                                pure")
        if abs(used_h - float(t["used_height"])) > 1e-6:
            errors.append(f"truck {idx}: used_height mismatch")
        if abs(total_w - float(t["total_weight"])) > 1e-6:
            errors.append(f"truck {idx}: total_weight mismatch")
        if used_h > USABLE_HEIGHT + 1e-9:
            errors.append(f"truck {idx}: used_height exceeds usable
                            height")
        if total_w > TRUCK2["Q"] + 1e-9:

```

```

        errors.append(f"truck {idx}: total_weight exceeds weight
                        limit")
    for g in GROUPS:
        if int(t["g_counts"][g]) != g_counts[g]:
            errors.append(f"truck {idx}: g_counts[{g}] mismatch")
    if g3_positions:
        g3_truck_count += 1
        if len(g3_positions) != 1:
            errors.append(f"truck {idx}: more than one G3 layer")
        pos = g3_positions[0]
        if pos != len(layers):
            errors.append(f"truck {idx}: G3 is not top layer")
        if pos == 1:
            errors.append(f"truck {idx}: G3 lacks standard support
                            below")
        else:
            below = layers[pos - 2]
            if int(below["G1"]) <= 0 and int(below["G2"]) <= 0:
                errors.append(f"truck {idx}: layer below G3 is not
                                standard support")
    for g in GROUPS:
        if total_counts[g] != ITEMS[g]["count"]:
            errors.append(f"global count mismatch for {g}: got {
                            total_counts[g]}, need {ITEMS[g]['count']}")
    return {
        "passed": len(errors) == 0,
        "error_count": len(errors),
        "warning_count": len(warnings),
        "errors": errors,
        "warnings": warnings,
        "g3_truck_count": g3_truck_count,
        "recomputed_total_counts": total_counts,
        "exact_counts_ok": all(total_counts[g] == ITEMS[g]["count"] for
                                g in GROUPS),
    }

```

```

def run(output_dir: str | Path | None = None) -> dict:
    if output_dir is None:
        output_dir = Path(__file__).resolve().parent / "
            outputs_type2_pure8_solver_single"
    output_dir = Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)
    pair_solutions = []

```

```

candidate_rows = []
for gA, gB in [("G2", "G4"), ("G1", "G5")]:
    opts = pair_option_family(gA, gB)
    pair_solutions.append(exact_equal_height_solution_pair(gA, gB,
        opts))
    for r in opts:
        row = {"pair_A": gA, "pair_B": gB, "G1_count": 0, "G2_count": 0, "G4_count": 0, "G5_count": 0}
        row.update(r)
        candidate_rows.append(row)
g3_best = best_g3_pure_option()
layers = build_exact_layer_multiset(pair_solutions, g3_best)
pack = exact_assign_layers_to_8_trucks(layers)
overview = {
    "truck_type": 2,
    "fixed_vertical_height": FIXED_VERTICAL_HEIGHT,
    "pair_solutions": pair_solutions,
    "g3_best": g3_best,
    "pack_result_summary": {
        "success": pack.get("success"),
        "stage": pack.get("stage"),
        "exact_counts_ok": pack.get("exact_counts_ok"),
        "truck_count": pack.get("truck_count"),
    },
}
}
write_csv(output_dir / "type2_pure8_fixedpose_candidates_all.csv",
    candidate_rows, [
        "pair_A", "pair_B", "pair_key", "layout_family", "height", "
        G1_count", "G2_count", "G4_count", "G5_count",
        "used_area_cm2", "space_utilization_2d", "layer_weight_kg", "
        notes", "objective"
    ])
save_json(output_dir / "type2_pure8_support_closer_overview.json",
    overview)
save_json(output_dir / "type2_pure8_support_closer_result.json",
    pack)
placements = convert_result_to_truck_csvs(pack, output_dir / "
    placements")
report = validate_pack(pack)
save_json(output_dir / "type2_pure8_unified_validation_report.json",
    , report)
print("=== 8辆车型2求解完成 ===")
print(f"是否成功: {pack.get('success')}")
print(f"计数精确闭合: {pack.get('exact_counts_ok')}")

```

```

print(f"车辆数: {pack.get('truck_count')}")
for truck in pack.get("trucks", []):
    counts = ",".join(f"{g}={truck['g_counts']}[g]}" for g in GROUPS
                      if truck["g_counts"][g] > 0)
    seq = " | ".join(truck["layer_sequence"])
    print(f"Truck {truck['truck_idx']}: 高度={truck['used_height']:.1f}/{USABLE_HEIGHT:.1f}, 重量={truck['total_weight']:.1f}/{TRUCK2['Q']:.1f}, {counts}")
    print(f" 层序: {seq}")
print(f"验证通过: {report['passed']} (errors={report['error_count']}, warnings={report['warning_count']})")
print(f"输出目录: {output_dir.resolve()}")
return {
    "output_dir": str(output_dir.resolve()),
    "result_json": str((output_dir / "
        type2_pure8_support_closer_result.json").resolve()),
    "placements_dir": str((output_dir / "placements").resolve()),
    "validation_json": str((output_dir / "
        type2_pure8_unified_validation_report.json").resolve()),
    "placements_overview": placements,
    "validation_report": report,
}

if __name__ == "__main__":
    run()

```

## 附录 3 问题二的 Python 代码

### 3.1 总运输车辆数最小目标问题的 Python 代码

```

from __future__ import annotations
from pathlib import Path
from typing import Dict, List, Tuple, Optional
import json, math, csv, ast, re
import pandas as pd

TRUCK = {"L": 680.0, "W": 245.0, "H": 250.0, "gap": 3.0, "Q": 10000.0}
ITEMS = {
    "G1": {"dims": (60.0, 40.0, 30.0), "weight": 12.0, "count": 800, "
        cat": "std"},
    "G2": {"dims": (50.0, 35.0, 25.0), "weight": 8.0, "count": 1000, "
        cat": "std"},

```

```

    "G3": {"dims": (70.0, 50.0, 40.0), "weight": 15.0, "count": 300, "
        cat": "frag"},
    "G4": {"dims": (80.0, 60.0, 50.0), "weight": 25.0, "count": 400, "
        cat": "dir", "vdim": "H"},
    "G5": {"dims": (40.0, 40.0, 60.0), "weight": 18.0, "count": 500, "
        cat": "dir", "vdim": "H"},
}
PAIRS = [("G2", "G4"), ("G1", "G5")]
GROUPS = ["G1", "G2", "G3", "G4", "G5"]
CAT = {"G1": "std", "G2": "std", "G3": "frag", "G4": "dir", "G5": "dir"}
WEIGHT = {"G1": 12.0, "G2": 8.0, "G3": 15.0, "G4": 25.0, "G5": 18.0}

def save_json(path, obj):
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as f:
        json.dump(obj, f, ensure_ascii=False, indent=2)

def get_bases(group, h):
    dims, cat = ITEMS[group]["dims"], ITEMS[group].get("cat", "std")
    bases = set()
    if cat == "dir":
        vdim = ITEMS[group].get("vdim", "H")
        vi = {"L":0,"W":1,"H":2}[vdim]
        if abs(dims[vi]-h) > 1e-9: return []
        base = [dims[j] for j in range(3) if j != vi]
        bases.add((int(base[0]),int(base[1]))); bases.add((int(base[1]),
            int(base[0])))
    else:
        for i, dh in enumerate(dims):
            if abs(dh-h) > 1e-9: continue
            base = [dims[j] for j in range(3) if j != i]
            bases.add((int(base[0]),int(base[1]))); bases.add((int(base
                [1]),int(base[0])))
    return sorted(bases)

def fill_count(L, W, group, h):
    best = (0, (0, 0))
    for bl, bw in get_bases(group, h):
        cnt = (L//bl)*(W//bw)
        if cnt > best[0]: best = (cnt, (bl, bw))
    return best

def pure_opts(group):

```



```

opts = []
for h in sorted(set(float(x) for x in ITEMS[group]["dims"])):
    cnt, base = fill_count(int(TRUCK["L"]), int(TRUCK["W"]), group,
                           h)
    if cnt <= 0: continue
    opts.append({"group":group,"h":float(h),"cnt":int(cnt),"base":
                base,
                "area":int(cnt*base[0]*base[1]),"util":float(cnt*
                    base[0]*base[1]/(TRUCK["L"]*TRUCK["W"])),
                "weight":float(cnt*ITEMS[group]["weight"]), "note":
                    f"{group}_h{int(h)}"})

return opts

def add_pair(rows, gA, gB, h, fam, a, b, area, note):
    if a<=0 and b<=0: return
    rows.append({"pair":f"{gA}+{gB}","fam":fam,"h":float(h),f"{gA}_cnt"
                :int(a),f"{gB}_cnt":int(b),
                "area":float(area),"util":float(area/(TRUCK["L"]*TRUCK
                    ["W"])),
                "weight":float(a*ITEMS[gA]["weight"]+b*ITEMS[gB]["
                    weight"]), "note":note})

def mix_layers(gA, gB, h, step=5):
    L, W = int(TRUCK["L"]), int(TRUCK["W"])
    rows = []
    for cut in range(step, L, step):
        for order in [(gA,gB),(gB,gA)]:
            lg,rg = order
            c1,b1 = fill_count(cut,W,lg,h); c2,b2 = fill_count(L-cut,W,
                rg,h)
            if c1<=0 or c2<=0: continue
            a = c1 if lg==gA else c2; b = c2 if rg==gB else c1
            add_pair(rows,gA,gB,h,"v1",a,b,c1*b1[0]*b1[1]+c2*b2[0]*b2
                [1],f"{lg}|{rg}|{cut}")
    for cut in range(step, W, step):
        for order in [(gA,gB),(gB,gA)]:
            bg,tg = order
            c1,b1 = fill_count(L,cut,bg,h); c2,b2 = fill_count(L,W-cut,
                tg,h)
            if c1<=0 or c2<=0: continue
            a = c1 if bg==gA else c2; b = c2 if tg==gB else c1
            add_pair(rows,gA,gB,h,"h1",a,b,c1*b1[0]*b1[1]+c2*b2[0]*b2
                [1],f"{bg}|{tg}|{cut}")
    for c1 in range(step,L-2*step,step):

```

```

for c2 in range(c1+step,L-step,step):
    segs = [c1,c2-c1,L-c2]
    for pat in [(gA,gB,gA),(gB,gA,gB)]:
        a=b=0; area=0.0; ok=True
        for seg,g in zip(segs,pat):
            cnt,base = fill_count(seg,W,g,h)
            if cnt<=0: ok=False; break
            if g==gA: a+=cnt
            else: b+=cnt
            area += cnt*base[0]*base[1]
        if ok: add_pair(rows,gA,gB,h,"v2",a,b,area,f"{pat}|{c1
            },{c2}")
for c1 in range(step,W-2*step,step):
    for c2 in range(c1+step,W-step,step):
        segs = [c1,c2-c1,W-c2]
        for pat in [(gA,gB,gA),(gB,gA,gB)]:
            a=b=0; area=0.0; ok=True
            for seg,g in zip(segs,pat):
                cnt,base = fill_count(int(TRUCK["L"]),seg,g,h)
                if cnt<=0: ok=False; break
                if g==gA: a+=cnt
                else: b+=cnt
                area += cnt*base[0]*base[1]
            if ok: add_pair(rows,gA,gB,h,"h2",a,b,area,f"{pat}|{c1
                },{c2}")
df = pd.DataFrame(rows)
if df.empty: return df
df["obj"] = df["util"] + 0.15*(df["weight"]/TRUCK["Q"])
return df.sort_values(["obj","util","weight"],ascending=False).
    drop_duplicates(subset=["pair","fam",f"{gA}_cnt",f"{gB}_cnt","h"
],keep="first").reset_index(drop=True)

def pair_opts(gA, gB):
    rows = []
    for opt in pure_opts(gA):
        rows.append({"pair":f"{gA}+{gB}","fam":"pure","h":opt["h"],f"{
            gA}_cnt":opt["cnt"],f"{gB}_cnt":0,
                    "area":opt["area"],"util":opt["util"],"weight":
                        opt["weight"],"note":opt["note"],
                    "obj":opt["util"]+0.15*(opt["weight"]/TRUCK["Q"])
                    })
    for opt in pure_opts(gB):
        rows.append({"pair":f"{gA}+{gB}","fam":"pure","h":opt["h"],f"{
            gA}_cnt":0,f"{gB}_cnt":opt["cnt"],

```

```

        "area":opt["area"], "util":opt["util"], "weight":
            opt["weight"], "note":opt["note"],
        "obj":opt["util"]+0.15*(opt["weight"]/TRUCK["Q"])
    })
common = sorted(set(float(x) for x in ITEMS[gA]["dims"]).
    intersection(set(float(x) for x in ITEMS[gB]["dims"])))
for h in common:
    df = mix_layers(gA,gB,h,step=5)
    if not df.empty: rows.extend(df.to_dict(orient="records"))
df = pd.DataFrame(rows)
return df.sort_values(["obj", "util", "weight"], ascending=False).
    drop_duplicates(subset=[f"{gA}_cnt", f"{gB}_cnt", "h"], keep="first
    ").reset_index(drop=True)

def pareto_filter(df, gA, gB):
    opts = []
    for _,r in df.iterrows():
        a,b = int(r[f"{gA}_cnt"]),int(r[f"{gB}_cnt"])
        h = int(round(float(r["h"])*10))
        if a<=0 and b<=0: continue
        opts.append({"a":a, "b":b, "h10":h, "fam":str(r["fam"]), "note":str
            (r["note"])}))
    out = []
    for i,o in enumerate(opts):
        if not any(p["a"]>=o["a"] and p["b"]>=o["b"] and p["h10"]<=o["
            h10"] and (p["a"]>o["a"] or p["b"]>o["b"] or p["h10"]<o["h10
            "])) for j,p in enumerate(opts) if i!=j):
            out.append(o)
    uniq = {(o["a"],o["b"],o["h10"]): o for o in out}
    return list(uniq.values())

def solve_pair(gA, gB, df):
    needA, needB = ITEMS[gA]["count"], ITEMS[gB]["count"]
    opts = pareto_filter(df, gA, gB)
    Wb = needB+1; INF = 10**9; size = (needA+1)*(needB+1)
    dp = [INF]*size; parent = [None]*size; dp[0] = 0
    for a0 in range(needA+1):
        base = a0*Wb
        for b0 in range(needB+1):
            idx = base+b0; cur = dp[idx]
            if cur >= INF: continue
            for o in opts:
                na, nb = a0+o["a"], b0+o["b"]
                if na>needA or nb>needB: continue

```

```

        nidx = na*Wb+nb; nc = cur+o["h10"]
        if nc < dp[nidx]: dp[nidx] = nc; parent[nidx] = (idx, o
    )
    target = needA*Wb+needB
    if dp[target] >= INF: return {"pair":f"{gA}+{gB}", "ok":False}
    sol = []; cur = target
    while cur != 0:
        prev, o = parent[cur]; sol.append(o); cur = prev
    sol.reverse()
    return {"pair":f"{gA}+{gB}", "ok":True, "h":dp[target]/10.0, "layers":
        len(sol), "sol":sol, "cands":len(opts)}

def best_g3():
    opts = pure_opts("G3"); best = None
    for o in opts:
        layers = math.ceil(ITEMS["G3"]["count"]/o["cnt"]); th = layers*
            o["h"]
        cand = {"opt":o, "layers":layers, "th":th}
        if best is None or cand["th"] < best["th"]: best = cand
    return best

def build_layers(pair_sols, g3_best):
    layers = []
    for ps in pair_sols:
        pair = ps["pair"]; gA, gB = pair.split("+")
        for o in ps["sol"]:
            layer = {"pair":pair, "h":o["h10"]/10.0, "note":o["note"], "
                fam":o["fam"],
                "G1":0, "G2":0, "G3":0, "G4":0, "G5":0}
            layer[gA] = int(o["a"]); layer[gB] = int(o["b"])
            layer["weight"] = sum(layer[g]*ITEMS[g]["weight"] for g in
                ITEMS)
            layer["std"] = (layer["G1"]>0 or layer["G2"]>0); layer["
                is_g3"] = False
            layers.append(layer)
    rem = ITEMS["G3"]["count"]; cap = g3_best["opt"]["cnt"]; h =
        g3_best["opt"]["h"]
    while rem > 0:
        c = min(cap, rem); rem -= c
        layers.append({"pair": "G3", "h":h, "note":g3_best["opt"]["note"],
            "fam": "pure",
            "G1":0, "G2":0, "G3":int(c), "G4":0, "G5":0,
            "weight":float(c*ITEMS["G3"]["weight"]), "std":
                False, "is_g3":True})

```

```

return layers

def assign_trucks(layers):
    non_g3 = [x for x in layers if not x["is_g3"]]
    g3_layers = [x for x in layers if x["is_g3"]]
    if len(g3_layers) != 4: return {"ok":False,"stage":"g3_count",
        "g3_cnt":len(g3_layers)}
    caps = [1770]*4 + [2470]*4
    non_g3 = sorted(non_g3, key=lambda x:(x["h"],x["std"],x["weight"]),
        reverse=True)
    items = [(int(round(x["h"]*10)),int(round(x["weight"])),1 if x["std"]
        " else 0,x) for x in non_g3]
    memo = set()
    def canon(bins):
        first = tuple(sorted((b["h10"],b["w"],1 if b["std"] else 0,len(
            b["layers"]))) for b in bins[:4]))
        last = tuple(sorted((b["h10"],b["w"],1 if b["std"] else 0,len(
            b["layers"]))) for b in bins[4:]))
        return first+last
    def dfs(i, bins):
        key = (i, canon(bins))
        if key in memo: return None
        if i == len(items):
            if all(b["std"] for b in bins[:4]): return bins
            memo.add(key); return None
        rem_std = sum(1 for j in range(i,len(items)) if items[j][2]==1)
        need_std = sum(1 for b in bins[:4] if not b["std"])
        if rem_std < need_std: memo.add(key); return None
        h10,w,std,layer = items[i]; choices = []
        for bi,b in enumerate(bins):
            if b["h10"]+h10<=caps[bi] and b["w"]+w<=int(TRUCK["Q"]):
                res = caps[bi]-(b["h10"]+h10)
                bonus = -1000 if (bi<4 and std and not b["std"]) else 0
                choices.append((res+bonus, bi))
        choices.sort(); tried = set()
        for _,bi in choices:
            b = bins[bi]; sig = (b["h10"],b["w"],b["std"],bi<4)
            if sig in tried: continue
            tried.add(sig)
            nb = [{"idx":x["idx"],"h10":x["h10"],"w":x["w"],"std":x["std"],
                "layers":x["layers"][:]} for x in bins]
            nb[bi]["h10"] += h10; nb[bi]["w"] += w; nb[bi]["std"] = nb[
                bi]["std"] or bool(std)
            nb[bi]["layers"].append(layer)

```

```

        ans = dfs(i+1, nb)
        if ans is not None: return ans
    memo.add(key); return None
bins = [{"idx":i+1,"h10":0,"w":0,"std":False,"layers":[]} for i in
    range(8)]
ans = dfs(0, bins)
if ans is None: return {"ok":False,"stage":"assign_failed"}
g3_layers = sorted(g3_layers, key=lambda x:x["weight"], reverse=
    True)
for i,g3 in enumerate(g3_layers):
    ans[i]["h10"] += int(round(g3["h"]*10)); ans[i]["w"] += int(
        round(g3["weight"]))
    ans[i]["layers"].append(g3)
total = {g:0 for g in ITEMS}; trucks = []; ok = True
for i,b in enumerate(ans, start=1):
    if b["h10"] > int(round((TRUCK["H"]-TRUCK["gap"])*10)): ok =
        False
    if b["w"] > int(TRUCK["Q"]): ok = False
    g_cnt = {g:0 for g in ITEMS}
    for layer in b["layers"]:
        for g in ITEMS: g_cnt[g] += int(layer[g]); total[g] += int(
            layer[g])
    g3_cnt = sum(1 for layer in b["layers"] if layer["is_g3"])
    if g3_cnt > 1: ok = False
    if g3_cnt==1 and not any(layer["std"] for layer in b["layers"]
        if not layer["is_g3"]): ok = False
    trucks.append({"idx":i,"type":2,"h":b["h10"]/10.0,
        "spare_h":(TRUCK["H"]-TRUCK["gap"])-b["h10"]
            /10.0,
        "weight":float(b["w"]), "spare_w":float(TRUCK["Q"]
            -b["w"]),
        "layers":len(b["layers"]), "g_cnt":g_cnt,
        "seq": [("G3" if layer["is_g3"] else "+").join([g
            for g in ["G1","G2","G4","G5"] if layer[g]
                >0])) for layer in b["layers"]],
        "data":b["layers"]})
exact = all(total[g]==ITEMS[g]["count"] for g in ITEMS)
ok = ok and exact
return {"ok":ok,"stage":"packed","exact":exact,"total":total,"
    trucks":trucks,"cnt":8}

def get_bases_cv(group, h):
    dims, cat = DIMS[group], CAT[group]; out = []
    if cat == "dir":

```

```

vdim = VDIM.get(group)
if vdim:
    vi = {"L":0,"W":1,"H":2}[vdim]
    if abs(dims[vi]-h) > 1e-9: return []
    base = [dims[j] for j in range(3) if j != vi]
    out.append((float(base[0]),float(base[1]))); out.append((
        float(base[1]),float(base[0])))
else:
    for i,dh in enumerate(dims):
        if abs(dh-h) > 1e-9: continue
        base = [dims[j] for j in range(3) if j != i]
        out.append((float(base[0]),float(base[1]))); out.append((
            float(base[1]),float(base[0])))
seen = set(); uniq = []
for x in out:
    if x not in seen: seen.add(x); uniq.append(x)
return uniq

def caps_cv(group, h, L, W):
    out = []
    for l,w in get_bases_cv(group, h):
        c = int(L//l)*int(W//w)
        if c > 0: out.append({"l":l,"w":w,"cap":c})
    out.sort(key=lambda x:(x["cap"],-(x["l"]*x["w"])), reverse=True)
    return out

def choose_ori(group, h, L, W, need):
    cands = caps_cv(group, h, L, W)
    if not cands: raise ValueError(f"No orientation for {group} h={h}")
    feasible = [c for c in cands if c["cap"]>=need]
    if feasible:
        feasible.sort(key=lambda c:(c["cap"]-need,-c["cap"],c["l"]*c["w"]
            )))
        return feasible[0]["l"], feasible[0]["w"], feasible[0]["cap"]
    return cands[0]["l"], cands[0]["w"], cands[0]["cap"]

def place(group, cnt, h, x0, y0, L, W, z, start):
    if cnt <= 0: return []
    l,w,cap = choose_ori(group, h, L, W, cnt)
    n_x = int(L//l); n_y = int(W//w); rows = []; idx = start; placed = 0
    for iy in range(n_y):
        for ix in range(n_x):
            if placed >= cnt: break

```

```

        rows.append({"id":f"{group}_{idx:05d}", "group":group, "cat":
            CAT[group],
            "x":x0+ix*l, "y":y0+iy*w, "z":z, "l":l, "w":w, "h":
            h, "weight":WEIGHT[group]})
        idx += 1; placed += 1
    if placed >= cnt: break
return rows

def parse_regs(layer):
    nonzero = [g for g in GROUPS if int(layer.get(g,0))>0]
    if len(nonzero) <= 1: return [{"group":nonzero[0], "x0":0.0, "y0":
        0.0, "L":TRUCK["L"], "W":TRUCK["W"]}]]
    note = str(layer.get("note","") or ""); fam = str(layer.get("fam","")
        or "")
    m = re.search(r"([A-Z0-9+)]\|([A-Z0-9+)]\|(\d+)", note)
    if m:
        g1,g2,cut = m.group(1),m.group(2),float(m.group(3))
        if "v1" in fam: return [{"group":g1, "x0":0.0, "y0":0.0, "L":cut, "
            W":TRUCK["W"]}, {"group":g2, "x0":cut, "y0":0.0, "L":TRUCK["L"]-
            cut, "W":TRUCK["W"]}]]
        if "h1" in fam: return [{"group":g1, "x0":0.0, "y0":0.0, "L":TRUCK
            ["L"], "W":cut}, {"group":g2, "x0":0.0, "y0":cut, "L":TRUCK["L"],
            "W":TRUCK["W"]-cut}]]
    m = re.search(r"\((.+)\)\|(\d+),(\d+)", note)
    if m:
        pat_text,c1,c2 = m.group(1),float(m.group(2)),float(m.group(3))
        pat = ast.literal_eval(f"({pat_text})")
        if "v2" in fam:
            widths = [c1,c2-c1,TRUCK["L"]-c2]; x = 0.0; regs = []
            for g,width in zip(pat,widths): regs.append({"group":g, "x0":
                x, "y0":0.0, "L":width, "W":TRUCK["W"]}); x += width
            return regs
        if "h2" in fam:
            heights = [c1,c2-c1,TRUCK["W"]-c2]; y = 0.0; regs = []
            for g,hh in zip(pat,heights): regs.append({"group":g, "x0":
                0.0, "y0":y, "L":TRUCK["L"], "W":hh}); y += hh
            return regs
    total = sum(int(layer[g]) for g in nonzero); x = 0.0; regs = []
    for i,g in enumerate(nonzero):
        frac = int(layer[g])/max(total,1)
        width = TRUCK["L"]-x if i==len(nonzero)-1 else round(TRUCK["L"]
            ]*frac,3)
        regs.append({"group":g, "x0":x, "y0":0.0, "L":width, "W":TRUCK["W"]
            }); x += width

```



```

    return regs

def alloc(group, total, h, regs):
    if total <= 0: return [0]*len(regs)
    cs = []
    for reg in regs:
        cands = caps_cv(group, h, float(reg["L"]), float(reg["W"]))
        cs.append(cands[0]["cap"] if cands else 0)
    if sum(cs) < total: raise ValueError(f"Capacity too small for {
        group}")
    order = sorted(range(len(regs)), key=lambda i:cs[i], reverse=True)
    assigned = [0]*len(regs); remain = total
    for idx in order:
        take = min(remain, cs[idx]); assigned[idx] = take; remain -=
            take
        if remain <= 0: break
    return assigned

def convert(result_json, output_dir):
    with result_json.open(encoding="utf-8") as f: data = json.load(f)
    output_dir.mkdir(parents=True, exist_ok=True)
    counter = {g:1 for g in GROUPS}; csvs = []
    for truck in data["trucks"]:
        rows = []; z = 0.0
        for layer in truck["data"]:
            h = float(layer["h"]); regs = parse_regs(layer)
            g2idx = {}
            for idx,reg in enumerate(regs): g2idx.setdefault(reg["group"], []).append(idx)
            assigned = [0]*len(regs)
            for g,idxs in g2idx.items():
                need = int(layer.get(g,0))
                if need <= 0: continue
                sub_regs = [regs[i] for i in idxs]; sub_assigned =
                    alloc(g, need, h, sub_regs)
                for loc,glb in enumerate(idxs): assigned[glb] =
                    sub_assigned[loc]
            for idx,reg in enumerate(regs):
                g = reg["group"]; cnt = assigned[idx]
                if cnt <= 0: continue
                placed = place(g, cnt, h, float(reg["x0"]), float(reg["
                    y0"]), float(reg["L"]), float(reg["W"]), z, counter[
                    g])
                rows.extend(placed); counter[g] += cnt

```

```

        z += h
    out_csv = output_dir / f"truck_{int(truck['idx'])}" / "
        placements.csv"
    out_csv.parent.mkdir(parents=True, exist_ok=True)
    with out_csv.open("w", encoding="utf-8-sig", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["id", "group", "cat", "
            x", "y", "z", "l", "w", "h", "weight"])
        writer.writeheader(); writer.writerows(rows)
    csvs.append(str(out_csv))
with (output_dir / "overview.json").open("w", encoding="utf-8") as
f:
    json.dump({"cnt":len(csvs), "csvs":csvs}, f, ensure_ascii=False,
        indent=2)
return {"cnt":len(csvs)}

def main():
    output_dir = Path(__file__).parent / "outputs"
    output_dir.mkdir(parents=True, exist_ok=True)
    print("=" * 60)
    print("最少车辆求解 (车型2, 8车)")
    print("=" * 60)

    pair_dfs, pair_sols = [], []
    for gA, gB in PAIRS:
        df = pair_opts(gA, gB); df.insert(0, "gA", gA); df.insert(1, "gB",
            gB)
        pair_dfs.append(df); pair_sols.append(solve_pair(gA, gB, df))
    all_df = pd.concat(pair_dfs, ignore_index=True)
    top_df = all_df.sort_values(["obj", "util", "weight"], ascending=False
        ).head(250).copy()
    g3_best = best_g3()
    layer_set = build_layers(pair_sols, g3_best)
    pack = assign_trucks(layer_set)

    all_df.to_csv(output_dir/"candidates_all.csv", index=False,
        encoding="utf-8-sig")
    top_df.to_csv(output_dir/"candidates_top.csv", index=False,
        encoding="utf-8-sig")
    save_json(output_dir/"overview.json", {"type":2, "pair_sols":
        pair_sols, "g3_best":g3_best,
            "pack":{"ok":pack["ok"], "cnt":pack.get("cnt")}})
    save_json(output_dir/"result.json", pack)

    print(f"求解成功 = {pack['ok']}")

```

```

print(f"车辆数 = {pack.get('cnt',0)}")

print("\n转换placements.csv...")
convert(output_dir/"result.json", output_dir/"placements")
print("完成")

if __name__ == "__main__":
    main()

```

### 3.2 总运输成本最低目标问题的 Python 代码

```

from __future__ import annotations
from pathlib import Path
from typing import Dict, List, Tuple, Optional
import json, math, csv, ast, re
import pandas as pd

TRUCKS = {
    1: {"L": 420.0, "W": 210.0, "H": 220.0, "gap": 3.0, "Q": 6000.0, "cost": 450.0},
    2: {"L": 680.0, "W": 245.0, "H": 250.0, "gap": 3.0, "Q": 10000.0, "cost": 700.0},
}

ITEMS = {
    "G1": {"dims": (60.0, 40.0, 30.0), "weight": 12.0, "count": 800, "cat": "std"},
    "G2": {"dims": (50.0, 35.0, 25.0), "weight": 8.0, "count": 1000, "cat": "std"},
    "G3": {"dims": (70.0, 50.0, 40.0), "weight": 15.0, "count": 300, "cat": "frag"},
    "G4": {"dims": (80.0, 60.0, 50.0), "weight": 25.0, "count": 400, "cat": "dir", "vdim": "H"},
    "G5": {"dims": (40.0, 40.0, 60.0), "weight": 18.0, "count": 500, "cat": "dir", "vdim": "H"},
}

PAIRS = [("G2", "G4"), ("G1", "G5")]
GROUPS = ["G1", "G2", "G3", "G4", "G5"]
CAT = {"G1": "std", "G2": "std", "G3": "frag", "G4": "dir", "G5": "dir"}
WEIGHT = {"G1": 12.0, "G2": 8.0, "G3": 15.0, "G4": 25.0, "G5": 18.0}

def save_json(path, obj):
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as f:

```

```

        json.dump(obj, f, ensure_ascii=False, indent=2)

def get_bases(group, h):
    dims, cat = ITEMS[group]["dims"], ITEMS[group].get("cat", "std")
    bases = set()
    if cat == "dir":
        vdim = ITEMS[group].get("vdim", "H")
        vi = {"L":0,"W":1,"H":2}[vdim]
        if abs(dims[vi]-h) > 1e-9: return []
        base = [dims[j] for j in range(3) if j != vi]
        bases.add((int(base[0]),int(base[1]))); bases.add((int(base[1]),
            int(base[0])))
    else:
        for i, dh in enumerate(dims):
            if abs(dh-h) > 1e-9: continue
            base = [dims[j] for j in range(3) if j != i]
            bases.add((int(base[0]),int(base[1]))); bases.add((int(base
                [1]),int(base[0])))
    return sorted(bases)

def fill_count(L, W, group, h):
    best = (0, (0, 0))
    for bl, bw in get_bases(group, h):
        cnt = (L//bl)*(W//bw)
        if cnt > best[0]: best = (cnt, (bl, bw))
    return best

def pure_opts(group, t_type=2):
    tk = TRUCKS[t_type]; opts = []
    for h in sorted(set(float(x) for x in ITEMS[group]["dims"])):
        cnt, base = fill_count(int(tk["L"]), int(tk["W"]), group, h)
        if cnt <= 0: continue
        opts.append({"group":group,"h":float(h),"cnt":int(cnt),"base":
            base,
                    "area":int(cnt*base[0]*base[1]),"util":float(cnt*
                        base[0]*base[1]/(tk["L"]*tk["W"])),
                    "weight":float(cnt*ITEMS[group]["weight"]),"note":
                        f"{group}_h{int(h)}","t_type":t_type})
    return opts

def add_pair(rows, gA, gB, h, fam, a, b, area, note, t_type=2):
    if a<=0 and b<=0: return
    tk = TRUCKS[t_type]

```

```

rows.append({"pair":f"{gA}+{gB}", "fam":fam, "h":float(h), f"{gA}_cnt"
            :int(a), f"{gB}_cnt":int(b),
            "area":float(area), "util":float(area/(tk["L"]*tk["W"])),
            ),
            "weight":float(a*ITEMS[gA]["weight"]+b*ITEMS[gB]["weight"]), "note":note, "t_type":t_type})

def mix_layers(gA, gB, h, t_type=2, step=5):
    tk = TRUCKS[t_type]; L, W = int(tk["L"]), int(tk["W"])
    rows = []
    for cut in range(step, L, step):
        for order in [(gA,gB),(gB,gA)]:
            lg,rg = order
            c1,b1 = fill_count(cut,W,lg,h); c2,b2 = fill_count(L-cut,W,rg,h)
            if c1<=0 or c2<=0: continue
            a = c1 if lg==gA else c2; b = c2 if rg==gB else c1
            add_pair(rows,gA,gB,h,"v1",a,b,c1*b1[0]*b1[1]+c2*b2[0]*b2[1],f"{lg}|{rg}|{cut}",t_type)
    for cut in range(step, W, step):
        for order in [(gA,gB),(gB,gA)]:
            bg,tg = order
            c1,b1 = fill_count(L,cut,bg,h); c2,b2 = fill_count(L,W-cut,tg,h)
            if c1<=0 or c2<=0: continue
            a = c1 if bg==gA else c2; b = c2 if tg==gB else c1
            add_pair(rows,gA,gB,h,"h1",a,b,c1*b1[0]*b1[1]+c2*b2[0]*b2[1],f"{bg}|{tg}|{cut}",t_type)
    for c1 in range(step,L-2*step,step):
        for c2 in range(c1+step,L-step,step):
            segs = [c1,c2-c1,L-c2]
            for pat in [(gA,gB,gA),(gB,gA,gB)]:
                a=b=0; area=0.0; ok=True
                for seg,g in zip(segs,pat):
                    cnt,base = fill_count(seg,W,g,h)
                    if cnt<=0: ok=False; break
                    if g==gA: a+=cnt
                    else: b+=cnt
                    area += cnt*base[0]*base[1]
                if ok: add_pair(rows,gA,gB,h,"v2",a,b,area,f"{pat}|{c1},{c2}",t_type)
    for c1 in range(step,W-2*step,step):
        for c2 in range(c1+step,W-step,step):
            segs = [c1,c2-c1,W-c2]

```

```

        for pat in [(gA,gB,gA),(gB,gA,gB)]:
            a=b=0; area=0.0; ok=True
            for seg,g in zip(segs,pat):
                cnt,base = fill_count(int(tk["L"]),seg,g,h)
                if cnt<=0: ok=False; break
                if g==gA: a+=cnt
                else: b+=cnt
                area += cnt*base[0]*base[1]
            if ok: add_pair(rows,gA,gB,h,"h2",a,b,area,f"{pat}|{c1
                },{c2}",t_type)
df = pd.DataFrame(rows)
if df.empty: return df
df["obj"] = df["util"] + 0.15*(df["weight"]/tk["Q"])
return df.sort_values(["obj","util","weight"],ascending=False).
    drop_duplicates(subset=["pair","fam",f"{gA}_cnt",f"{gB}_cnt","h"
        ,"t_type"],keep="first").reset_index(drop=True)

def pair_opts(gA, gB):
    rows = []
    for t_type in [1, 2]:
        tk = TRUCKS[t_type]
        for opt in pure_opts(gA, t_type):
            rows.append({"pair":f"{gA}+{gB}", "fam": "pure", "h":opt["h"],
                f"{gA}_cnt":opt["cnt"],f"{gB}_cnt":0,
                "area":opt["area"],"util":opt["util"],"weight
                    ":opt["weight"],"note":opt["note"],"t_type
                        ":t_type,
                "obj":opt["util"]+0.15*(opt["weight"]/tk["Q
                    ]))})
        for opt in pure_opts(gB, t_type):
            rows.append({"pair":f"{gA}+{gB}", "fam": "pure", "h":opt["h"],
                f"{gA}_cnt":0,f"{gB}_cnt":opt["cnt"],
                "area":opt["area"],"util":opt["util"],"weight
                    ":opt["weight"],"note":opt["note"],"t_type
                        ":t_type,
                "obj":opt["util"]+0.15*(opt["weight"]/tk["Q
                    ]))})
    common = sorted(set(float(x) for x in ITEMS[gA]["dims"]).
        intersection(set(float(x) for x in ITEMS[gB]["dims"])))
    for h in common:
        df = mix_layers(gA,gB,h,t_type,step=5)
        if not df.empty: rows.extend(df.to_dict(orient="records"))
df = pd.DataFrame(rows)

```

```

return df.sort_values(["obj", "util", "weight"], ascending=False).
    drop_duplicates(subset=[f"{gA}_cnt", f"{gB}_cnt", "h", "t_type"],
        keep="first").reset_index(drop=True)

def pareto_filter(df, gA, gB):
    opts = []
    for _, r in df.iterrows():
        a, b = int(r[f"{gA}_cnt"]), int(r[f"{gB}_cnt"])
        h = int(round(float(r["h"])*10))
        if a <= 0 and b <= 0: continue
        opts.append({"a": a, "b": b, "h10": h, "fam": str(r["fam"]), "note": str(
            r["note"]), "t_type": int(r.get("t_type", 2))})
    out = []
    for i, o in enumerate(opts):
        if not any(p["a"] >= o["a"] and p["b"] >= o["b"] and p["h10"] <= o["h10"]
            and (p["a"] > o["a"] or p["b"] > o["b"] or p["h10"] < o["h10"]
            )) for j, p in enumerate(opts) if i != j:
            out.append(o)
    uniq = {(o["a"], o["b"], o["h10"], o["t_type"]): o for o in out}
    return list(uniq.values())

def solve_pair(gA, gB, df):
    needA, needB = ITEMS[gA]["count"], ITEMS[gB]["count"]
    opts = pareto_filter(df, gA, gB)
    Wb = needB+1; INF = 10**9; size = (needA+1)*(needB+1)
    dp = [INF]*size; parent = [None]*size; dp[0] = 0
    for a0 in range(needA+1):
        base = a0*Wb
        for b0 in range(needB+1):
            idx = base+b0; cur = dp[idx]
            if cur >= INF: continue
            for o in opts:
                na, nb = a0+o["a"], b0+o["b"]
                if na > needA or nb > needB: continue
                nidx = na*Wb+nb; nc = cur+o["h10"]
                if nc < dp[nidx]: dp[nidx] = nc; parent[nidx] = (idx, o
                )
    target = needA*Wb+needB
    if dp[target] >= INF: return {"pair": f"{gA}+{gB}", "ok": False}
    sol = []; cur = target
    while cur != 0:
        prev, o = parent[cur]; sol.append(o); cur = prev
    sol.reverse()

```

```

return {"pair":f"{gA}+{gB}", "ok":True, "h":dp[target]/10.0, "layers":
        len(sol), "sol":sol, "cands":len(opts)}

def best_g3():
    best = None
    for t_type in [1, 2]:
        opts = pure_opts("G3", t_type)
        for o in opts:
            layers = math.ceil(ITEMS["G3"]["count"]/o["cnt"]); th =
                layers*o["h"]
            cand = {"opt":o, "layers":layers, "th":th, "t_type":t_type}
            if best is None or cand["th"] < best["th"]: best = cand
    return best

def build_layers(pair_sols, g3_best):
    layers = []
    for ps in pair_sols:
        pair = ps["pair"]; gA,gB = pair.split("+")
        for o in ps["sol"]:
            t_type = o.get("t_type", 2)
            layer = {"pair":pair, "h":o["h10"]/10.0, "note":o["note"], "
                fam":o["fam"],
                "G1":0, "G2":0, "G3":0, "G4":0, "G5":0, "t_type":t_type
            }
            layer[gA] = int(o["a"]); layer[gB] = int(o["b"])
            layer["weight"] = sum(layer[g]*ITEMS[g]["weight"] for g in
                ITEMS)
            layer["std"] = (layer["G1"]>0 or layer["G2"]>0); layer["
                is_g3"] = False
            layers.append(layer)
    rem = ITEMS["G3"]["count"]; cap = g3_best["opt"]["cnt"]; h =
        g3_best["opt"]["h"]; t_type = g3_best["t_type"]
    while rem > 0:
        c = min(cap, rem); rem -= c
        layers.append({"pair": "G3", "h":h, "note":g3_best["opt"]["note"],
            "fam": "pure",
            "G1":0, "G2":0, "G3":int(c), "G4":0, "G5":0, "t_type":
                t_type,
            "weight":float(c*ITEMS["G3"]["weight"]), "std":
                False, "is_g3":True})
    return layers

def assign_trucks(layers):
    non_g3 = [x for x in layers if not x["is_g3"]]

```



```

g3_layers = [x for x in layers if x["is_g3"]]
if len(g3_layers) != 4: return {"ok":False,"stage":"g3_count",
    g3_cnt":len(g3_layers)}
non_g3 = sorted(non_g3, key=lambda x:(x["h"],x["std"],x["weight"]),
    reverse=True)
items = [(int(round(x["h"]*10)),int(round(x["weight"])),1 if x["std
    "] else 0,x) for x in non_g3]
def try_assign(max_t):
    trucks = [{"idx":i+1,"type":None,"h10":0,"w":0,"std":False,"
        layers":[]} for i in range(max_t)]
    def dfs(i):
        if i == len(items): return all(t["std"] for t in trucks
            [:4])
        h10,w,std,layer = items[i]
        for ti,truck in enumerate(trucks):
            for t_type in [1, 2]:
                tk = TRUCKS[t_type]
                cap_h = int((tk["H"]-tk["gap"])*10); cap_w = int(tk
                    ["Q"])
                if truck["h10"]+h10<=cap_h and truck["w"]+w<=cap_w:
                    if truck["type"] is not None and truck["type"
                        ]!=t_type: continue
                    truck["type"] = t_type; truck["h10"] += h10;
                    truck["w"] += w
                    truck["std"] = truck["std"] or bool(std); truck
                        ["layers"].append(layer)
                    if dfs(i+1): return True
                    truck["layers"].pop(); truck["h10"] -= h10;
                    truck["w"] -= w
                    truck["std"] = any(l["std"] for l in truck["
                        layers"])
                    if not truck["layers"]: truck["type"] = None
            return False
        if dfs(0): return {"trucks":trucks,"ok":True}
        return None
    result = None
    for max_t in range(4, 13):
        r = try_assign(max_t)
        if r: result = r; break
    if not result: return {"ok":False,"stage":"assign_failed"}
    trucks = result["trucks"]
    g3_layers = sorted(g3_layers, key=lambda x:x["weight"], reverse=
        True)
    for i,g3 in enumerate(g3_layers):

```

```

        if i < len(trucks):
            trucks[i]["h10"] += int(round(g3["h"]*10)); trucks[i]["w"]
                += int(round(g3["weight"]))
            trucks[i]["layers"].append(g3)
total = {g:0 for g in ITEMS}; total_cost = 0.0; truck_results = [];
    ok = True
for i,t in enumerate(trucks):
    if not t["layers"]: continue
    t_type = t["type"] if t["type"] else 2; tk = TRUCKS[t_type]
    if t["h10"] > int((tk["H"]-tk["gap"])*10): ok = False
    if t["w"] > int(tk["Q"]): ok = False
    g_cnt = {g:0 for g in ITEMS}
    for layer in t["layers"]:
        for g in ITEMS: g_cnt[g] += int(layer.get(g,0)); total[g]
            += int(layer.get(g,0))
    g3_cnt = sum(1 for layer in t["layers"] if layer["is_g3"])
    if g3_cnt > 1: ok = False
    if g3_cnt==1 and not any(layer["std"] for layer in t["layers"]
        if not layer["is_g3"]): ok = False
    total_cost += tk["cost"]
    truck_results.append({"idx":i+1,"type":t_type,"h":t["h10"]
        ]/10.0,
        "spare_h":(tk["H"]-tk["gap"])-t["h10"]/10.0,
        "weight":float(t["w"]), "spare_w":float(tk["Q"]-
            t["w"]),
        "layers":len(t["layers"]), "g_cnt":g_cnt, "cost":
            tk["cost"],
        "seq":[(("G3" if layer["is_g3"] else "+").join([g
            for g in ["G1","G2","G4","G5"] if layer.get
                (g,0)>0])) for layer in t["layers"]],
        "data":t["layers"]})
exact = all(total[g]==ITEMS[g]["count"] for g in ITEMS)
ok = ok and exact
return {"ok":ok,"stage":"packed","exact":exact,"total":total,"
    trucks":truck_results,"cnt":len(truck_results),"total_cost":
    total_cost}

def get_bases_cv(group, h):
    dims, cat = DIMS[group], CAT[group]; out = []
    if cat == "dir":
        vdim = VDIM.get(group)
        if vdim:
            vi = {"L":0,"W":1,"H":2}[vdim]
            if abs(dims[vi]-h) > 1e-9: return []

```

```

        base = [dims[j] for j in range(3) if j != vi]
        out.append((float(base[0]),float(base[1]))); out.append((
            float(base[1]),float(base[0])))
    else:
        for i,dh in enumerate(dims):
            if abs(dh-h) > 1e-9: continue
            base = [dims[j] for j in range(3) if j != i]
            out.append((float(base[0]),float(base[1]))); out.append((
                float(base[1]),float(base[0])))
    seen = set(); uniq = []
    for x in out:
        if x not in seen: seen.add(x); uniq.append(x)
    return uniq

def caps_cv(group, h, L, W):
    out = []
    for l,w in get_bases_cv(group, h):
        c = int(L//l)*int(W//w)
        if c > 0: out.append({"l":l,"w":w,"cap":c})
    out.sort(key=lambda x:(x["cap"],-(x["l"]*x["w"])), reverse=True)
    return out

def choose_ori(group, h, L, W, need):
    cands = caps_cv(group, h, L, W)
    if not cands: raise ValueError(f"No orientation for {group} h={h}")
    feasible = [c for c in cands if c["cap"]>=need]
    if feasible:
        feasible.sort(key=lambda c:(c["cap"]-need,-c["cap"],c["l"]*c["w"]
            ))
        return feasible[0]["l"], feasible[0]["w"], feasible[0]["cap"]
    return cands[0]["l"], cands[0]["w"], cands[0]["cap"]

def place(group, cnt, h, x0, y0, L, W, z, start):
    if cnt <= 0: return []
    l,w,cap = choose_ori(group, h, L, W, cnt)
    n_x = int(L//l); n_y = int(W//w); rows = []; idx = start; placed = 0
    for iy in range(n_y):
        for ix in range(n_x):
            if placed >= cnt: break
            rows.append({"id":f"{group}_{idx:05d}", "group":group, "cat":
                CAT[group],
                "x":x0+ix*l, "y":y0+iy*w, "z":z, "l":l, "w":w, "h":
                    h, "weight":WEIGHT[group]})
            placed += 1
            idx += 1

```

```

        idx += 1; placed += 1
    if placed >= cnt: break
return rows

def parse_regs(layer, t_type=2):
    tk = TRUCKS[t_type]
    nonzero = [g for g in GROUPS if int(layer.get(g,0))>0]
    if len(nonzero) <= 1: return [{"group":nonzero[0],"x0":0.0,"y0":0.0,"L":tk["L"],"W":tk["W"]}]]
    note = str(layer.get("note","") or ""); fam = str(layer.get("fam","") or "")
    m = re.search(r"([A-Z0-9]+)\|([A-Z0-9]+)\|(\d+)", note)
    if m:
        g1,g2,cut = m.group(1),m.group(2),float(m.group(3))
        if "v1" in fam: return [{"group":g1,"x0":0.0,"y0":0.0,"L":cut,"W":tk["W"]},{ "group":g2,"x0":cut,"y0":0.0,"L":tk["L"]-cut,"W":tk["W"]}]]
        if "h1" in fam: return [{"group":g1,"x0":0.0,"y0":0.0,"L":tk["L"],"W":cut},{ "group":g2,"x0":0.0,"y0":cut,"L":tk["L"],"W":tk["W"]-cut}]
    m = re.search(r"\((.+)\)\|(\d+),(\d+)", note)
    if m:
        pat_text,c1,c2 = m.group(1),float(m.group(2)),float(m.group(3))
        pat = ast.literal_eval(f"({pat_text})")
        if "v2" in fam:
            widths = [c1,c2-c1,tk["L"]-c2]; x = 0.0; regs = []
            for g,width in zip(pat,widths): regs.append({"group":g,"x0":x,"y0":0.0,"L":width,"W":tk["W"]}]); x += width
            return regs
        if "h2" in fam:
            heights = [c1,c2-c1,tk["W"]-c2]; y = 0.0; regs = []
            for g,hh in zip(pat,heights): regs.append({"group":g,"x0":0.0,"y0":y,"L":tk["L"],"W":hh}); y += hh
            return regs
    total = sum(int(layer[g]) for g in nonzero); x = 0.0; regs = []
    for i,g in enumerate(nonzero):
        frac = int(layer[g])/max(total,1)
        width = tk["L"]-x if i==len(nonzero)-1 else round(tk["L"]*frac,3)
        regs.append({"group":g,"x0":x,"y0":0.0,"L":width,"W":tk["W"]}]);
        x += width
    return regs

def alloc(group, total, h, regs):

```

```

if total <= 0: return [0]*len(regs)
cs = []
for reg in regs:
    cands = caps_cv(group, h, float(reg["L"]), float(reg["W"]))
    cs.append(cands[0]["cap"] if cands else 0)
if sum(cs) < total: raise ValueError(f"Capacity too small for {
    group}")
order = sorted(range(len(regs)), key=lambda i:cs[i], reverse=True)
assigned = [0]*len(regs); remain = total
for idx in order:
    take = min(remain, cs[idx]); assigned[idx] = take; remain -=
        take
    if remain <= 0: break
return assigned

def convert(result_json, output_dir):
    with result_json.open(encoding="utf-8") as f: data = json.load(f)
    output_dir.mkdir(parents=True, exist_ok=True)
    counter = {g:1 for g in GROUPS}; csvs = []
    for truck in data["trucks"]:
        t_type = truck.get("type", 2); tk = TRUCKS[t_type]
        rows = []; z = 0.0
        for layer in truck["data"]:
            h = float(layer["h"]); regs = parse_regs(layer, t_type)
            g2idx = {}
            for idx,reg in enumerate(regs): g2idx.setdefault(reg["group
                "],[]).append(idx)
            assigned = [0]*len(regs)
            for g,idxs in g2idx.items():
                need = int(layer.get(g,0))
                if need <= 0: continue
                sub_regs = [regs[i] for i in idxs]; sub_assigned =
                    alloc(g, need, h, sub_regs)
                for loc,glb in enumerate(idxs): assigned[glb] =
                    sub_assigned[loc]
            for idx,reg in enumerate(regs):
                g = reg["group"]; cnt = assigned[idx]
                if cnt <= 0: continue
                placed = place(g, cnt, h, float(reg["x0"]), float(reg["
                    y0"]), float(reg["L"]), float(reg["W"]), z, counter[
                        g])
                rows.extend(placed); counter[g] += cnt
            z += h

```

```

    out_csv = output_dir / f"truck_{int(truck['idx'])}" / "
        placements.csv"
    out_csv.parent.mkdir(parents=True, exist_ok=True)
    with out_csv.open("w", encoding="utf-8-sig", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["id", "group", "cat", "
            x", "y", "z", "l", "w", "h", "weight"])
        writer.writeheader(); writer.writerows(rows)
    csvs.append(str(out_csv))
    with (output_dir / "overview.json").open("w", encoding="utf-8") as
    f:
        json.dump({"cnt":len(csvs),"csvs":csvs}, f, ensure_ascii=False,
            indent=2)
    return {"cnt":len(csvs)}

def main():
    output_dir = Path(__file__).parent / "outputs"
    output_dir.mkdir(parents=True, exist_ok=True)
    print("=" * 60)
    print("最小化运输成本求解 (车型1+车型2)")
    print("=" * 60)

    pair_dfs, pair_sols = [], []
    for gA, gB in PAIRS:
        df = pair_opts(gA, gB); df.insert(0, "gA", gA); df.insert(1, "gB",
            gB)
        pair_dfs.append(df); pair_sols.append(solve_pair(gA, gB, df))
    all_df = pd.concat(pair_dfs, ignore_index=True)
    top_df = all_df.sort_values(["obj", "util", "weight"], ascending=False
        ).head(250).copy()
    g3_best = best_g3()
    layer_set = build_layers(pair_sols, g3_best)
    pack = assign_trucks(layer_set)

    all_df.to_csv(output_dir/"candidates_all.csv", index=False,
        encoding="utf-8-sig")
    top_df.to_csv(output_dir/"candidates_top.csv", index=False,
        encoding="utf-8-sig")
    save_json(output_dir/"overview.json", {"pair_sols":pair_sols, "
        g3_best":g3_best,
            "pack":{"ok":pack["ok"],"cnt":pack.get("cnt"),"total_cost"
                ":pack.get("total_cost")}})
    save_json(output_dir/"result.json", pack)

    print(f"求解成功 = {pack['ok']}")

```

```
print(f"车辆数 = {pack.get('cnt',0)}")
print(f"总成本 = {pack.get('total_cost',0)} 元")

print("\n转换placements.csv...")
convert(output_dir/"result.json", output_dir/"placements")
print("完成")

if __name__ == "__main__":
    main()
```